

Scalable Clustering of Modern Networks

Dissertation

Presented in Partial Fulfillment of the Requirements for the Degree
Doctor of Philosophy in the Graduate School of The Ohio State
University

By

Venu M. Satuluri, B.Tech.

Graduate Program in Department of Computer Science and Engineering

The Ohio State University

2012

Dissertation Committee:

Srinivasan Parthasarathy, Advisor

Gagan Agrawal

Eric Fosler-Lussier

© Copyright by

Venu M. Satuluri

2012

Abstract

Graphs (or networks) provide a simple yet powerful model for capturing the interactions or relationships between the entities in many different domains. This dissertation focusses on a fundamental analytical or data management primitive related to graphs - the discovery of natural groups or clusters from a graph, or graph clustering in short. Due to the representational versatility of graphs, this problem has varied applications - the discovery of protein complexes from protein interaction networks, data partitioning for distributed computing, community discovery in online social networks, general data clustering (via similarity-weighted graphs), image segmentation, optimizing transistor layout for VLSI design and so on.

The increasing scale, complexity and diversity of modern (graph) data has rendered classical solutions for this task, such as spectral clustering, either very slow, incomplete or inaccurate. I address this situation in this thesis by approaching the graph clustering problem from two directions - in one direction, I develop algorithms for the core clustering task; in the second direction, I build intelligent pre-processing strategies that transform the graph to enable fast and accurate clustering subsequently. Such a two-pronged strategy allows us to concentrate on the novel aspects of new variants of the clustering problem, while exploiting existing solutions where we can. In the direction of algorithms for the core clustering task, my contribution to the state-of-the-art is a novel algorithm, MLR-MCL, based on the multi-level simulation of stochastic flows. MLR-MCL is a significantly enhanced version of a popular existing algorithm called MCL, and is fast, accurate, noise-tolerant and allows easy adjustment of the balance in the output cluster sizes. In the direction of intelligent pre-processing strategies, I have made three important contributions: (i) Graph sparsification algorithms that clarify the cluster structure and thereby speed-up subsequent clustering, by upto 50x; (ii) Algorithms that symmetrize directed graphs into similarity-weighted, undirected graphs suitable for subsequent clustering; (iii) Novel hashing-based algorithms that efficiently convert general non-graph data into similarity-weighted graphs suitable for clustering.

Acknowledgments

Let me begin by thanking my advisor, Dr. Srinivasan Parthasarathy (Srini), for his support, material and non-material, over all these years. I have had my share of disappointments, especially in my early years, and Srini's encouragement and frank advice have been invaluable for me on all of those occasions. When it comes to helping out his students, Srini frequently goes beyond what is expected of an advisor - be it actively scouting for internship leads on the student's behalf, burning the midnight oil writing papers, or minutely dissecting conference presentations. And, needless to say, all of this research is a fruit of the many, neverending discussions we have had, which I will miss in the future.

I am grateful to the National Science Foundation for supporting my research through grants IIS-0347662, RI-CNS-0403342, CCF-0702587, IIS-0742999, IIS-0917070 and IIS-1141828. Any opinions, findings, and conclusions or recommendations expressed here are those of the author and, if applicable, his adviser and collaborators, and do not necessarily reflect the views of the National Science Foundation.

I am also thankful to Dr. Rajeev Rastogi, who took me on for an internship in Yahoo! Labs Bangalore when I was still unproven, and exposing me to interesting problems as well as people. My former labmate Greg Buehrer also deserves thanks for taking me on board for an internship in beautiful Bellevue, and letting me work on some great problems with real data.

The members of the lab have been a great part of my experience. In the order of the amount of time I have shared the lab with them, they are : Xintian, Shirish, Matt, Ye, Sitaram, Duygu, Yiye, Faisal, Yu-Keng, Yang, Dave, Greg, Chao, Amol and Keith. Thank you, all of you, for the memorable dinners, 888 discussions, snack sharing after trips back home, help dealing with latex's idiosyncracies, and everything else.

I would also like to thank the many faculty in our department, interacting with whom was molded me. Thanks to Dr. Gagan Agrawal, Dr. Eric Fosler-Lussier, and

Dr. Mark Berliner for serving on my thesis committee. I am also thankful to Dr. P. Sadayappan (Saday), Dr. Mikhail Belkin and Dr. Luis Rademacher with whom I have had some interesting discussions. A special thanks to Arnab Nandi for helping me out on my job search. Many thanks to the department's administrative staff - Tamera, Tom, Lynn, Catrena, Cary, Don - for making it all work smoothly.

A special thanks to Jeremy Morris and Dr. Fosler for helping me out when I taught *Intro to Artificial Intelligence* – that was an experience I will always cherish. I am also thankful to Srini, Dr. Bruce Weide and Kitty Reeves for giving me that opportunity.

My Ph.D. life would not have been much fun without my room-mates¹ and friends over the years. Karthik, Vivek, Jatin, Sugosh, Vijay, Jay, “Bhaya”, Kirti, Srikanth, Chris, Sriram, Sonali, Sowmya and the others - I will always cherish memories of all the inane jokes, pedantic debates, “cooking turn” arguments, Diwali parties, “potluck” dinners, RPAC sessions and everything else.

I cannot thank my family - Amma, Nannagaru and Annayya - enough for being supportive all these years. They have always encouraged me to aim high and have instilled in me the basic values without which I would be lost.

Finally, to my wife, Sravya - you bring nothing but good luck and happiness to my life. I hope I can do the same for you.

¹excluding the bed-bugs

Vita

Sep 2001 - May 2005	B.Tech. Computer Science National Institute of Technology Karnataka (NITK) Surathkal, India
Jul 2005 - Jun 2006	Software Engineer D E Shaw India Software Hyderabad, India
Sep 2006 - Mar 2012	Ph.D. student Dept. of Computer Science and Engineering The Ohio State University
Sep 2006 - Aug 2007	University Fellow, The Ohio State University
Jul 2008 - Sep 2008	Research Intern, Yahoo! Labs, Bangalore, India
Apr 2010 - Jun 2010	Research Intern, Microsoft, Redmond

Publications

Research Publications

V. Satuluri and S. Parthasarathy “Bayesian Locality Sensitive Hashing for Fast Similarity Search”. In *PVLDB 5(5):430-441, 2012*.

V. Satuluri, S. Parthasarathy and Y. Ruan “Local Graph Sparsification for Scalable Clustering”. In *Proc. 37th ACM Conf. on Management of Data (SIGMOD), 2011*.

V. Satuluri and S. Parthasarathy “Symmetrizations for Clustering Directed Graphs.” In *Proc. 14th Int’l Conf. on Extending Database Technology (EDBT)*, 2011

V. Satuluri, S. Parthasarathy and D. Ucar “Markov Clustering of Protein Interaction Network with Improved Balance and Scalability.” In *Proc. 1st ACM Conf. on Bioinformatics and Computational Biology (ACM-BCB)* 2010

M. Kshirsagar, R. Rastogi, S. Satpal, S. Sangamedu and V. Satuluri “High-Precision Web Extraction Using Site Knowledge” In *Proc. 16th Int’l Conf. on Management of Data (COMAD)* 2010

V. Satuluri and S. Parthasarathy “Scalable Graph Clustering using Stochastic Flows: Applications to Community Discovery”. In *Proc. 15th ACM SIGKDD Intl. Conference on Knowledge Discovery and Data Mining, KDD ’09*, pages 737-746.

C. Wang, V. Satuluri and S. Parthasarathy “Local probabilistic models for link prediction.” In *Proc. 7th IEEE Int’l Conf. on Data Mining (ICDM)* 2007

Fields of Study

Major Field: Computer Science and Engineering

Table of Contents

	Page
Abstract	ii
Acknowledgments	iii
Vita	v
List of Tables	xi
List of Figures	xiii
1. Introduction	1
1.1 Graph clustering	2
1.1.1 Example application	6
1.2 Limitations of existing work	7
1.3 Thesis Statement	8
1.4 Contributions and Organization of the Thesis	9
2. Background and Related Work	13
2.1 Graph Clustering: definitions and current algorithms	13
2.1.1 Quality Functions	14
2.1.2 Algorithms for community discovery	17
2.2 Review of Sparsification and Sampling approaches	24
3. Multi-level Regularized MCL: Graph Clustering using Stochastic Flows	27
3.1 Preliminaries	29
3.1.1 Stochastic matrices and flows	29
3.1.2 Markov Clustering (MCL) Algorithm	30
3.1.3 Toy example	32

3.1.4	Limitations of MCL	33
3.2	Our Algorithms	35
3.2.1	Regularized MCL (R-MCL)	35
3.2.2	Multi-level Regularized MCL (MLR-MCL)	38
3.2.3	MCL in a Multi-level framework?	42
3.2.4	Discussion of MLR-MCL	42
3.3	Proof of Theorem 1	44
3.4	Experiments	47
3.4.1	Evaluation criteria	47
3.4.2	Comparison with MCL	49
3.4.3	Comparison with Graclus and Metis	49
3.4.4	Clustering PPI networks: A Case Study	50
3.5	Conclusion	55
4.	Graph Clustering with Adjustable Balance	56
4.1	Regularized MCL with adjustable balance	58
4.1.1	Effect of the balance parameter b	59
4.2	Results on PPI networks	61
4.2.1	Datasets	61
4.2.2	Experimental Setup	62
4.2.3	Quality and balance comparison between MCL and MLR-MCL	62
4.2.4	Speed comparison between MLR-MCL and MCL	64
4.2.5	Effect of varying balance parameter b	64
4.2.6	Comparison With Metis	69
4.3	Results on Other Graphs	69
4.3.1	Synthetic graphs	69
4.3.2	Results on Wikipedia	70
4.3.3	Results on clustering text documents	71
4.4	Conclusion	74
5.	Pre-processing graphs using Local Graph Sparsification	75
5.1	Similarity sparsification	78
5.1.1	Minwise Hashing for Fast Similarity	83
5.2	Empirical Evaluation	86
5.2.1	Datasets	86
5.2.2	Baselines	87
5.2.3	Evaluation method	88
5.2.4	Results	90
5.2.5	Examining L-Spar sparsification in depth	92
5.2.6	Results on Randomized version of L-Spar	98

5.3	Discussion	99
5.4	Conclusion	100
6.	Symmetrizations for Directed Graphs	101
6.1	Prior work	104
6.1.1	Normalized cuts for directed graphs	104
6.1.2	Bibliographic coupling and co-citation matrices	106
6.2	Graph symmetrizations	107
6.2.1	$A + A^T$	108
6.2.2	Random walk symmetrization	108
6.2.3	Bibliometric symmetrization	109
6.2.4	Degree-discounted symmetrization	110
6.2.5	Pruning the symmetrized matrix	112
6.2.6	Complexity analysis	113
6.3	Experimental Setup	114
6.3.1	Datasets	114
6.3.2	Setup	115
6.3.3	Evaluation method	116
6.4	Results	116
6.4.1	Characteristics of symmetrized graphs	116
6.4.2	Results on Cora	116
6.4.3	Results on Wikipedia	120
6.4.4	Results on Livejournal and Flickr	122
6.4.5	Effect of varying α and β	123
6.4.6	Significance of obtained improvements	124
6.4.7	A case study of Wikipedia clusters	124
6.4.8	Top-weight edges in Wikipedia symmetrizations	126
6.5	Conclusion	127
7.	Bayesian Locality Sensitive Hashing for Fast Nearest Neighbors	128
7.1	Background	132
7.2	Classical similarity estimation for LSH	134
7.2.1	Difficulty of tuning the number of hashes	134
7.2.2	Ignores the potential for early pruning	136
7.3	Candidate pruning and similarity estimation using BayesLSH	136
7.3.1	BayesLSH for Jaccard similarity	140
7.3.2	BayesLSH for Cosine similarity	143
7.3.3	Optimizations	146
7.3.4	The Influence of Prior vs. Data	147
7.4	Experiments	149

7.4.1	Experimental setup	150
7.4.2	Results comparing BayesLSH variants with baselines	152
7.4.3	Effect of varying parameters of BayesLSH	159
7.5	Conclusions	162
8.	Conclusions and Future Work	163
8.1	Future work	167
8.1.1	Parallel algorithms for large-scale data	167
8.1.2	Novel data models and clustering variants	168
8.1.3	Dimensionality Reduction, Similarity Search and Clustering	170
	Bibliography	172

List of Tables

Table	Page
3.1 Details of real datasets	47
3.2 Comparison of MLR-MCL, R-MCL and MCL.	48
4.1 PPI networks used in the experiments	62
4.2 Quality of clustering 20 newsgroups dataset	73
5.1 Dataset details.	86
5.2 Quality and speedups after sparsification.	93
5.3 Examples of retained and discarded edges using L-Spar sparsification	94
6.1 Details of the datasets	114
6.2 Details of symmetrized graphs.	117
6.3 Effect of varying pruning threshold	122
6.4 Effect of varying α, β (Metis). The best results are indicated in bold.	123
6.5 Edges with highest weights for different symmetrizations on Wiki	126
7.1 Dataset details. Nnz stands for number of non-zeros.	150
7.2 Comparison of fastest BayesLSH variant with baselines.	152
7.3 Recalls for AP+BayesLSH and AP+BayesLSH-Lite.	155

7.4	Percentage of similarity estimates with errors greater than 0.05	155
7.5	The effect of varying the parameters γ, δ, ϵ . WikiWords100K, $t=0.7$. .	157

List of Figures

Figure	Page
1.1 Examples of graphs/networks	3
1.2 Examples of graphs/networks	4
1.3 Clustering a toy graph into four clusters	5
1.4 Social network of Zachary’s Karate Club [124], clustered using R-MCL.	7
1.5 A schematic of the thesis contributions	9
3.1 Toy example graph for illustrating MCL.	32
3.2 A high-level overview of Multi-level Regularized MCL.	38
3.3 Comparison of Avg. N-Cut scores between MLR-MCL and MCL. . .	48
3.4 Comparison of Avg N-Cut scores: MLR-MCL, Graclus and Metis. . .	51
3.5 Comparison of timing between MLR-MCL, Graclus and Metis. . . .	52
3.6 Significance of discovered protein clusters.	54
4.1 (a) Quality and (b) Balance, on Yeast PPI (DIP)	65
4.2 (a) Quality and (b) Balance, on Yeast PPI data from Biogrid	65
4.3 (a) Quality and (b) Balance, on Human PPI data	66
4.4 (a) Quality and (b) Balance, for varying b on Yeast PPI data from DIP	66

4.5	(a) Quality and (b) Balance, for varying b on Yeast PPI (Biogrid)	67
4.6	Timing comparison on (a) Yeast PPI (Biogrid) and (b) Human PPI	67
4.7	(a) Quality and (b) Balance, MLR-MCL vs Metis,(BioGRID)	68
4.8	(a) Timing and (b) Quality, on Synthetic graphs	70
4.9	(a) Timing and (b) Quality, on Wikipedia	71
4.10	Importance of balance, Wikipedia	72
5.1	Proposed method on an example graph with 30 vertices and 3 clusters.	76
5.2	Global vs. local sparsifications	80
5.3	The performance of L-Spar under varying conditions	96
5.4	The performance of L-Spar under varying conditions	97
6.1	Toy example illustrating limitations of prior work.	106
6.2	Schematic of our framework	108
6.3	Scenarios illustrating the intuition behind degree-discounting.	111
6.4	Distributions of node degrees for different symmetrizations of Wiki.	117
6.5	Quality comparisons on Cora using (a) MLR-MCL and (b) Graclus	117
6.6	Degree-discounted vs BestWCut [86] (a) Effectiveness (b) Speed.	118
6.7	Quality Comparisons on Wiki using (a) MLR-MCL and (b) Metis.	118
6.8	Clustering times on Wiki using (a) MLR-MCL (b) Metis	119
6.9	Clustering times using MLR-MCL on (a) Flickr and (b) LiveJournal	119
6.10	Wiki subgraph of plant species of the genus <i>Guzmania</i>	125
7.1	Hashes vs. similarity	135

7.2	Different priors converge to similar posteriors	148
7.3	Timing comparisons between different algorithms.	153
7.4	Timing comparisons between different algorithms (binary datasets).	154
7.5	The pruning power of BayesLSH.	156
7.6	LSH+BayesLSH - varying γ, δ, ϵ on WikiWords100K, $t=0.7$	160

Chapter 1: Introduction

The inexorable march of information technologies - computing clouds, superior communication networks, ubiquitous computing devices, cheaper storage hardware, Web 2.0 applications etc. - has facilitated a culture of large-scale data collection followed by analysis, in a wide variety of domains. The World Wide Web today is dominated by companies such as Google and Amazon which have risen to prominence due, not in insignificant part, to their superior ability to collect, analyze and derive value out of data. Similarly, many areas of science - including biology, astronomy, ocean sciences, and the social sciences - have been noted as entering a paradigm of “data-intensive scientific discovery” [1]. The challenge for Computer Scientists here, however, is that of developing *fast* as well as *intelligent* algorithms for deriving *useful knowledge* from such data. The field of *Data Mining* has arisen in response to this challenge, synthesizing relevant insights from Databases, Machine Learning, Statistics, Algorithms and High Performance Systems.

Of the different abstractions using which data can be represented and analyzed as a part of the Data Mining process, the *graph* (or *network*) is a highly flexible and widely used abstraction. Simply stated, a graph consists of a set of *nodes*, with an accompanying set of *edges* that connect pairs of nodes. Complex systems can be modeled using graphs by using nodes to represent the entities in the system, and edges to represent the inter-relationships among the entities. Each edge may be associated with a *weight*, a positive real number indicating the strength of the relationship being modeled. The edges may also be *directed*, indicating that the relationship is asymmetrical. Some examples of domains or systems that are amenable to representation as graphs include:

- Networks of web pages, with web pages as nodes, and hyperlinks as edges (see Figure 1.1 (b)).
- Networks of research papers, with papers as nodes and citations as edges.

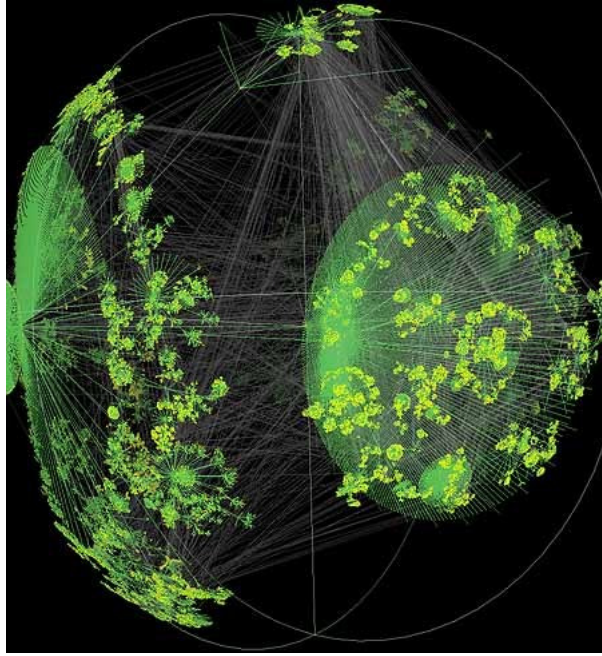
- Social networks, with nodes used to model individuals, and edges used to model different relationships, such as friendship, biological relatedness, workplace collaboration, scientific co-authorship etc. (see Figure 1.2 (a))
- Biological networks, such as protein-protein interaction (PPI) networks and regulatory networks. In PPI networks, the nodes represent proteins and edges indicate that the corresponding proteins interacted as part of executing a biological process or function. In regulatory networks, the nodes represent genes and an edge indicates that one of the corresponding genes was regulated by the other.
- Communication networks, where the nodes may be routers or nodes in a sensor network and edges represent communication links between them (see Figure 1.1 (a)).
- Similarity graphs, which may be used to represent similarity between entities in any domain. Two kinds of similarity graphs are worth stressing: (i) *k-nearest neighbor graphs*, where each object is connected to the k most similar objects in the domain; and (ii) *ϵ -nearest neighbor graphs*, where each object is connected to all other objects with which it has similarity $\geq \epsilon$.

The wide-spread applicability of the graph representation allows us to design algorithms for many different domains at once by designing algorithms on graphs. Furthermore, working with graphs allows us to capitalize on the large amount of previous research that has been conducted on them [31].

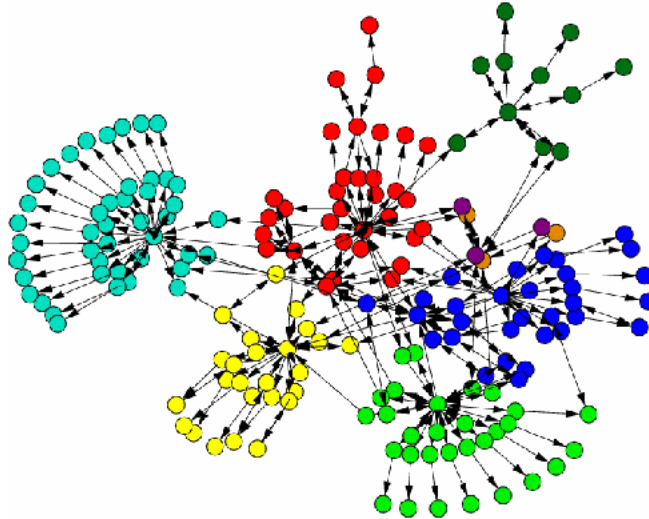
1.1 Graph clustering

An important problem for data represented in the form of graphs is: how do we discover groups of nodes in the graph that are well-connected amongst themselves but are weakly connected to the rest of the graph? This problem is generally referred to as *graph clustering*. (Note that this problem is different from the problem of clustering sets of graphs, where the objective is to find similar groups of graphs themselves, rather than similar groups of nodes within a single graph.) This problem has numerous applications in diverse domains, some of which we list below:

- **Community detection in social networks:** Human society is fundamentally organized in groups: families, companies, neighborhoods, villages etc.,

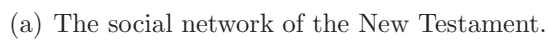


(a) The Internet graph, ca 2001. The nodes represent IP addresses and the edges represent communication links. Produced by the Cooperative Association for Internet Data Analysis (CAIDA)



(b) A directed graph consisting of the pages of a website and their hyperlinks [89]

Figure 1.1: Examples of graphs/networks



4

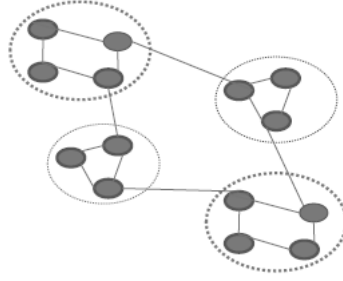


Figure 1.3: Clustering a toy graph into four clusters

and this group organization is often reflected in the various networks of social interactions. The rise of online social networks such as Facebook, as well as greater ease of preparing other social networks such as scientific collaboration networks, allows us the chance to analyze human social networks at a larger scale and in different contexts. Graph clustering helps discover the communities in such networks and is an important tool for understanding their structure and organization.

- **Community detection in biological networks:** Biological entities such as proteins and genes often work in groups in order to achieve a common goal. Recently, many high-throughput experimental methodologies have been developed that allow us to detect interactions between proteins on a large scale [46, 116]. These interactions can be modeled as graphs and graph clustering can be subsequently applied in order to discover protein (or gene) complexes.
- **Clustering applications in the World Wide Web:** Online retailers may wish to cluster their customers according to their tastes or other attributes, in order to both understand their customer base better as well as to set up efficient recommendation systems. Web search engines, which depend on revenue from advertisers bidding on keywords, may wish to cluster the advertiser-keyword graph so as to recommend new keywords for advertisers to bid on.
- **Data clustering using similarity graphs:** Notwithstanding the flexibility of the graph model, the data from a large number of domains may not be directly

representable as a graph. In such situations, traditional data clustering represents each data object as a vector in some multi-dimensional space, and groups the data points using a distance metric intrinsic to the space. An alternative, arguably more general, approach completely avoids representing the objects as vectors, and instead only defines a relevant notion of similarity between the objects. The data is then represented as either a k -nearest neighbor or ϵ -nearest neighbor graph, with objects represented as nodes and similarities between objects used to assign weights to the edges. This graph can then be clustered to reveal the clusters in the data. Examples include clustering of text documents, clustering of images etc.

- **Decomposition of large VLSI units:** Modern VLSI units are extremely large, consisting of millions of units, making it very hard for them to be handled by design and analysis tools. Traditionally, such large units have been manually partitioned into smaller units for analysis in relative isolation, but this approach becomes increasingly untenable as the size and complexity of VLSI chips increases. One can instead use graph clustering for this problem, by representing such large VLSI units as graphs, with nodes as units and edges representing the strength of interaction between the units.

Other applications include data partitioning for parallel computing, circuit partitioning for VLSI design, image segmentation and many others.

1.1.1 Example application

We illustrate the utility of graph clustering using a classic social network among members of a karate club compiled by Zachary [124]. The nodeset of this network represents the 34 members of the club, and edges connect members who were observed to interact outside club activities, over a period of three years. The network is visualized in Figure 1.4, with the nodes colored according to the cluster they were grouped into by Regularized Markov Clustering (R-MCL), a graph clustering algorithm that will be discussed in Chapter 3. A conflict between the club president and instructor led to the fission of the club into two separate groups, one supporting the instructor and the other supporting the president. The composition of the two resulting groups matches well with the two clusters that are found by clustering the social network, indicating the real-world applicability of graph clustering.

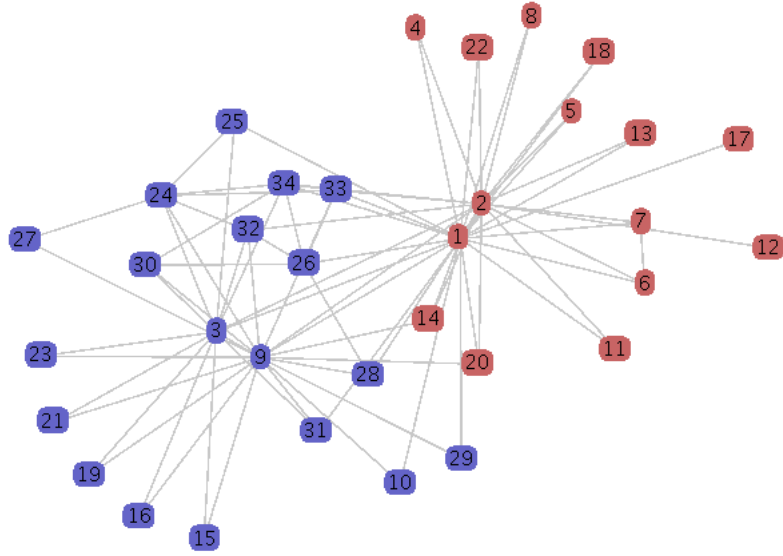


Figure 1.4: Social network of Zachary’s Karate Club [124], clustered using R-MCL.

1.2 Limitations of existing work

Despite the significant amount of research that has been conducted on the graph clustering problem (see [48] and [102] for recent surveys on the topic), the nature of real-world instances of this problem has been changing quickly, leaving the state-of-the-art behind.

The first main stumbling block for many well-known graph clustering algorithms is the large scale of modern graph datasets. Spectral clustering algorithms relying on eigenvector computations [104, 26], the original Markov Clustering algorithm [41], betweenness-centrality based edge removal algorithms [89] are examples of popular graph clustering algorithms which typically do not scale beyond graphs with thousands of nodes (on a single processor). Leaving aside web-scale graphs with billions of nodes, this means that even curated versions of the Wikipedia article-article graph and social networks such as Livejournal [88] with millions of nodes, are beyond the reach of these algorithms.

The second challenge posed by modern graphs is the presence of certain structural characteristics leading to heavy imbalance in the sizes of the output clusters. A recent empirical study [74] found that optimizing well-known objectives such as conductance [74] and modularity [89] often results in the graph being clustered into a giant core surrounded by smaller whiskers. In general, existing clustering algorithms rarely allow users to adjust the balance in output cluster sizes, instead either looking for strictly balanced clusterings, (e.g. Metis [63]) or purely optimizing cluster quality as measured by objective functions (e.g. spectral algorithms [104, 26, 5]) leading to heavily imbalanced clusterings.

A third limitation of current work is that modern graphs rarely come in the undirected, uni-modal format, which is the assumed input for many graph clustering algorithms. Many graphs are at least directed - Web graphs, citation networks, who-follows-whom social networks (such as Twitter) - and many contain additional information apart from link structure, e.g. nodes and edges may be associated with text or other attributes. The existing work on clustering directed graphs [86, 126] is both very slow as well as misses out on important classes of clusters in directed graphs. Similarly, modern graphs often contain multiple overlapping clustering arrangements, yet the existing methods for discovering such structure [92, 55] leave a lot to be desired in terms of scalability and ability to work on top of any standard graph clustering algorithm.

A fourth limitation of existing work is the lack of efficient algorithms for preparing nearest neighbor graphs from general non-graph data such as text documents or images, or indeed even graphs themselves when the edges do not directly indicate similarity. This drawback is particularly striking in the case of more complex representations (which generally tend to lead to more meaningful knowledge discovery), such as representation of a document as a vector of tf-idf scores instead of as just a bag of words. Both exact state-of-the-art methods such as AllPairs [12] as well as approximate methods such as Locality Sensitive Hashing [59, 20] have proved to be very expensive.

1.3 Thesis Statement

The discovery of clusters from modern graph data can be done effectively and efficiently by using a combination of pre-processing algorithms that clarify the local

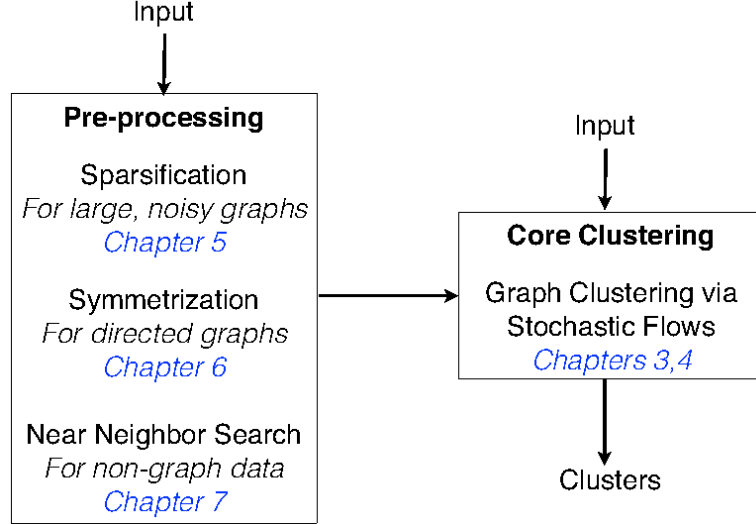


Figure 1.5: A schematic of the thesis contributions

similarity structure of the input data and clustering algorithms based on multi-level simulations of stochastic flows.

1.4 Contributions and Organization of the Thesis

This thesis presents several methods that contribute towards solving the graph clustering problem. Our contributions can be understood as approaching the graph clustering problem from two directions: in one direction, our methods directly solve the graph clustering problem; in another direction, our methods serve as pre-processing methods that clarify the local similarity structure so that the resultant graph can be efficiently and accurately clustered subsequently. A schematic representing this two-pronged approach is depicted in Figure 1.5.

The advantage with such a modular approach is that our pre-processing methods can be used not just for clustering, but also for other purposes, thereby increasing their overall applicability. Secondly, as the field continues to make advances in solving these problems, the obsolescence of any one method (either the pre-processing or the clustering itself) need not impact the utility of the other methods, since the methods make no assumptions about the internal implementations of the other methods. We

would also like to note at this point that there are many applications where directly applying the core clustering algorithms proposed in this thesis works well, and the pre-processing algorithms have been developed in order to enable better performance for certain (important) classes of problems. This is one reason why, in the organization of the material, we have given precedence to describing the core clustering algorithms first, even though looking at Figure 1.5 the ordering may look backwards. The second reason for this ordering is that the algorithms developed for core clustering are an important ingredient in the experiments showing the effectiveness of the pre-processing.

The main contributions of our thesis, divided according to whether they are meant for core clustering or pre-processing for clustering, are as follows:

Core Clustering algorithms:

In the first two chapters that cover our contributions, we discuss algorithms that solve the core task of clustering an input graph.

In Chapter 3, we identify weaknesses in the popular Markov Clustering Algorithm (MCL), namely its slowness and a tendency to fragment clusters. We propose Regularized MCL (R-MCL), a simple modification of MCL, which produces more coarse-grained clusters. We also describe Multi-level Regularized MCL (MLR-MCL) which uses a multi-level strategy to improve the quality (by incorporating the global topology of the graph) as well as the speed (by executing on smaller graphs first). MLR-MCL scales to graphs with million plus nodes, while MCL does not scale beyond graphs with tens of thousands of nodes. MLR-MCL is also shown to be faster than Metis and Graclus on million-plus node graphs. On a protein-protein interaction network, the clusters found by MLR-MCL were found to be more biologically significant than those found by either Graclus or MCL.

In Chapter 4, we propose a variation on the R-MCL process that allows the user to tune the balance of the output clustering arrangement. This is important in applications where the goal is to obtain relatively balanced clusters (such as protein complex discovery from PPI networks or data partitioning for parallel computing), or for applications where the natural tendency of MLR-MCL is to output an excessively unbalanced clustering arrangement. We show that for three different PPI networks, balanced MLR-MCL discovers more accurate protein complexes in comparison with both Graclus and MCL. We also show results on clustering k-nearest neighbor graphs

of text documents and show that balanced MLR-MCL again outperforms other baseline clustering algorithms, and traditional data clustering algorithms such as spherical K-Means.

Pre-processing algorithms for subsequent clustering:

In the next three chapters, we discuss different algorithms that perform pre-processing and ultimately output undirected, possibly similarity weighted graphs which can be clustered using off-the-shelf clustering algorithms. A common thread that connects the different pre-processing algorithms is that they all can be understood as clarifying the similarity structure of the input data at the local level i.e. for each object or node. Performing this effectively can make the subsequent clustering much more accurate and/or fast.

In Chapter 6, we show how to cluster directed graphs using a two-stage framework: in the first stage different symmetrizations may be used to obtain an undirected graph, which is clustered in the second stage using an (undirected) graph clustering algorithm, such as MLR-MCL or Graclus. We show how previous work on clustering directed graphs fits in our framework. We carefully analyze the weaknesses of existing symmetrizations and propose a novel *Degree-discounted Symmetrization* algorithm that incorporates the degrees of the nodes into account (crucial for robust performance on power-law graphs). Clustering our symmetrized graph yields performance improvements of 22% over a state-of-the-art directed spectral clustering algorithm on a research paper citation network and 12% improvement on a directed graph of Wikipedia pages. Moreover, the undirected graph clustering times reduce 2-4 times typically on the large-scale graphs.

In Chapter 5, we propose a simple pre-processing of input graphs where we discard a majority of the edges in the graph as a way to clarify the cluster structure of the graph. This pre-processing step is particularly effective for noisy, dense and large graphs, and can help a subsequent clustering algorithm extract clusters with greater accuracy and speed. We tested this approach for a variety of datasets from domains such as online social networks, information networks and biological networks, and found that the pre-processing typically enables 5x-50x speedups while also retaining or even improving the accuracy.

In Chapter 7, we investigate in depth the problem of similarity search or nearest neighbor search, which is a necessary step for converting non-graph data into a

similarity-weighted graph that is suitable for subsequent clustering. Our main contributions here extend upon Locality Sensitive Hashing [8], a popular randomized approach for indexing and generating candidates for nearest neighbors. We show how a simple Bayesian framework can help us prune candidates generated by Locality Sensitive Hashing in a principled and extremely fast fashion, enabling speedups of 2x-20x over existing approaches on a variety of datasets.

Conclusions and Future Work:

The long-term goal of the research in this dissertation is the scalable discovery of structure with minimal supervision for diverse, complex domains and fields such as the World Wide Web, modern social science and computational biology. The understanding of the structure that characterizes a domain is helpful both to human experts, who can use this knowledge to formulate novel hypotheses (as happens regularly in Biology), and to computer algorithms, which can use it to solve even bigger tasks (as happens regularly in A.I.). The next steps in achieving this long-term vision are outlined in Chapter 8.

Before we get to our contributions, which start from Chapter 3, we first cover some background and related work on the topics of this thesis in the next Chapter.

Chapter 2: Background and Related Work

In this chapter, we will first review common definitions of network communities in the literature, as well as some popular algorithms for graph clustering or community discovery. Subsequently, we will also review prior work on graph sparsification and sampling that is related to the our own algorithms for graph sparsification discussed in Chapter 5. For the symmetrization and near neighbor search, we provide a discussion of the related work together with the discussion of our own contributions in the relevant chapters (6 and 7 respectively).

2.1 Graph Clustering: definitions and current algorithms

Informally, a cluster/community in a graph/network is a group of nodes with greater ties internally than to the rest of the network. This intuitive definition has been formalized in a number of competing ways, usually by way of a quality function. Such quality functions, rather than giving a binary decision as to whether a group of nodes qualifies as a community, instead output a number quantifying the “quality” of the community. This computation of quality relies on a particular formalization of the intuition that good communities have more internal than external edges i.e. they measure quality in a way that using purely the graph connectivity. We review the popular quality functions in Section 2.1.1. In this thesis, in addition to evaluating our approaches using such internal quality measures, we also evaluate on the basis of some external ground truth, as well as by providing qualitative examples of the output where appropriate.

Algorithms for community discovery vary on a number of important dimensions, including their approach to the problem as well as their performance characteristics. An important dimension on which algorithms vary in their approaches is whether or not they explicitly optimize a specific quality metric. Spectral methods [104, 91], the

Kernighan-Lin algorithm [64] and flow-based postprocessing [72] are all examples of algorithms which explicitly try to optimize a specific quality metric, while other algorithms, such as Markov Clustering (MCL) [41] and clustering via shingling [51] do not do so. Another dimension on which algorithms vary is in how (or even whether) they let the user control the granularity of the division of the network into communities. Some algorithms (such as spectral methods) are mainly meant for bi-partitioning the network, but this can be used to recursively subdivide the network into as many communities as desired. Other algorithms such as agglomerative clustering or MCL allow the user to indirectly control the granularity of the output communities through certain parameters. Still other algorithms, such as certain algorithms optimizing the Modularity function [29], do not allow (or require) the user to control the output number of communities at all. Another important characteristic differentiating community discovery algorithms is the importance they attach to a balanced division of the network - while metrics such as the KL objective function explicitly encourage balanced division, other metrics capture balance only implicitly or not at all. Coming to performance characteristics, algorithms also vary in their scalability to big networks, with multi-level clustering algorithms such as Metis [63] and Graclus [36] scaling better than many other approaches. We review some of the more popular graph clustering algorithms in Section 2.1.2.

2.1.1 Quality Functions

A variety of quality functions or measures have been proposed in the literature to capture the goodness of a division of a graph into clusters. In what follows, A denotes the adjacency matrix of the network or graph, with $A(i, j)$ representing the edge weight or affinity between nodes i and j , and V denotes the vertex or node set of the graph or network.

The normalized cut of a group of vertices $S \subset V$ is defined as [104, 87]

$$Ncut(S) = \frac{\sum_{i \in S, j \in \bar{S}} A(i, j)}{\sum_{i \in S} degree(i)} + \frac{\sum_{i \in S, j \in \bar{S}} A(i, j)}{\sum_{j \in \bar{S}} degree(j)} \quad (2.1)$$

In words, the normalized cut of a group of nodes S is the sum of weights of the edges that connect S to the rest of the graph, normalized by the total edge weight of S and that of the rest of the graph $\bar{S}$. Intuitively, groups with low normalized cut make for

good communities, as they are well connected amongst themselves but are sparsely connected to the rest of the graph.

The *conductance* of a group of vertices $S \subset V$ is closely related and is defined as [61]

$$\text{Conductance}(S) = \frac{\sum_{i \in S, j \in \bar{S}} A(i, j)}{\min(\sum_{i \in S} \text{degree}(i), \sum_{i \in \bar{S}} \text{degree}(i))} \quad (2.2)$$

The normalized cut (or conductance) of a division of the graph into k clusters $V_1, \dots, V_k$ is the sum of the normalized cuts (or conductances) of each of the clusters $V_i \{i = 1, \dots, k\}$ [36].

The *Kernighan-Lin (KL) objective* looks to minimize the edge cut (or the sum of the inter-cluster edge weights) under the constraint that all clusters be of the same size (making the simplifying assumption that the size of the network is a multiple of the number of clusters):

$$KL\text{Obj}(V_1, \dots, V_k) = \sum_{i \neq j} A(V_i, V_j) \text{ subject to } |V_1| = |V_2| = \dots = |V_k| \quad (2.3)$$

Here $A(V_i, V_j)$ denotes the sum of edge affinities between vertices in V_i and V_j , i.e. $A(V_i, V_j) = \sum_{u \in V_i, v \in V_j} A(u, v)$

Modularity [89] has recently become quite popular as a way to measure the goodness of a clustering of a graph. One of the advantages of modularity is that it is independent of the number of clusters that the graph is divided into. The intuition behind the definition of modularity is that the farther the subgraph corresponding to each community is from a random subgraph (i.e. the null model), the better or more significant the discovered community structure is. The modularity Q for a division of the graph into k clusters $\{V_1, \dots, V_k\}$ is given by:

$$Q = \sum_{c=1}^k \left[\frac{A(V_i, V_i)}{m} - \left(\frac{\text{degree}(V_i)}{2m} \right)^2 \right] \quad (2.4)$$

In the above, the V_i s are the clusters, m is the number of edges in the graph and $\text{degree}(V_i)$ is the total degree of the cluster V_i . For each cluster, we take the difference between the fraction of edges internal to that cluster and the fraction of edges that would be expected to be inside a random cluster with the same total degree.

Optimizing any of these objective functions is NP-hard [50, 104, 19].

External quality measures The above quality functions all measure the quality of a given clustering solely with respect to the connectivity structure of the input graph itself. However, domain experts who are merely trying to solve their own problem using graph clustering, may not care about such internal quality, and rather may care about the extent of match between the predicted set of clusters and some independently derive set of “ground truth” clusters. For example, in Bioinformatics, biologists may wish to evaluate how well a certain clustering algorithm succeeds in retrieving clusters from a protein interaction network that are closest to the set of protein complexes independently identified by biologists in the lab using other means. Similarly, in clustering text documents, one may wish to check how well the clusters output by an algorithm match with a human annotated set of labels for the documents.

In such a case, we use *Weighted average F-scores* to quantify the closeness of the output clusters and ground truth clusters. Let the output clustering be $\mathcal{C} = \{C_1, C_2, \dots, C_i, \dots, C_k\}$, and let the ground truth clustering be $\mathcal{G} = \{G_1, G_2, \dots, G_j, \dots, G_n\}$. Each cluster in both $\mathcal{C}$ and $\mathcal{G}$ is nothing but a set of elements. For any output cluster C_i , the precision and recall of this cluster against a given ground truth category, say G_j , are defined as:

$$Prec(C_i, G_j) = \frac{|C_i \cap G_j|}{|C_i|}$$

and

$$Rec(C_i, G_j) = \frac{|C_i \cap G_j|}{|G_j|}$$

High precision may be achieved simply by generating very small clusters (which are much more likely to be contained within a ground truth cluster and therefore achieve high precision); while high recall may be achieved by doing the opposite i.e. large clusters (which are likely to contain ground truth clusters in their entirety, thus leading to high recall). For this reason, both precision and recall need to be taken into account simultaneously in evaluating the goodness of a cluster with respect to a ground truth clustering. This is achieved by taking their harmonic mean (the harmonic mean of two numbers is more biased towards the lower of the two numbers than either the geometric or the arithmetic means), which is referred to as the F-score. The F-score $F(C_i, G_j)$ is the harmonic mean of the precision and the recall. We match each output cluster C_i with the ground truth cluster G_j for which $F(C_i, G_j)$ is the highest among all ground truth clusters. This is the F-score that is subsequently associated with

this cluster, and is referred to as $F(C_i)$; i.e.

$$F(C_i) = \max_j F(C_i, G_j)$$

The average F-score of the entire clustering $\mathcal{C}$ is defined as the average of the F-scores of all the clusters, weighted by their sizes.

$$Avg.F(\mathcal{C}) = \frac{\sum_i |C_i| * F(C_i)}{\sum_i |C_i|}$$

2.1.2 Algorithms for community discovery

The Kernighan-Lin(KL) algorithm

The KL algorithm [64] is one of the classic graph partitioning algorithms which optimizes the KL objective function i.e. minimize the edge cut while keeping the cluster sizes balanced (see Equation 2.3). The algorithm is iterative in nature and starts with an initial bipartition of the graph. At each iteration, the algorithm searches for a subset of vertices from each part of the graph such that swapping them will lead to a reduction in the edge cut. The identification of such subsets is via a greedy procedure. The *gain* g_v of a vertex v is the reduction in edge-cut if vertex v is moved from its current partition to the other partition. The KL algorithm repeatedly selects from the larger partition the vertex with the largest gain and moves it to the other partition; a vertex is not considered for moving again if it has already been moved in the current iteration. After a vertex has been moved, the gains for its neighboring vertices will be updated in order to reflect the new assignment of vertices to partitions. While each iteration in the original KL algorithm [64] had a complexity of $O(|E| \log |E|)$, Fiduccia and Mattheyses improved it to $O(|E|)$ per iteration using appropriate data structures. This algorithm can be extended to multi-way partitions by improving each pair of partitions in the multi-way partition in the above described way.

Agglomerative/Divisive Algorithms

Agglomerative algorithms begin with each node in the social network in its own community, and at each step merge communities that are deemed to be sufficiently similar, continuing until either the desired number of communities is obtained or the

remaining communities are found to be too dissimilar to merge any further. Divisive algorithms operate in reverse; they begin with the entire network as one community, and at each step, choose a certain community and split it into two parts. Both kinds of hierarchical clustering algorithms often output a *dendrogram* which is a binary tree, where the leaves are nodes of the network, and each internal node is a community. In the case of divisive algorithms, a parent-child relationship indicates that the community represented by the parent node was divided to obtain the communities represented by the child nodes. In the case of agglomerative algorithms, a parent-child relationship in the dendrogram indicates that the communities represented by the child nodes were agglomerated (or merged) to obtain the community represented by the parent node.

Girvan and Newman’s divisive algorithm: Newman and Girvan [89] proposed a divisive algorithm for community discovery, using ideas of *edge betweenness*. Edge betweenness measures are defined in a way that edges with high betweenness scores are more likely to be the edges that connect different communities. That is, inter-community edges are designed to have higher edge betweenness scores than intra-community edges do. Hence, by identifying and discarding such edges with high betweenness scores, one can disconnect the social network into its constituent communities.

Shortest path betweenness is one example of an edge betweenness measure: the intuitive idea here is that since there will only be a few inter-community edges, shortest paths between nodes that belong to different communities will be constrained to pass through those few inter-community edges.

The general form of their algorithms is as follows:

1. Calculate betweenness score for all edges in the network using any measure.
2. Find the edge with the highest score and remove it from the network.
3. Recalculate betweenness for all remaining edges.
4. Repeat from step 2.

The above procedure is continued until a sufficiently small number of communities are obtained, and a hierarchical nesting of the communities is also obtained as a natural by-product. The motivation for the recalculation step is as follows: if the edge

betweenness scores are only calculated once and edges are then removed by the decreasing order of scores, these scores won't get updated and no longer reflect the new network structure after edge removals. Therefore, recalculation is in fact the most critical step in the algorithm to achieve satisfactory results. The main disadvantage of this approach is the high computational cost: simply computing the betweenness for all edges takes $O(|V||E|)$ time, and the entire algorithm requires $O(|V|^3)$ time.

Newman's greedy optimization of modularity: Newman [90] proposed a greedy agglomerative clustering algorithm for optimizing modularity. The basic idea of the algorithm is that at each stage, groups of vertices are successively merged to form larger communities such that the modularity of the resulting division of the network increases after each merge. At the start, each node in the network is in its own community, and at each step one chooses the two communities whose merger leads to the biggest increase in the modularity. We only need to consider those communities which share at least one edge, since merging communities which do not share any edges cannot result in an increase in modularity - hence this step takes $O(|E|)$ time. An additional data structure which maintains the fraction of shared edges between each pair of communities in the current partition is also maintained, and updating this data structure takes worst-case $O(|V|)$ time. There are a total of $|V| - 1$ iterations (i.e. mergers), hence the algorithm requires $O(|V|^2)$ time. Clauset et al. [29] later improved the complexity of this algorithm by the use of efficient data structures such as max-heaps, with the final complexity coming to $O(|E|d \log |V|)$, where d is the depth of the dendrogram describing the successive partitions found during the execution of the algorithm.

Spectral Algorithms

Spectral algorithms are among the classic methods for clustering and community discovery. Spectral methods generally refer to algorithms that assign nodes to communities based on the eigenvectors of matrices, such as the adjacency matrix of the network itself or other related matrices. The top k eigenvectors define an embedding of the nodes of the network as points in a k -dimensional space, and one can subsequently use classical data clustering techniques such as K-means clustering to derive the final assignment of nodes to clusters [119]. The main idea behind spectral clustering is that the low-dimensional representation, induced by the top eigenvectors, exposes

the cluster structure in the original graph with greater clarity. From an alternative perspective, spectral clustering can be shown to solve real relaxations of different weighted graph cut problems, including the normalized cut defined above [119, 104].

The main matrix that is used in spectral clustering applications is the Laplacian matrix $\mathcal{L}$. If A is the adjacency matrix of the network, and D is the diagonal matrix with the degrees of the nodes along the diagonal, then the unnormalized Laplacian L is given as $L = D - A$. The Laplacian (or the normalized Laplacian) $\mathcal{L}$ is given by $\mathcal{L} = D^{-1/2}(D - A)D^{-1/2} = I - D^{-1/2}AD^{-1/2}$. It can be verified that both L and $\mathcal{L}$ are symmetric and positive definite, and therefore have real and positive eigenvalues [27, 119]. The Laplacian has 0 as an eigenvalue with multiplicity equal to the number of connected components in the graph. The eigenvector corresponding to the smallest non-zero eigenvalue of $\mathcal{L}$ is known as the Fiedler vector [45], and usually forms the basis for bi-partitioning the graph.

The main disadvantage of spectral algorithms lies in their computational complexity. Most modern implementations for eigenvector computation use iterative algorithms such as the Lanczos algorithm, where at each stage a series of matrix vector multiplications are performed to obtain successive approximations to the eigenvector currently being computed. The complexity for computing the top eigenvector is $O(kM(m))$, where k is the number of matrix-vector multiplications and $M(m)$ is the complexity of each such multiplication, dependent primarily on the number of non-zeros m in the matrix. k depends on the specific properties of the matrix at hand - such as the spectral gap i.e. the difference between the current eigenvalue and the next eigenvalue; the smaller this gap, the more number of matrix-vector multiplications are required for convergence. In practice, spectral clustering is hard to scale up to networks with more than tens of thousands of vertices without employing parallel algorithms.

Dhillon et al. [36] showed that the weighted cut measures such as normalized cut that are often optimized using spectral clustering can also be optimized using an equivalent weighted kernel k-means algorithm. This is the core idea behind their algorithm **Grclus**, which can cluster graphs at a comparable quality to spectral clustering without paying the same computational cost, since k-means is much faster compared to eigenvector computation.

Multi-level Graph Partitioning

Multi-level methods provide a powerful framework for fast and high-quality graph partitioning, and in fact have been used for solving a variety of other problems as well [112]. The main idea here is to shrink or coarsen the input graph successively so as to obtain a small graph, partition this small graph and then successively project this partition back up to the original graph, refining the partition at each step along the way. Multi-level graph partitioning methods include multi-level spectral clustering [11], **Metis** (which optimizes the KL objective function) [63] and **Graclus** (which optimizes normalized cuts and other weighted cuts) [36]

The main components of a multi-level graph partitioning strategy are:

1. **Coarsening.** The goal here is to produce a smaller graph that is similar to the original graph. This step may be applied repeatedly to obtain a graph that is small enough to be partitioned quickly and with high-quality. A popular coarsening strategy is to first construct a *matching* on the graph, where a matching is defined as a set of edges no two of which are incident on the same vertex. For each edge in the matching, the vertices at the ends of the edge are collapsed together and are represented by a single node in the coarsened graph. Coarsening can be performed very quickly using simple randomized strategies [63].
2. **Initial partitioning.** In this step, a partitioning of the coarsest graph is performed. Since the graph at this stage is small enough, one may use strategies like spectral partitioning which are slow but are known to give high quality partitions.
3. **Uncoarsening.** In this phase, the partition on the current graph is used to initialize a partition on the finer (bigger) graph. The finer connectivity structure of the graph revealed by the uncoarsening is used to refine the partition, usually by performing local search. This step is continued until we arrive at the original input graph.

At a finer level, Metis uses a variant of the KL algorithm in its uncoarsening phase to refine the partition obtained from previous steps. Graclus, on the other hand, uses weighted kernel k-means for refining the partition.

Markov Clustering

Stijn van Dongen’s Markov Clustering algorithm (MCL) clusters graphs via manipulation of the stochastic matrix or transition probability matrix corresponding to the graph [41]. In what follows, the transition probability between two nodes is also referred to as *stochastic flow*. The MCL process consists of two operations on stochastic matrices, *Expand* and *Inflate*. $\text{Expand}(M)$ is simply $M * M$, and $\text{Inflate}(M, r)$ raises each entry in the matrix M to the inflation parameter r (> 1 , and typically set to 2) followed by re-normalizing the columns to sum to 1. These two operators are applied in alternation iteratively until convergence, starting with the initial transition probability matrix.

We study MCL in greater detail in Chapter 3, since our own contributions build on top of MCL.

Other Approaches

Local Graph Clustering: A *local algorithm* is one that finds a solution containing or near a given vertex (or vertices) without looking at the whole graph. Local algorithms are interesting in the context of large graphs since their time complexity depends on the size of the solution rather than the size of the graph to a large extent. (Although if the clusters need to cover the whole graph, then it is not possible to be independent of the size of the graph.) The main intuition is that random walks simulated from inside a group of internally well-connected nodes will not mix well enough/soon enough, as the cluster boundary acts a bottleneck that prevents the probability from seeping out of the cluster easily. Low-probability vertices are removed at each step to keep the complexity of the algorithm tractable.

Spielman and Teng [108, 107] described the first such local clustering algorithm using random walks. Let $p_{t,v}$ be the probability distribution of the t -step random walk starting at v . ($p_{t,v}$ is truncated i.e. low probability entries are set to zero, in order to avoid exploring too much of the graph.) For each t , let π be the permutation on the vertices of the graph that indicates the sorted order of the degree-normalized probabilities i.e.

$$\frac{p_t(\pi(i))}{d(\pi(i))} \geq \frac{p_t(\pi(i+1))}{d(\pi(i+1))} \quad (2.5)$$

The sweep sets $S_1^t, S_2^t, \dots, S_n^t$ are defined as $S_j^t = \{\pi(1), \dots, \pi(j)\}$. Let ψ_V be the final stationary distribution of the random walk (all random walks within a component

converge to the same stationary distribution.) The main theoretical result exploited says that the difference between $p^t(S_j^t)$ and $\psi_V(S_j^t)$ is either small, or there exists a cut with low conductance among the sweep sets. Therefore by checking the conductance of the sweep sets S_j^t at each time step t , we discover clusters of low conductance.

Andersen and Lang [7] extended this work to handle seed sets (instead of just a seed vertex). On real datasets such as web graph, IMDB graph etc. they select a random subset of nodes belonging to a known community and show that the local clustering approach is able to recover the original community.

Andersen et al. [5] improved upon Spielman and Teng’s algorithm by simulating random walks with restarts (i.e. Personalized PageRank), instead of just plain random walks. The notion of sweep sets for probability distributions, obtained by sorting the degree-normalized probabilities, is the same. The theoretical results here involve pagerank vectors though; if there is a set of vertices whose probability in the pagerank vector is significantly greater than their probability in the general stationary distribution, then some sweep set of the pagerank vector has low conductance. They show that they can compute an approximate page rank vector in time depending only on the error of the approximation and the truncation threshold (and not on the graph size). Once the approximate pagerank vector is computed, conductances of successive sweep sets are calculated to discover a set of vertices with low conductance.

Flow-Based Post-Processing for Improving Community Detection: We will discuss how algorithms for computing the maximum flow in flow networks can be used to post-process or improve existing partitions of the graph. Flake et al. [47] proposed to discover web communities by using a focused crawler to first obtain a coarse or approximate community and then set up a max-flow/min-cut problem whose solution can be used to obtain the actual set of pages that belong to the same community. Lang and Rao [72] discuss a strategy for improving the conductance of any arbitrary bipartition or cut of the graph. Given a cut of the graph $(S, \bar{S})$, their algorithm finds the best improvement among all cuts $(S', \bar{S}')$ such that S' is a strict subset of S . Their main approach is to construct a new instance of a max-flow problem, such that the solution to this problem (which can be found in polynomial time) can be used to find the set S' with the lowest conductance among all subsets of S . They refer to their method as MQI (Max-Flow Quotient-Cut Improvement). They use Metis+MQI to recursively bi-partition the input graph; at each step they bi-partition using Metis first and then improve the partition using MQI and repeat the process

on the individual partitions. Anderson and Lang [7] find that MQI can improve the partitions found by local clustering as well.

Community Discovery via Shingling: Broder et al. [21] introduced the idea of clustering web documents through the use of *shingles* and fingerprints (also denoted as *sketches*). In short, a length- s *shingle* is s of all parts of the object. For example, a length- s shingle of a graph node contains s outgoing links of the node; a length- s shingle of a document is a contiguous subsequence of length s of the document. Meanwhile, a *sketch* is a constant-size subset of all shingles with a specific length, with the remarkable property that the similarity between sets of two objects’ sketches approximates the similarity between the objects themselves (here the definition of similarity being used is Jaccard similarity, i.e. $\text{sim}(A, B) = |A \cap B| / |A \cup B|$). This property makes sketch an object’s fingerprint.

Gibson et al. [51] attempt to extract dense communities from large-scale graphs via a recursive application of shingling. In this algorithm, the first-level shingling is performed on each graph node using its outgoing links. That is, each node v is associated with a sketch of c_1 shingles, each of which stands for s_1 nodes selected from all nodes that v points to. Then an inverted index is built, containing each first-level shingle and a list of all nodes that the shingle is associated with. The second-level shingling function is then performed on each first-level shingle, producing second-level shingles (also called meta-shingles) and sketches. Two first-level shingles are considered as relevant if they share at least one meta-shingle in common, and the interpretation is that these two shingles are associated with some common nodes. If a new graph is constructed in such a way that nodes stand for first-level shingles and edges indicate the above-defined relation, then clusters of first-level shingles correspond to connect components in this new graph. Finally, communities can be extracted by mapping first-level shingles clusters back to original nodes plus including associated common meta-shingles. This algorithm is inherently applicable to both bipartite and directed graph, and can also be extended to the case of undirected graph. It is also very efficient in terms of both memory usage and running time, thus can handle graph of billions of edges.

2.2 Review of Sparsification and Sampling approaches

In this section, we will review prior literature that is related to the graph sparsification algorithms we propose later in this thesis.

Previous researchers have developed approaches for sparsifying graphs, but not always with the same goals as ours. In theoretical computer science, graph sparsification approaches that approximately preserve (unnormalized) edge cuts have been developed, such as random sampling [62], sampling in proportion to edge connectivities [15] and sampling in proportion to the effective resistance of an edge [107]. The last scheme - which has the best theoretical properties - favors the retention of edges connecting nodes with few short paths, as such edges have high effective resistance. However, this is quite different from our own approach, where instead we discard such edges, since they distract from the overall cluster structure of the graph. Furthermore, calculating effective resistances requires invoking a specific linear system solver [108], which runs in time at least $O(m \log^{15} n)$ - meaning that the overall approach is too time-consuming to be practical for large-scale graphs.²

The statistical physics community has studied the problems of *edge filtering* [115] and *complex network backbone detection* [53]. This line of work also does not aim to preserve the cluster structure, instead looking to identify the most interesting edges, or construct spanning-tree-like network backbones. Furthermore, these approaches are inapplicable in the case of unweighted networks.

There has also been work on *graph sampling* [66, 73] where the goal is to produce a graph with both fewer edges as well as fewer nodes than the original, and which preserves some properties of interest of the original graph such as the degree distribution, eigen-value distribution etc. The fact that these approaches select only a small subset of the nodes in the original graph makes them unsuitable to our task, since we aim to cluster the (entire) nodeset of the original graph. Recently, Maiya and Berger-Wolf [82] propose to find representative subgraphs that preserve community structure based on algorithms for building expander-like subgraphs of the original graph. The idea here is that sampling nodes with good expansion will tend to include representatives from the different communities in the graph. Since partitioning the sampled subgraph will only give us cluster labels for the nodes included in the subgraph, they propose to assign cluster labels to the unsampled nodes of the original graph by using collective inference algorithms [81]. However, collective inference itself utilizes the original graph in its entirety, and becomes a scalability bottleneck as one operates on bigger graphs.

² n and m denote the number of nodes and number of edges in the graph respectively.

Sparsifying matrices (which are equivalent representations for graphs) has also been studied [2, 10]. These approaches, in the absence of edge weights, boil down to random sampling of edges - an approach we evaluate in our experiments.

Chapter 3: Multi-level Regularized MCL: Graph Clustering using Stochastic Flows

In this chapter, we describe the graph clustering algorithms that are one of the main contributions of this thesis. We first study the Markov Clustering (MCL) algorithm, proposed by Stijn Van Dongen [41, 117], and based on a careful study of its weaknesses, design two novel variants of MCL, called Regularized Markov Clustering (R-MCL) and a multi-level variant of R-MCL, called Multi-level Regularized Markov Clustering (MLR-MCL).

Markov Clustering (MCL) [41, 117], is a graph clustering algorithm based on the simulation of stochastic flows or transition probabilities among vertices in a graph. MCL is a simple and elegant algorithm, and while not completely non-parametric, varying a simple parameter can result in clusterings of different granularities. The MCL algorithm has attained much popularity in the bioinformatics, where two independent studies [22, 118] comparing it with alternative graph clustering algorithms have found it to be much more robust to topological noise (a desirable property for a number of domains) and effective at discovering protein families from protein interaction networks. However, in spite of its popularity in the bioinformatics community for the above reasons, MCL has drawn limited attention from the data mining community primarily because it does not scale very well even to moderate sized graphs[24]. Additionally, the algorithm tends to fragment communities, a less than desirable feature in many situations.

We seek to develop an algorithm that retains the strengths of MCL while redressing its weaknesses. We first analyze the basic MCL algorithm carefully to understand the cause for these two limitations. We then identify a simple regularization step that can help alleviate the fragmentation problem. We call this algorithm Regularized-MCL (R-MCL). Subsequently we realize a scalable multi-level variant of R-MCL. The

basic idea of the multi-level procedure is to coarsen the graph (in a manner reminiscent of Metis [63]), run R-MCL on the coarsened graph, and then refine the graph in incremental steps. The central intuition is that using flow values derived from simulation on coarser graphs can lead to good initializations of flow in the refined graphs. Key to the refinement operation is a novel way to project flows such that the sanctity of the clustering algorithm is maintained. The multi-level approach also allows us to obtain large gains in speed. We refer to this algorithm as Multi-Level Regularized MCL (MLR-MCL).

In our empirical study we compare and contrast R-MCL and MLR-MCL with the original MCL algorithm[41], Graclus[36] and Metis[63] along the twin axes of scalability and quality on several real and synthetic datasets. Key highlights of our study include:

- **MLR-MCL vs R-MCL and MCL:** MLR-MCL typically outperforms both R-MCL and MCL in terms of quality and delivers performance that is roughly 2-3 orders of magnitude faster than MCL.
- **MLR-MCL vs. Graclus and Metis :** MLR-MCL delivers significant (10-15%) improvements in N-Cut values over Graclus in 4 of 6 datasets. We also typically outperform Graclus in terms of speed and are competitive with Metis.
- **MLR-MCL vs. Graclus and MCL on PPI networks:** The top 8 clusters of proteins found by MLR-MCL are rated better than the best cluster returned by either Graclus and MCL.

3.1 Preliminaries

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be our input graph with $\mathcal{V}$ and $\mathcal{E}$ denoting the node set and edge set respectively. Let A be the $|\mathcal{V}| \times |\mathcal{V}|$ adjacency matrix corresponding to the graph, with $A(i, j)$ denoting the weight of the edge between the vertex v_i and the vertex v_j . This weight can represent the strength of the interaction in the original network - e.g. in an author collaboration network, the edge weight between two authors could be the frequency of their collaboration. If the graph is unweighted, then the weight on each edge is fixed to 1. As many interaction networks are undirected, we also assume that $\mathcal{G}$ is undirected, although our method is easy to extend to directed graphs. Therefore A will be a symmetric matrix.

3.1.1 Stochastic matrices and flows

A column-stochastic matrix is simply a matrix where each column sums to 1. A column stochastic (square) matrix M with as many columns as vertices in a graph $\mathcal{G}$ can be interpreted as the matrix of the *transition probabilities* of a random walk (or a Markov chain) defined on the graph. The i^{th} column of M represents the transition probabilities *out* of the v_i ; therefore $M(j, i)$ represents the probability of a transition from vertex v_i to v_j . We use the terms stochastic matrix and column-stochastic matrix interchangeably.

We also refer to the transition probability from v_i to v_j as the *stochastic flow* or simply the *flow* from v_i to v_j . Correspondingly, a column-stochastic transition matrix of the graph $\mathcal{G}$ is also referred to as a flow matrix of $\mathcal{G}$ or simply a *flow of $\mathcal{G}$* . Given a flow matrix M , the i^{th} column contains the flows *out* of node v_i , or its *out-flows*; correspondingly the i^{th} row contains the *in-flows*. Note that while all the columns (or out-flows) sum to 1, the rows (or in-flows) are not required to do so.

The most common way of deriving a column-stochastic transition matrix M for a graph is to simply normalize the columns of the adjacency matrix to sum to 1

$$M(i, j) = \frac{A(i, j)}{\sum_{k=1}^n A(k, j)}$$

In matrix notation, $M := AD^{-1}$, where D is the diagonal degree matrix of $\mathcal{G}$ with $D(i, i) = \sum_{j=1}^n A(j, i)$. We will refer to this particular transition matrix for the graph

as the *canonical transition matrix* M_G . However, it is worth keeping in mind that one can associate other stochastic matrices with the graph $\mathcal{G}$.

Both MCL and our methods introduced in Section 3.2 can be thought of as simulating stochastic flows (or simulating random walks) on graphs according to certain rules. For this reason, we refer to these processes as *flow simulations*.

3.1.2 Markov Clustering (MCL) Algorithm

We next describe the Markov Clustering (MCL) algorithm for clustering graphs, proposed by Stijn van Dongen [41], in some detail as it is relevant to understanding our own method.

The MCL algorithm is an iterative process of applying two operators - *expansion* and *inflation* - on an initial stochastic matrix M , in alternation, until convergence. Both expansion and inflation are operators that map the space of column-stochastic matrices onto itself. Additionally, a *prune* step is performed at the end of each inflation step in order to save memory. Each of these steps is defined below:

Expand: Input M , output M_{exp} .

$$M_{exp} = Expand(M) \stackrel{def}{=} M * M$$

The i^{th} column of M_{exp} can be interpreted as the final distribution of a random walk of length 2 starting from vertex v_i , with the transition probabilities of the random walk given by M . One can take higher powers of M instead of a square (corresponding to longer random walks), but this gets computationally prohibitive very quickly.

Inflate: Input M and inflation parameter r , output M_{inf} .

$$M_{inf}(i, j) \stackrel{def}{=} \frac{M(i, j)^r}{\sum_{k=1}^n M(k, j)^r}$$

M_{inf} corresponds to raising each entry in the matrix M to the power r and then normalizing the columns to sum to 1. By default $r = 2$. Because the entries in the matrix are all guaranteed to be less than or equal to 1, this operator has the effect of exaggerating the inhomogeneity in each column (as long as $r > 1$). In other words, flow is strengthened where it is already strong and weakened where it is weak.

Prune: In each column, we remove those entries which have very small values (where “small” is defined in relation to the rest of the entries in the column), and the retained

entries are rescaled to have the column sum to 1. This step is primarily meant to reduce the number of non-zero entries in the matrix and hence save memory. We use the *threshold pruning* heuristic, where we compute a threshold based on the average and maximum values within a column, and prune entries below the threshold. [41]

Pseudo-code for MCL is presented in Algorithm 1. The addition of self-loops to the input graph avoids dependence of the flow distribution on the length of the random walk simulated so far, besides ensuring at least one non-zero entry per column.

Algorithm 1 MCL

```

 $A := A + I$  // Add self-loops to the graph
 $M := AD^{-1}$  // Initialize M as the canonical transition matrix

repeat
   $M := M_{exp} := \text{Expand}(M)$ 
   $M := M_{inf} := \text{Inflate}(M, r)$ 
   $M := \text{Prune}(M)$ 
until  $M$  converges

Interpret  $M$  as a clustering

```

Intuitively, the MCL process may be understood as expanding and contracting the flow in the graph alternately. The expansion step spreads the flow out of a vertex to potentially new vertices and also enhances the flow to those vertices which are reachable by multiple paths. This has the effect of enhancing within-cluster flows as there are more paths between two nodes that are in the same cluster than between those in different clusters. However, just applying the expansion step repeatedly will result in all the columns of M becoming equal to the principal eigenvector of the canonical transition matrix M_G . The inflation step is meant to prevent this from happening by introducing a non-linearity into the process, while also having the effect of strengthening intra-cluster flow and weakening inter-cluster flow. At the start of the process, the distribution of flows out of a node is relatively smooth and uniform; as more iterations are executed, the distribution becomes more and more peaked. *Crucially, all the nodes within a tightly-linked group of nodes will start to flow to one node within the group towards the end of the process.* This allows us to identify all the vertices that flow to the same “attractor” node as belonging to one cluster.

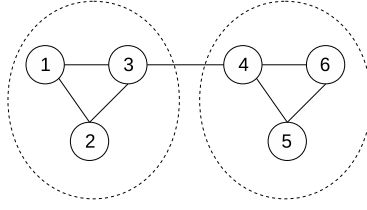


Figure 3.1: Toy example graph for illustrating MCL.

Interpretation of M as a clustering: As just mentioned, after some number of iterations, most of the nodes will find one “attractor” node to which all of their flow is directed i.e. there will be only one non-zero entry per column in the flow matrix M . We declare convergence at this stage, and assign nodes which flow into the same node as belonging to one cluster.

3.1.3 Toy example

We give a simple example of the MCL process in action for the graph in Figure 3.1.3. The initial stochastic matrix M_0 obtained by adding self-loops to the graph and normalizing each column is given below

$$M_0 = \begin{pmatrix} 0.33 & 0.33 & 0.25 & 0 & 0 & 0 \\ 0.33 & 0.33 & 0.25 & 0 & 0 & 0 \\ 0.33 & 0.33 & 0.25 & 0.25 & 0 & 0 \\ 0 & 0 & 0.25 & 0.25 & 0.33 & 0.33 \\ 0 & 0 & 0 & 0.25 & 0.33 & 0.33 \\ 0 & 0 & 0 & 0.25 & 0.33 & 0.33 \end{pmatrix}$$

The result of applying one iteration of Expansion, Inflation and the Prune steps is given below:

$$M_1 = \begin{pmatrix} 0.33 & 0.33 & 0.2763 & 0 & 0 & 0 \\ 0.33 & 0.33 & 0.2763 & 0 & 0 & 0 \\ 0.33 & 0.33 & 0.4475 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.4475 & 0.33 & 0.33 \\ 0 & 0 & 0 & 0.2763 & 0.33 & 0.33 \\ 0 & 0 & 0 & 0.2763 & 0.33 & 0.33 \end{pmatrix}$$

Note that the flow along the lone inter-cluster edge ($M_0(4,3)$) has evaporated to 0. Applying one more iteration results in convergence.

$$M_2 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Hence, vertices 1, 2 and 3 flow completely to vertex 3, where as the vertices 4, 5 and 6 flow completely to vertex 4. Hence, we group 1, 2 and 3 together with 3 being the “attractor” of the cluster, and similarly for 4, 5 and 6.

3.1.4 Limitations of MCL

The MCL algorithm is a simple and intuitive algorithm for clustering graphs that takes an approach that is different from that of the majority of other approaches to graph clustering such as spectral clustering [104, 36], divisive/agglomerative clustering [89], heuristic methods [64] and so on. Further more, it does not require a specification of the number of clusters to be returned; the coarseness of the clustering can instead be indirectly affected by varying the inflation parameter r , with lower values of r (upto 1) leading to coarser clusterings of the graph. MCL has received a lot of attention in the bioinformatics field, with multiple researchers finding it to be very effective at clustering biological interaction networks ([22, 77]).

However, there are two major limitations to MCL:

Lack of scalability: That MCL is slow has been noted by data mining researchers before ([43, 24]).

The Expand step: This step involves matrix multiplication, and is very time consuming in the first few iterations when many entries in the flow matrix have not been pruned out and is the main component of the overall running time. The time complexity of computing the square of a sparse matrix of size $|\mathcal{V}| \times |\mathcal{V}|$ is given by $O(\sum_{i=1}^{|\mathcal{V}|} k_i^2)$, where k_i indicates the number of non-zero elements in the i^{th} column. By this calculation, the Expand step in the first iteration of the algorithm in particular takes $O(\sum_{i=1}^{|\mathcal{V}|} d_i^2)$ operations, where d_i is the degree of vertex v_i , which is unacceptably slow for large graphs. Expansion steps in subsequent iterations take $O(\sum_{i=1}^{|\mathcal{V}|} k_i^2)$ time, where k_i is the number of non-zero entries in the i^{th} column. In the first few iterations before the flow matrix becomes sparse, k_i is typically in the range of hundreds or thousands for large graphs, still leading to an unacceptable time complexity.

Inflate and Prune steps: Each Inflate step requires two passes over the input matrix, and hence does not significantly contribute to the overall time complexity. The Prune step involves the computation of a threshold for each column and removing entries below the threshold and renormalizing. Heuristics for the computation of this threshold [41] require only the average and maximum values of the column, and hence requires only one pass over each column. Hence, the Prune step is also linear in the number of non-zero entries in the matrix.

Fragmentation of output: MCL tends to produce too many clusters. For example, on the yeast protein-protein interaction network of 4741 nodes, MCL outputs 1615 clusters. While it still manages to find some significant clusters (as evidenced by its success in bioinformatics [22, 118]), clearly such high fragmentation is undesirable. Theoretically the granularity of the clustering can be modified by varying the inflation parameter r , this has the undesirable side-effect of dramatically slowing down MCL (lower values of r imply slower convergence of the process, meaning more matrix multiplications) while also producing clusters with great imbalance. (We obtained similar results on other datasets, as well as with varying values of the inflation parameter r .)

3.2 Our Algorithms

We seek to develop an algorithm for graph clustering that retains the strengths of MCL while alleviating the weaknesses. We do this by first making a modification to the basic MCL process, resulting in an algorithm we call *Regularized MCL*, and then we embed this latter algorithm in a multi-level framework that further improves both the quality of the output and the speed of the algorithm.

A weight transformation step: Before discussing the algorithm, we first describe a weight transformation step that we apply to the input graphs. This step was suggested by Dhillon et al. [36], who use it as part of the coarsening process in their multi-level framework. Given the input adjacency matrix A and the degree diagonal matrix D , define the transformed adjacency matrix A^* as:

$$A^*(i, j) = \frac{A(i, j)}{D(i, i)} + \frac{A(i, j)}{D(j, j)}$$

The purpose of the above step is to downweight the edges involving high-degree (or hub) nodes, as they can have an undue influence on the clustering process.

3.2.1 Regularized MCL (R-MCL)

Why does MCL output so many clusters? One way of looking at the issue is to understand it as MCL allowing the columns of many pairs of neighbouring nodes in the flow matrix M to diverge significantly. This happens because the MCL algorithm uses the adjacency structure of the input graph only at the start, to initialize the flow matrix to the canonical transition matrix M_G . After that, the algorithm works only with the current flow matrix M , and there is nothing in the algorithm that prevents the columns of neighbouring nodes to differ widely without any penalty. This is what allows MCL to “overfit” the graph by outputting too many clusters.

We seek to address this issue by *regularizing* or *smoothing* the flow distributions out of a node w.r.t. its neighbors. Let $\mathbf{q}_i$, $i=1:k$, be the flow distributions of the k neighbors of a given node in the graph. (Each $\mathbf{q}_i$ is basically a column from the current flow matrix M .) Let w_i , $i=1:k$, be the respective normalized edge weights, i.e. $\sum_{i=1}^k w_i = 1$. Note that since we add self-loops, each node is also one of its own neighbors. We ask the following question: how do we update the flow distribution out of the node (call it $\mathbf{q}^*$) so that it is, in some sense, the least divergent from the flow

distributions of its neighbours? Following [114], we can formalize this requirement for $\mathbf{q}^*$ as:

$$\mathbf{q}^* = \arg \min_{\mathbf{q}} \sum_{i=1}^k w_i KL(\mathbf{q}_i || \mathbf{q}) \quad (3.1)$$

where $KL(\mathbf{p} || \mathbf{q})$ is the KL divergence between two probability distributions $\mathbf{p}$ and $\mathbf{q}$ - a commonly used divergence measure for probability distributions - defined as:

$$KL(\mathbf{p} || \mathbf{q}) = \sum_{x \in \text{sup}(\mathbf{p})} \mathbf{p}(x) \log \frac{\mathbf{p}(x)}{\mathbf{q}(x)}$$

where $\text{sup}(\mathbf{p})$ refers to the support of the distribution $\mathbf{p}$.

Proposition 1. *The penalty given by Equation 3.1 is minimized by the following solution for $\mathbf{q}^*$:*

$$\mathbf{q}^*(x) = \sum_{i=1}^k w_i \mathbf{q}_i(x) \quad (3.2)$$

Proof. Let $D(\mathbf{q}) = \sum_{i=1}^k w_i KL(\mathbf{q}_i || \mathbf{q})$ be the penalty for an arbitrary distribution $\mathbf{q}$. We want to minimize $D(\mathbf{q})$ enforcing the constraint that $\sum_x \mathbf{q}(x) = 1$. We use a Lagrange multiplier λ to enforce the constraint and minimize the quantity

$$\sum_{i=1}^k w_i \left(\sum_x \mathbf{q}_i(x) (\log \mathbf{q}_i(x) - \log \mathbf{q}(x)) \right) + \lambda \left(\sum_x \mathbf{q}(x) - 1 \right)$$

Differentiating the above w.r.t $\mathbf{q}(x)$ and equalling to 0 yields

$$\mathbf{q}(x) = \frac{\sum_{i=1}^k w_i \mathbf{q}_i(x)}{\lambda}$$

λ is obtained by using the constraint $\sum_x \mathbf{q}(x) = 1$.

$$\begin{aligned} \lambda &= \sum_x \sum_{i=1}^k w_i \mathbf{q}_i(x) \\ &= \sum_{i=1}^k w_i \sum_x \mathbf{q}_i(x) \\ &= \sum_{i=1}^k w_i \\ &= 1 \end{aligned}$$

The last step follows from our earlier assumption that the weights on the edges to the neighbors sum to 1, which is true for the columns of the canonical transition matrix M_G . Hence $D(\mathbf{q})$ has an extremum at $\mathbf{q}(x) = \sum_{i=1}^k w_i \mathbf{q}_i(x)$.

The Hessian matrix containing the second derivatives is diagonal with all the diagonal entries being positive, hence it is positive definite. Hence the above extremum corresponds to a minimum. $\square$

Hence, we replace the *Expand* operator in the MCL process with a new operator which updates the flow distribution of each node according as Equation 3.2. We call this the *Regularize* operator, and it can be conveniently expressed in matrix notation as right multiplication with the canonical transition matrix M_G of the graph.

$$\text{Regularize}(M) = M_{reg} \stackrel{\text{def}}{=} M * M_G$$

Pseudocode for Regularized MCL is given in Algorithm 2. The Inflate and Prune steps are the same as for MCL, and the interpretation of M as a clustering is also the same as has been described in Section 3.1.2.

Algorithm 2 Regularized MCL

```

A := A + I // Add self loops and transform weights
M := M_G := AD-1 // Initialize M as the canonical transition matrix
repeat
  M := Mreg := M * M_G
  M := Minf := Inflation(M, r)
  M := Prune(M)
until M converges
Interpret M as a clustering as described in Section 3.1.2

```

While, as we shall see in Section 3.4, Regularized MCL does produce fewer clusters of better quality, it still suffers from the scalability issues of the original MCL. We address this issue next and also discuss how the qualitative performance of Regularized MCL can be further improved.

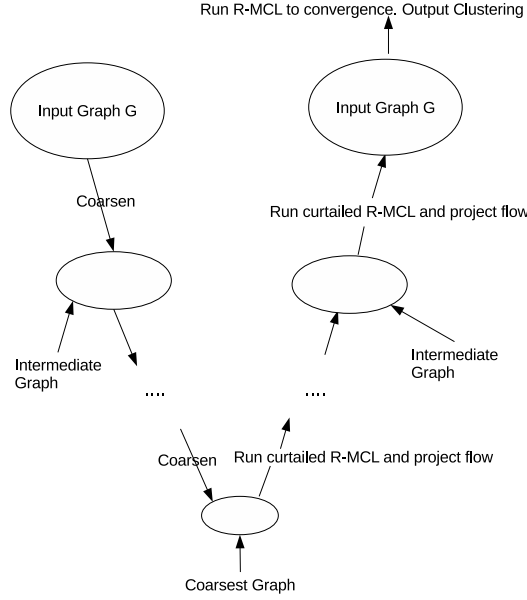


Figure 3.2: A high-level overview of Multi-level Regularized MCL.

3.2.2 Multi-level Regularized MCL (MLR-MCL)

We next explain a multi-level version of Regularized MCL, which we call Multi-level Regularized MCL, or MLR-MCL. The main intuition behind using a multi-level framework in our context is that the flow values resulting from simulation on the coarser graphs can be effectively used to initialize the flows for simulation on the bigger graphs. The algorithm also runs much faster because the initial few iterations, which are also the most time taking, are run on the smallest graphs first.

A schematic providing a high-level overview of the algorithm is given in Figure 3.2. MLR-MCL operates in three phases:

1. **Coarsening:** The input graph $\mathcal{G}$ is coarsened successively into a series of smaller graphs $\mathcal{G}_1, \mathcal{G}_2, \dots$ until we are left with a graph $\mathcal{G}_l$ of manageable size (a few hundred nodes typically). Each coarsening step consists of first constructing a *matching* on the graph, where a matching is defined as a set of edges no two of which are incident on the same vertex. The two vertices that are incident on each edge in the matching are collapsed to form a super-node, and the edges of

the super-node are the union of the edges of its constituent nodes. We use two arrays in each coarse graph, NodeMap1 and NodeMap2, to keep track of the coarsening. NodeMap1 maps a node in the coarse graph to its first constituent node; similarly NodeMap2 maps a node to its second constituent node (at most two nodes can be collapsed to one super node). We use a particular kind of matching known as *heavy edge matching*; both heavy edge matching and efficient randomized algorithms for constructing it are described elsewhere [63].

2. **Curtailed R-MCL along with refinement:** Beginning with the coarsest graph, R-MCL is run for a few iterations (typically 4 to 5). We refer to this abbreviated version of R-MCL as Curtailed R-MCL. The flow values at the end of this Curtailed R-MCL run are then projected on to the refined graph of the current graph as per Algorithm 4, and R-MCL is run again for a few iterations on the refined graph, and this is repeated until we reach the original graph.
3. **R-MCL on original graph:** With flow values initialized from the previous phase, R-MCL is run on the final graph until convergence. The flow matrix at the end is converted into a clustering in the usual way, with all the nodes that flow into the same “attractor” node being assigned into one cluster.

What is the intuition behind running R-MCL for only a few iterations on the coarse graphs from $\mathcal{G}_l$ down to $\mathcal{G}_1$? We do this as we do not want the flows in a coarse graph to converge; if we run R-MCL until convergence on one of the intermediate graphs, then the same cluster assignments will likely carry over till the original graph, thus not utilizing the additional adjacency information present in the bigger graphs. At the same time we want the flow values to capture some of the high-level cluster structure of the coarser graphs, and also want the flow matrix to be sparse enough to make running R-MCL on the bigger graph computationally tractable. So we strike a balance and run R-MCL for a small number of iterations. In practice, we have observed that running R-MCL for 4 to 5 iterations on the intermediate graphs gives good results.

The problem of flow projection: The remaining part of MLR-MCL is the algorithm for projecting flow from a coarse (smaller) graph to a refined (bigger) graph. Projection of flow is concerned with using the flow values of a smaller graph to provide a good initialization of the flow values in the bigger graph. It is not obvious at first sight how this should be done - since there are two nodes in the bigger graph

Algorithm 3 Multi-level Regularized MCL

Input: Original graph $\mathcal{G}$, Inflation parameter r , Size of coarsest graph c

```
// Phase 1: Coarsening
// Coarsen graph successively down to at most  $c$  nodes.
 $\{\mathcal{G}_0, \mathcal{G}_1, \dots, \mathcal{G}_l\} = \text{CoarsenGraph}(\mathcal{G}, c)$ 
//  $\mathcal{G}_0$  is the original graph and  $\mathcal{G}_l$  is the coarsest graph

// Phase 2: Curtailed R-MCL along with refinement
// Initialize  $M$  to canonical flow matrix of the coarsest graph  $\mathcal{G}_l$ 
 $M := M_G := A_l D_l^{-1}$ 

// Starting with the coarsest graph, iterate through successively refined graphs.
for  $i = l$  down to 1 do
    // Run R-MCL for a small number of iterations.
    for small number of iterations do
         $M := \text{Regularize}(M) = M * M_G$ 
         $M := \text{Inflate}(M, r)$ 
         $M := \text{Prune}(M)$ 
    end for

    // Project flow from the coarse graph  $\mathcal{G}_i$  onto the refined graph  $\mathcal{G}_{i-1}$ 
     $M := \text{ProjectFlow}(\mathcal{G}_i, M)$ 
    // Canonical transition matrix of the refined graph  $\mathcal{G}_{i-1}$ , for the next round
    of Curtailed R-MCL
     $M_G := A_{i-1} D_{i-1}^{-1}$ 
end for

// Phase 3: Run R-MCL on original graph until convergence
repeat
     $M := \text{Regularize}(M) = M * M_G$ 
     $M := \text{Inflate}(M)$ 
     $M := \text{Prune}(M)$ 
until  $M$  converges
Interpret  $M$  as a clustering as described in Section 3.1.2
```

Algorithm 4 ProjectFlow

Inputs: Coarse graph $\mathcal{G}_c$, Flow on the coarse graph M_c .

Output: Projected flow matrix on the refined graph M_r

NodeMap1 := $\mathcal{G}_c$.NodeMap1

NodeMap2 := $\mathcal{G}_c$.NodeMap2

for each non-zero entry (i,j) in M_c **do**

$M_r(\text{NodeMap1}(i), \text{NodeMap1}(j)) := M_c(i, j)$

$M_r(\text{NodeMap1}(i), \text{NodeMap2}(j)) := M_c(i, j)$

$M_r(\text{NodeMap2}(i), \text{NodeMap1}(j)) := 0$

$M_r(\text{NodeMap2}(i), \text{NodeMap2}(j)) := 0$

end for

return M_r

corresponding to each node in the smaller graph, a flow value between two nodes in the smaller graph must be used to derive the flow values between four pairs of nodes in the bigger graph. To look at it another way, if n_c is the size of the coarse graph, then the n_c^2 entries of the flow matrix of the coarse graph must be used to derive the $4 * n_c^2$ entries of the refined graph. How to do this?

Our solution: The naive strategy here is to assign the flow between two nodes in the refined graph as the flow between their respective parents in the coarse graph. However, this doubles the number of nodes that any node in the refined graph flows out to. This, combined with the fact that the out-flows of each node sum to 1, results in excessive smoothing of the out-flows of each node. (Recall that as the MCL process converges, the out-flow distribution of the nodes gets more and more peaked.) Hence, we instead choose only one child node for each parent node and project all the flow into the parent node to the chosen child node.

However, this raises the question: which child node do we pick in order to assign all the flow into? It turns out that it doesn't matter which child node we pick, as long as for each parent, the choice is consistent. We state this in Theorem 1, and it is proved in Section 3.3.

Theorem 1. *The MLR-MCL algorithm produces the same final clustering regardless of which child node is picked at each parent node to be assigned all its in-flows.*

For this reason, for each node v_i in the coarse graph, we arbitrarily pick the first child node $\text{NodeMap1}(i)$ and assign all the flow that was going into v_i to $\text{NodeMap1}(i)$.

While we treat the two child nodes asymmetrically when we are assigning the flows into them, the flows *out* of the two child nodes are assigned the same values. This being the case, can the algorithm treat these two nodes differently? Recall that the *Regularize* step utilizes the adjacency information in the graph by assigning a linear combination of the flows of a node’s neighbours as the flows of a node. *Hence, even if $\text{NodeMap1}(i)$ and $\text{NodeMap2}(i)$ start out with the same flows out of them, they will have different flows out of them after the Regularize step if they have different neighbours.* This ensures that the additional adjacency information that is present in the refined graph is used to re-adjust the flows of the nodes.

3.2.3 MCL in a Multi-level framework?

We have so far talked about how to embed R-MCL in a multi-level framework; is it possible to similarly embed MCL in a multi-level framework? Designing a refinement phase for MCL so that the adjacency information in the bigger graphs actually gets used does not prove to be easy. The hitch is that MCL does not use the adjacency information in the graph anywhere except at the initialization of the flow matrix. This means that when a node in the smaller graph is refined to its two constituent nodes, these two nodes will continue to have the exact same flow distribution from that point onwards. We have explored alternative ways of designing ProjectFlow to overcome this problem so as to take the new adjacency information in the bigger graph (i.e. the graph onto which the flow is being projected) into account, but none of the alternatives proved to be either theoretically satisfying or practically successful, and hence we do not discuss them further.

3.2.4 Discussion of MLR-MCL

We now discuss the time-complexity of MLR-MCL and the quality of its output. **Scalability and time-complexity:** The main component of the running time of R-MCL is the Regularize step which involves matrix multiplication (the Inflate and Prune steps are the same as in MCL and hence the analysis does not change). The Regularize step in the first iteration of R-MCL is the same as the Expand step in the first iteration of MCL (as M is initialized to M_G). Hence, the time complexity of the Regularize step in the first iteration is also $O(\sum_{i=1}^{|V|} d_i^2)$, similar to that for MCL. However, the time complexity of the subsequent Regularize steps is different. If the

number of non-zero entries in the i^{th} column of the flow matrix is k_i and the degree of v_i is d_i , then the time complexity of a Regularize step (subsequent to the first one) is $O(\sum_{i=1}^{|\mathcal{V}|} k_i d_i)$, which can be approximated as $O(k|\mathcal{E}|)$, with k being the average number of non-zero entries per column. Due to the high cost of the initial iteration, R-MCL cannot be directly applied on large graphs.

The analysis is similar with MLR-MCL, but with a crucial difference. The Regularize step of the first iteration is carried out on the coarsest graph, so the time complexity of $O(\sum_{i=1}^{|\mathcal{V}|} d_i^2)$ for the first Regularize step now applies to the coarsest graph. As the coarsest graph is small, this is an affordable step. As the algorithm proceeds, we simulate flow on bigger graphs, but at the same time the flow matrix also becomes sparser, enabling the algorithm to scale easily. Empirically we observe that after the first Curtailed R-MCL run on the coarsest graph, there are rarely more than a few tens of non-zero entries per column. The overall time complexity of the algorithm is well approximated as $O(k|\mathcal{E}| + \sum_{i=1}^{|\mathcal{V}_c|} d_i^2)$, where the d_i s are the degrees of the nodes in the coarsest graph, and k is a small constant, typically in the tens.

Quality: Embedding R-MCL in a multi-level framework leads to improvements in quality as well, as we show in Section 3.4.2. Coarsening the graph allows the algorithm to utilize the global topology of the graph to provide an effective initialization of the flow values for the simulations on the bigger graphs. At the same time because iterations are run on the final graph as well, the algorithm is able to adjust suitably to the local topology. All of this translates into clusters that are of higher quality than those that are produced by either MCL or R-MCL.

3.3 Proof of Theorem 1

Defintion 1 (Permutation matrix). *A square matrix P is a permutation matrix if there exists exactly one entry 1 in each row and column and zeroes elsewhere.*

Defintion 2 (Row Permutable). *Two matrices A and B of the same size are row permutable if it is possible to permute the rows of B in order to obtain A , and vice versa.*

Fact 1. *A and B are row permutable if and only if there exists a permutation matrix P such that $A = PB$ (and $B = P^{-1}A$).*

Defintion 3 (Preservation of row permutability). *An operator σ on matrices preserves row permutability if the following holds: if A and B are row permutable, then so are $\sigma(A)$ and $\sigma(B)$.*

Proposition 2. *Right multiplication preserves row permutability.*

Proof. If A and B are row permutable matrices, we need to show that for any matrix C , the products AC and BC are also row permutable.

Let P be the permutation matrix such that $A = PB$ (from Fact 1). We have $AC = (PB)C = P(BC)$. Since $AC = P(BC)$, we have from Fact 1 that AC and BC are row permutable.

Hence right multiplication preserves row permutability. $\square$

Proposition 3. *Converting a matrix to a stochastic matrix by normalizing the columns preserves row permutability*

Proof. Let A be a matrix and S be a diagonal matrix with the column sums of A along its diagonal. Then the stochastic matrix corresponding to A is simply AS^{-1} . Hence by Proposition 2, row permutability is preserved. $\square$

Proposition 4. *Let M_c be a flow matrix on one of the coarse graphs in MLR-MCL. Let M_r^1 be the result of flow projection from M_c by randomly choosing for each parent node which of its child nodes will be assigned all the in-flows of the parent node. Let M_r^2 be the result of another flow projection along the same lines, with a (possibly different) random choice at each parent node. Then M_r^1 and M_r^2 are row permutable.*

Proof. Consider a fixed node i in the coarse graph. Assume that in M_r^1 , the choice at i is to project the flow to the first child node $\text{NodeMap1}(i)$, whereas in M_r^2 the flow is projected to $\text{NodeMap2}(i)$ instead. Then the row corresponding to $\text{NodeMap1}(i)$ in M_r^1 is the same as the row corresponding to $\text{NodeMap2}(i)$ in M_r^2 , since they are both projections of the in-flows of the same parent node i . Also, both the row corresponding to $\text{NodeMap2}(i)$ in M_r^1 and the row corresponding to $\text{NodeMap1}(i)$ in M_r^2 consist of zeroes. Hence, the two rows corresponding to the two child nodes of i in M_r^1 have been permuted in M_r^2 . We can extend this argument for every node at which different choices are made in the two flow projections, and therefore conclude that M_r^1 can be converted to M_r^2 using a series of row permutations, one for each different choice. $\square$

Proposition 5. *Both R-MCL and curtailed R-MCL preserve row permutability.*

Proof. We prove this by proving that each of the component operators of a single iteration of R-MCL preserves row permutability.

It follows from Proposition 2 that the *Regularize* operator preserves row permutability, since it is simply right multiplication of the input matrix by the canonical transition matrix M_G .

The *Inflate* operator consists of two operations: (a) raising each entry to power r , which clearly preserves row permutability, and (b) renormalizing the columns, which from Proposition 3 preserves row permutability. Hence *Inflate* as a whole preserves row permutability.

The *Prune* operator computes a threshold for each column and prunes entries below that threshold; because the computation of the threshold is independent of the order of elements in the column, this operator also preserves row permutability. $\square$

Proposition 6. *If two (converged) flow matrices A and B are row permutable, their interpretation as clusterings (according to Section 3.1.2) are the same.*

Proof. If a pair of columns of A (say the i^{th} and j^{th}) are equal, then the i^{th} and j^{th} columns of B are also equal; this is because in going from A to B , the same permutation has been applied to the elements of both the columns. This can be generalized to a set of columns as well; if a set of columns of A are equal to each other, then the same set of columns of B will also be equal to each other.

Next, observe that, as described in Section 3.1.2, all nodes which flow to the same node are grouped into the same cluster. This is the same as saying that a group of nodes with the same columns in the flow matrix will be grouped into one cluster. But

we just saw above that if a group of columns are equal in A , they are also equal in B . Hence, A and B induce the same clustering. $\square$

Theorem 2. *The MLR-MCL algorithm produces the same final clustering regardless of which child node is picked at each parent node to be assigned all its in-flows.*

Proof. From Propositions [4](#), [5](#) and [6](#). $\square$

Name	$ \mathcal{V} $	$ \mathcal{E} $	Avg. degree
Cora	17604	74180	8.42
Dblp	16196	45031	5.56
Astro-Ph	17903	196972	22.00
Hep-Ph	11204	117619	21.00
Hep-Th	8638	24806	5.78
Epinions	75877	405739	10.69
Yeast-PPI	4741	15148	6.39

Table 3.1: Details of real datasets

3.4 Experiments

We performed experiments on 7 real world datasets; four of these were author collaboration networks - three from the Physics community (Astro, HepPh and HepTh), and one from the Computer Science community (DBLP) -, one is a who-trusts-whom network from Epinions.com (Epinions)³, one is a paper citation network (Cora)⁴ and the last is the Protein-Protein Interaction network of yeast (Yeast-PPI)⁵. Details are given in Table 3.1.

The experiments were performed on a dual core machine (Dual 250 Opteron) with 2.4GHz of processor speed and 8GB of main memory. The software for each of our baselines was downloaded from the respective author’s webpages. Our implementation was in C/C++, as were the implementations of all of our baselines. The matrices were stored using a sparse matrix representation in our implementation.

3.4.1 Evaluation criteria

Except for the Yeast PPI network where we use a domain-specific evaluation, we will use *normalized cut* or *conductance* as our measure of cluster quality (see Section 2.1.1, Eqn 2.1).

³Astro, HepPh, HepTh and Epinions were obtained from <http://cs-www.cs.yale.edu/homes/mmahoney/NetworkData/>

⁴Obtained from Andrew McCallum’s web page: <http://www.cs.umass.edu/mccallum/code-data.html>

⁵Obtained from the Database of Interacting Proteins: <http://dip.doe-mbi.ucla.edu/dip/Main.cgi>

Dataset	MLR-MCL				R-MCL				MCL			
	Clusters	N-Cut	Avg. N-Cut	Time	Clusters	N-Cut	Avg. N-Cut	Time	Clusters	N-Cut	Avg. N-Cut	Time
Hep-Ph	264	76.77	0.29	0.91	458	190.03	0.41	5.41	1464	827.31	0.56	85
Cora	670	238.19	0.35	1.26	880	367.62	0.41	3.58	2991	1888.6	0.63	82
Astro	411	153.43	0.37	2.62	343	124.40	0.36	8.84	1940	1301.5	0.67	515
Dblp	723	152.24	0.21	0.51	1750	575.19	0.32	1.57	1943	648.48	0.33	12.0
Epinions	1632	735.87	0.45	26.86	4025	1863	0.46	32.3	15663	10041	0.64	4383
Hep-Th	795	293	0.37	0.37	735	266.4	0.36	1.05	1655	855	0.51	12

Table 3.2: Comparison of MLR-MCL, R-MCL and MCL.

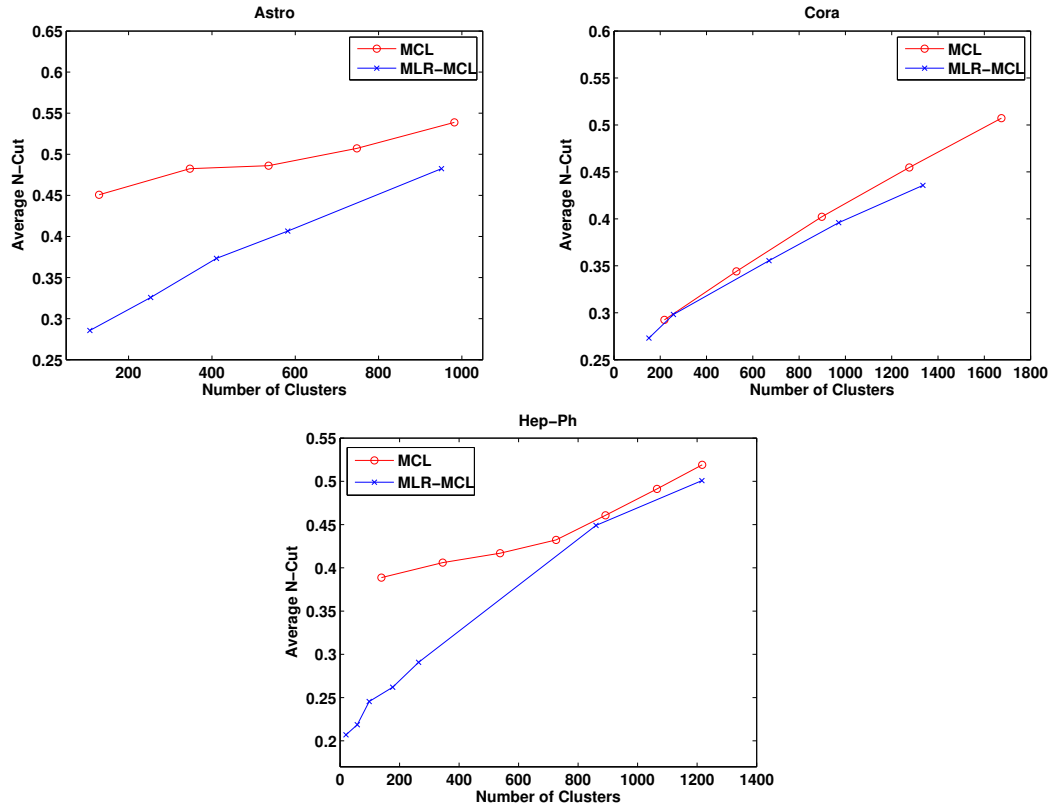


Figure 3.3: Comparison of Avg. N-Cut scores between MLR-MCL and MCL.

3.4.2 Comparison with MCL

In our first set of experiments we compare the performance of R-MCL and MLR-MCL with the baseline MCL algorithm. Table 3.2 documents results obtained on 6 real datasets (also see Figure 3.3). The key trends one can glean from this study are as follows.

First, MLR-MCL clearly dominates both R-MCL and MCL in terms of scalability. It is about 2 orders of magnitude faster than MCL and about one order of magnitude faster than R-MCL for most of the datasets. Second, in all cases both MLR-MCL and R-MCL report far fewer clusters than MCL (keeping the inflation parameter constant across all three methods). Third, in terms of average normalized cut scores MLR-MCL dominates MCL and also usually outperforms R-MCL. On two datasets, namely Astro and Hep-Th, we find that R-MCL achieves a marginally better average N-Cut score. The reason for MLR-MCL achieving better quality is that running curtailed R-MCL on the coarsest graph is beneficial in terms of capturing the global topology of the graph.

3.4.3 Comparison with Graclus and Metis

In the next set of experiments we compare the qualitative performance of MLR-MCL with Graclus and Metis. We detail the results here on 6 real datasets. In all experiments MLR-MCL is run with the inflation parameter $r = 2.0$. For MLR-MCL we vary the coarseness of the clustering by varying the size of the coarsest graph; we subsequently run Graclus and Metis to output the same number of clusters as has been found by MLR-MCL. We plot the average normalized cut of each algorithm as a function of the number of clusters. (It must be kept in mind that with more number of clusters, seemingly small differences in average N-Cut can translate into significant differences in the total N-Cut.) From Figure 3.4a-f one can easily observe that MLR-MCL is either competitive with Graclus (DBLP and Hep-Ph) or better (Astro-Ph, Cora, Epinions and Hep-Th) in terms of the normalized cut objective. The improvement in Avg Ncut over Graclus for these latter four datasets is in the range of 10-15%, if we consider the median number of clusters, which is quite significant. Another obvious trend is that both these algorithms outperform Metis, often significantly. Drilling deeper into the data we find that this can be explained by the fact that both Graclus and MLR-MCL admit a more skewed (actually the skew

for both is quite similar) clustering arrangement whereas Metis tends to force a more balanced partitioning. Another interesting observation is that when the number of clusters discovered is more, MLR-MCL typically performs better. A third point of note is that the two datasets where Graclus is competitive with MLR-MCL - Dblp and Hep-Th - are the two datasets with the lowest average degree of the 6 datasets (5.56 and 5.78 respectively).

Scalability Evaluation: In the next set of experiments we compare and contrast the scalability of MLR-MCL with Metis and Graclus on three of the real datasets, while varying the number of clusters. From Figure 3.5a for the Epinions dataset we find that MLR-MCL is competitive with Metis, and that both algorithms are faster than Graclus. The trends in Figures 3.5b and c are consistent in that Metis outperforms MLR-MCL, which in turn outperforms Graclus, especially with increasing number of clusters.

3.4.4 Clustering PPI networks: A Case Study

The goal of analyzing protein-protein interaction (PPI) networks is to extract groups of proteins that either take part in the same biological process (*induction of cell death* is an example) or perform similar molecular functions (e.g. *RNA binding*). This is a challenging problem; it is estimated that the protein function of about one-fourth of the proteins is unknown even for the most well-studied organisms such as yeast [103].

We use as our dataset the PPI network of *S. cerevisiae* or yeast, which contains 4741 proteins with 15148 known interactions. We perform a domain-specific evaluation using The Gene Ontology database [113], which provides three vocabularies (or annotations) of known associations – Molecular Function, Biological Process and Cellular Component. The first two have functional significance while the last one refers to the localization of proteins within a cell. Researchers have used this ontology in the past to validate the biological significance of clusters. Merely counting the number of proteins that share an annotation within each extracted cluster is misleading since the underlying frequency of the annotations is not uniform - more proteins are characterized by an annotation at the top of the hierarchy than at the bottom. For this reason, p-values are often used to calculate the statistical significance of such clusters [9]. Intuitively these values capture the probability of seeing a particular grouping, or better, by random chance using a background distribution (typically

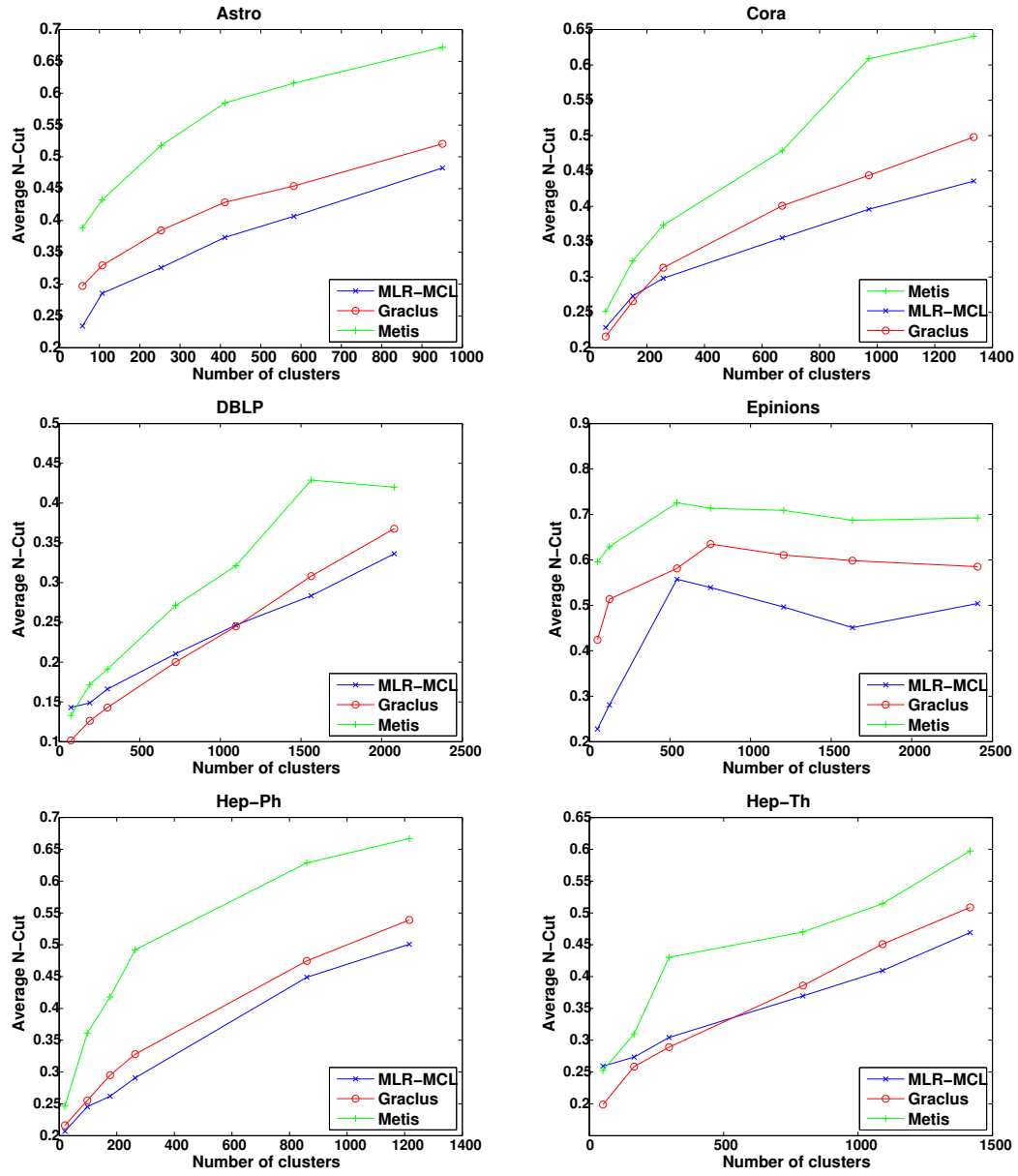


Figure 3.4: Comparison of Avg N-Cut scores: MLR-MCL, Graclus and Metis.

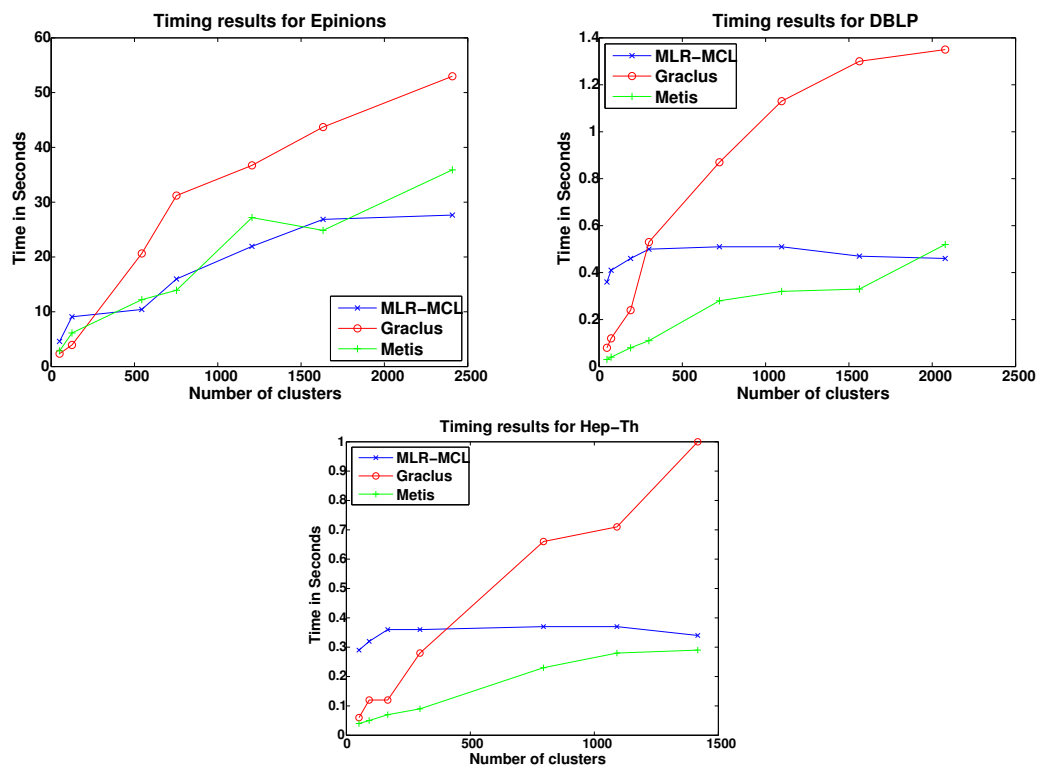


Figure 3.5: Comparison of timing between MLR-MCL, Graclus and Metis.

hyper-geometric). Let the total number of proteins be N with a total of M proteins sharing a particular annotation. The p-value of observing m or more proteins that share the same annotation in a cluster of n proteins, using the Hypergeometric Distribution is:

$$p - value = \sum_{i=m}^n \frac{\binom{M}{i} \binom{N-M}{n-i}}{\binom{N}{n}}$$

Smaller p-values imply that the grouping is less likely to be random. ⁶

It is worth remarking before we discuss the results that earlier research has shown that MCL typically outperforms MCODE and several other domain specific community discovery algorithms for such biological networks[22]. In our study we compared the performance of MCL, MLR-MCL and Graclus on this domain specific qualitative metric. Metis was found to perform poorly on this metric, primarily because of its tendency to favor balanced clusters.

We set the inflation parameter r to 1.6 for both MCL and MLR-MCL. The size of the coarsest graph was set to 1000 nodes for MLR-MCL. MLR-MCL returned 427 clusters where as MCL returned 1615 clusters. We then ran Graclus to output 427 clusters to keep the comparison fair. Each cluster is associated with the annotation that minimizes the p-value for that cluster, and the corresponding p-value was retained as the p-value to represent that cluster.

Figures 3.6a and b compare the p-values of the top 100 clusters returned by each algorithm, under the Biological Process (P) and Molecular Function (F) vocabularies respectively. The Y-axis represents negative log p-values while the X-axis is simply an ordered list of the top-scoring clusters produced by the different graph clustering algorithms. Since better clusters have lower p-values, higher values on the graph represent a higher quality of clustering. As we see from the charts, MLR-MCL clearly outperforms Graclus and MCL among the top set of clusters and is clearly competitive or better than either across the board for both the Molecular Function and Biological Process ontologies.

Under the evaluation using Biological Process annotations, the top-scoring cluster returned by MLR-MCL obtained a p-value of $1.8e - 80$, which is significantly better than $2.4e - 30$ and $1.6e - 28$, the top scoring p-values for Graclus and MCL respectively. In fact MLR-MCL returns 8 clusters which score better p-values than the best p-value

⁶We used a publicly available package called GO-TermFinder for calculating the p-values. The url is <http://search.cpan.org/dist/GO-TermFinder/>

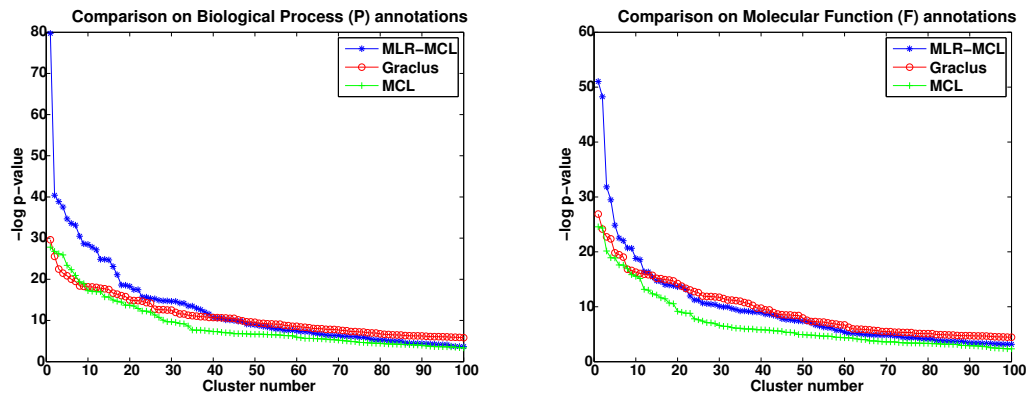


Figure 3.6: Significance of discovered protein clusters.

scored by Graclus. The p-value of $1.8e - 80$ was scored by a cluster of 88 proteins returned by MLR-MCL, out of which 55 are proteins currently known to be involved in the process of *nuclear mRNA splicing via spliceosome*. It is very interesting that the top scoring cluster for MCL was also in fact matched with the same annotation, but MCL managed to retrieve only 25 of the proteins known to be involved in this process. This clearly illustrates that MLR-MCL overcomes the main qualitative limitation of MCL - fragmentation of output.

Figure 3.6c compares the p-value distributions of the clusters discovered by MCL under three different settings for the inflation parameter r - 1.2, 1.6 and 2.0. As can be seen from the figure, MCL discovers very similar clusters for all three settings - it is hard to distinguish the p-value distributions for the three settings. In addition, the same number of clusters - 1615 - were discovered by MCL for all three values of r . This demonstrates that the fragmentation of output problem for MCL cannot be alleviated by merely varying the inflation parameter r .

3.5 Conclusion

In this chapter, we have presented Regularized MCL and Multi-Level Regularized MCL, two flow based algorithms for graph clustering. Results on several real and synthetic datasets highlight the utility of the approach when compared with MCL, Metis and Graclus, three state-of-the-art graph clustering algorithms. Specifically, we find that the new algorithms are 2-3 orders of magnitude faster than MCL, and improve significantly on the quality of the output clusters. Similarly we find that our approaches outperform Metis and Graclus in terms of quality and are competitive in terms of scalability.

In the next chapter, we will discuss modified versions of R-MCL and MLR-MCL that allow the user to adjust the skew in the size of the output clusters. We will also discuss why such a feature might be useful.

Chapter 4: Graph Clustering with Adjustable Balance

In this previous chapter we discussed two algorithms, R-MCL and MLR-MCL, which significantly improve upon MCL in terms of quality and scalability. In this chapter, we will discuss a further modification of R-MCL (and thereby MLR-MCL) that allows users to adjust the balance or the variance in sizes of the output clusters. This variant allows the user to adjust the balance of the output clustering arrangement using a balance parameter b ; setting $b = 0$ recovers the original algorithm.

There are multiple reasons for wanting such a feature in graph clustering algorithms. A powerful motivation for obtaining balanced clusters is in the setting of protein-protein interaction (PPI) networks in Bioinformatics. A balanced clustering arrangement is definitely among the desiderata for such problems. It has been shown that the size-frequency distributions of protein complexes exponentially decrease [16]. In both the CYC2008 [94] and the CORUM [96] protein complex data, more than 90% of the proteins belong to complexes that are no larger than 20 proteins. All of this implies that algorithms for clustering protein interaction networks should strive to avoid producing large clusters as well as singletons. Further complicating the issue on hand is the noisy or uncertain nature of such data, i.e., presence of false positive interactions and absence of false negative interactions can influence the resulting clusters.

A second motivation has to do with the inherent topological structure of modern networks. Recent empirical studies [74, 75, 71] have demonstrated that many real world networks have tight clusters only at the peripheries and that the majority of the nodes in the graph belong to an expander-like core which is hard to partition into clusters of balanced sizes. This means that popular graph clustering algorithms that do not explicitly aim to maintain balance will have a tendency to cluster the graph into a giant core surrounded by many whiskers, and in fact it has been discovered that spectral clustering algorithms suffer from this weakness [71]. While such a partitioning

of the graph is certainly a valid clustering insofar as it respects the graph topology and yields cuts with extremely low conductance, from a data mining point of view a user or other downstream applications wishing to derive some useful intelligence out of the graph will not learn much from such a clustering. Therefore, while clusterings with more balanced cluster sizes will have poorer conductances, they are still important as they represent more useful information.

In the domain of bio-informatics, we exhaustively compare our algorithm MLR-MCL with MCL on three real protein interaction networks; two Yeast PPI networks (one of which is significantly noisier) and a Human PPI network. We evaluate all algorithms using manually curated protein complex data as the ground truth. Our study looks at the quality of the clusters discovered by each algorithm, the balance of the clustering arrangements as well as the scalability. We briefly summarize the main results:

- MLR-MCL significantly out-performs MCL in terms of cluster quality on all three networks, with improvements of 20%, 300% and 21% respectively. Furthermore MLR-MCL's performance remains more stable across a range of clustering granularities and balance parameter settings.
- MLR-MCL shows much better balance of resulting clusters in comparison with MCL. Most of the nodes in the graph belong to clusters of sizes 4-20 for MLR-MCL, with relatively few singletons or large clusters. MCL, in contrast, is found to significantly suffer from poor balance, especially in the noisier networks.
- MLR-MCL is much faster than MCL; the speedups vary from a minimum of 50 to a maximum of 500. MLR-MCL never takes more than half a minute to cluster even our biggest network (containing close to 200,000 interactions).
- We also compare against Metis [63], a popular balanced clustering algorithm, and find that although Metis shows much better balance than MCL, the clusters discovered by MLR-MCL are still of 30% higher quality.

4.1 Regularized MCL with adjustable balance

Our modification to R-MCL preserves the original feature that each node in the graph updates its out-flows to the weighted average of the out-flows of its neighbors. The weights between all pairs of nodes for this purpose is specified by the canonical flow matrix M_G . In our modification, however, we no longer restrict ourselves to using M_G as the weight matrix. Instead, we construct a new regularization M_R using the current flow matrix M and the canonical flow matrix M_G . The intuition behind our construction is as follows. When a node incorporates the flows of its neighbors in the *Regularize* operator, it gives greater weightage to those nodes which are likely to be a part of smaller clusters, and contra-wise, lesser weightage to those nodes likely to be a part of large clusters. This way, this node itself is more likely to join a smaller cluster, enabling the cluster sizes to be more balanced.

We will need to define two concepts before we can specify more details of the modification. For a particular flow on the graph i.e. given a flow matrix M associated with the graph, we give the following definitions:

1. The *mass* of a node is defined as the sum total of the transition probabilities (or flows) into the node. The physical analogy here is that the nodes with higher mass attract more nodes in the graph towards them. The mass of a node can be calculated as simply the sum of the row corresponding to this node in M i.e. $mass(i) = \sum_j M(j, i)$. In matrix notation, the *mass* vector $\mathbf{m}$ is $\mathbf{m} = M\mathbf{1}$ where $\mathbf{1}$ is the column vector of all 1s.
2. The *propensity* of a node is defined as the weighted average of the masses of the nodes it flows into i.e. $propensity(i) = \sum_j M(j, i) * mass(j)$. In matrix notation, the *propensity* vector $\mathbf{p}$ is $\mathbf{p} = M^T \mathbf{m}$ where $\mathbf{m}$ is the mass vector.

Recall that the *Regularize* step updates the out-flows of each node to be the weighted average of the out-flows of all its neighbors i.e. $Regularize(M) = M * M_G$. The distribution of masses of the “attractor” nodes is a proto-indicator of the distribution of final cluster sizes i.e. imbalanced clusters are output only when there is imbalance in the distribution of masses of the “attractor” nodes during the MLR-MCL process. Therefore we aim to push more nodes towards attractor nodes with lower mass, thus evening out the distribution of masses. Achieving this, in turn, requires that weight of each neighbor of a given node in the regularization matrix M_G

is set in inverse proportion to the propensity of the neighbor. This means that we no longer use the same matrix M_G for regularization but construct a matrix M_R afresh at each iteration.

The algorithm for constructing a new regularization matrix given M and M_G is given in Algorithm 5. Let P be the diagonal matrix with the propensity vector $\mathbf{p}$ along the diagonal i.e. $P = \text{diag}(\mathbf{p})$. Given a user-specified *balance parameter* b , the new regularization matrix M_R is given by:

$$M_R = \text{normalize}(M_G * P^{-b})$$

where the *normalize* operation rescales each column so that each column sums to 1. b specifies the extent to which higher-propensity neighbors are to be penalized. The higher the value of b , the greater the penalty on high-propensity neighbors.

The *Regularize* step is subsequently performed as $M := M * M_R$. The rest of R-MCL is left unchanged. The overall algorithm for R-MCL is given in Algorithm 6. The Multi-level version of this modified R-MCL simply substitutes Algorithm 6 in place of Algorithm 2, and this is explained in Algorithm 7.

4.1.1 Effect of the balance parameter b

Note that setting the balance parameter b to 0 recovers the original R-MCL algorithm, as given in Algorithm 2, and for this reason Algorithm 6 is a generalization of Algorithm 2. Whenever we want to refer to the original R-MCL (or MLR-MCL), we simply refer to them as R-MCL (or MLR-MCL) with balance parameter $b = 0$. This allows us to avoid introducing new names with the potential for confusion.

If the user wishes for a more balanced clustering than currently output, they can achieve that by increasing b ; conversely, a less balanced clustering may be obtained by lowering b . This is because higher values of b lead to a more severe down-weighting of nodes with high propensity values. Higher values of b also lead to a slower convergence of R-MCL (and MLR-MCL). This is because higher values of b encourage out-flows towards nodes with low in-flows; the in-flows of such nodes would have gone to 0 with lower values of b . Despite this slowing down, MLR-MCL is still very fast, as can be seen from the results in Section 4.2.

Algorithm 5 RegularizationMatrix(M, M_G, b)

Input: Current flow matrix M , Canonical flow matrix M_G , Balance parameter b

Output: New regularization matrix M_R

```
// Compute mass vector
 $\mathbf{m} = M\mathbf{1}$ 
// Compute propensity vector
 $\mathbf{p} = M^T \mathbf{m}$ 
 $P := \text{diag}(\mathbf{p})$ 
 $M_R := M_G * P^{-b}$ 
Normalize  $M_R$  so that each column sums to 1.
return  $M_R$ 
```

Algorithm 6 Regularized MCL (modified for balance)

Input: Graph adjacency matrix A , Balance parameter b , Inflation parameter r

$A := A + I$ // Add self loops and transform weights

$M := M_G := AD^{-1}$ // Initialize M as the canonical flow matrix

repeat

$M_R := \text{RegularizationMatrix}(M, M_G, b)$

$M := M_{reg} := M * M_R$

$M := M_{inf} := \text{Inflate}(M, r)$

$M := \text{Prune}(M)$

until M converges

Interpret M as a clustering [41, 98]

Algorithm 7 Multi-level Regularized MCL (modified for balance)

Same as Algorithm 3, except that we substitute Algorithm 6 in place of Algorithm 2 for Regularized MCL.

4.2 Results on PPI networks

4.2.1 Datasets

We evaluate our proposed approach using three real protein-protein interaction networks. See Table 4.1. The first is a PPI network of Yeast, obtained from the Database of Interacting Proteins (DIP) [120]. The second is also a PPI network of Yeast, obtained from the BioGRID database [109], version 2.0.61. The data from BioGRID includes a higher percentage of interactions derived from high-throughput interaction identification methods, such as quantitative genetic interaction data generated in high-throughput studies [32] and Yeast Two Hybrid analysis. Therefore, we expect BioGRID data to be more noisy than the DIP dataset, which includes a lower rate of interactions derived from large-scale experiments. It should be noted that the number of edges in the BioGRID network is more than 10 times the number of edges in the DIP network. The third network is a Human PPI network, obtained from the iRefIndex [97] integrates interaction data from ten different interaction databases (including DIP and BioGRID).

For evaluating the clustering results of the two Yeast PPI networks, we used the CYC2008 [94] protein complex data as the gold standard for ground truth validation. CYC2008 is “a comprehensive catalogue of manually curated 408 heteromeric protein complexes in *S. cerevisiae* reliably backed by small-scale experiments from the literature”. The CYC2008 data has 1920 annotations involving a total of 1627 proteins (some proteins have multiple annotations). Out of the original 1627 proteins with annotations, 288 proteins are completely absent in the DIP data, and 3 are completely absent in the BioGRID data. For evaluating the results of clustering the DIP network, we used only the subset of the CYC2008 annotations involving proteins present in the DIP network, yielding 1621 annotations (out of the original 1920) and 359 complexes involving at least 2 proteins.

Similarly, for evaluating the clustering results of the Human PPI network, we used the CORUM [96] protein complex data, which is also a manually curated complex data for human proteins. The original data has 938 complexes, involving 2083 proteins and a total of 5846 annotations (again some proteins have multiple annotations). We removed those annotations involving proteins absent in our PPI network, obtaining 520 complexes with 3043 annotations involving 1520 proteins.

Note that a significant fraction of the proteins in all three networks do not have any ground truth annotations (see the right-most column in Table 4.1). This means

Organism	Source	Proteins	Interactions	Percent of proteins with ground truth annotations
Yeast	DIP	4,741	15,147	28.2%
Yeast	Biogrid	5,964	192,851	27.2%
Human	iRefWeb (Wodak Lab)	11,750	63,419	16.3%

Table 4.1: PPI networks used in the experiments

that the precision (i.e. the fraction of proteins with correct annotations per output cluster) is expected to be low for many output clusters, and these low precision values are therefore not an accurate reflection of the goodness of the clustering algorithms.

4.2.2 Experimental Setup

We implemented MLR-MCL using sparse matrix representations in C/C++. For MCL, we used the implementation available for download from the author’s website. All the experiments were performed on a dual core machine (Dual 250 Opteron) with 2.4GHz of processor speed and 8GB of main memory. However, the programs were single-threaded so only one core was utilized.

For MLR-MCL, we tried five balance settings, $b = \{0, 0.5, 1, 1.5, 2\}$ and chose the balance setting with the best performance to compare with MCL. The number of clusters was varied by varying the size of the coarsest graph (parameter c in Algorithm 3). The inflation parameter was fixed to $r = 2.0$ always since this seems to work best uniformly across all experiments we report for this algorithm.

For MCL, we generated varying number of clusters by varying the inflation parameter r . For comparing the balance between MLR-MCL and MCL, we choose the parameter setting with the best F-score for each method.

4.2.3 Quality and balance comparison between MCL and MLR-MCL

We compare MCL and MLR-MCL in terms of quality and scalability on the three networks. We also show the distribution of cluster sizes i.e. the balance of the two clustering methods, as this also provides important insights. Note that in order to

assess balance, just showing the number of clusters of a certain size is not as insightful as showing the *number of nodes* which belong to clusters of a certain size. This is because even one large cluster of size (say) 500 severely affects the balance (and utility) of the clustering arrangement, and this is reflected better by plotting the number of nodes belonging to clusters of a certain size than the number of clusters of a certain size.

Figure 4.1(a) shows the Avg. F scores of MCL and MLR-MCL for varying granularities on the Yeast PPI (DIP) network. MLR-MCL achieves the best performance in the range of 500-800 clusters, with an F-score of 24.76% for 558 clusters and 25.09% for 753 clusters. In contrast, MCL’s performance peaks at 899 clusters, with the best F-score being 20.81%. MLR-MCL therefore achieves a significant 20% improvement over MCL. Figure 4.1(b) compares the cluster size distributions of MCL and MLR-MCL. Ideally, we want very few nodes to belong to clusters of size less than 3 or 4, and also very few nodes belonging to clusters of size greater than 15 or 20. We can observe that most nodes of MLR-MCL belong to clusters of size 4-10, with the maximum cluster size being 52. MCL tends to produce smaller clusters, with many nodes in clusters of size range 2-8, and the top two cluster sizes are 167 and 98.

Figure 4.2(a) shows the Avg. F scores on the Yeast PPI (BioGRID) network. The best scores for MLR-MCL are 23.49% for 727 clusters and 22.71% for 437 clusters. MCL, on the other hand, performs very poorly on this network, with a best F score of only 6% for 1361 clusters. The main reason for MCL’s poor performance is the lack of balance in the output clustering, as can be seen from Figure 4.2(b) (note that the y-axis is on log scale). MCL places nearly 1000 nodes in singleton clusters, and further more produces a giant cluster with 3175 nodes in it. In contrast, MLR-MCL produces a much more balanced clustering, with only 140 nodes in singleton clusters and only one cluster with size greater than 100 (of size 373). Furthermore, most nodes are placed in clusters of size 4-20.

Figure 4.3(a) shows the Avg. F scores on the Human PPI network. MLR-MCL’s performs best in the range 900-1700 clusters, with peak F-scores of 10.46% and 10.48% for 1268 and 1692 clusters respectively. MCL, in contrast performs very poorly when generating anything less than 1000 clusters, and its best F score is 8.6% for 1992 clusters. This is in fact lower than any F score for MLR-MCL in the range 300-3000 clusters. The balance of MLR-MCL and MCL can be seen in Figure 4.3(b). This time MCL outputs fewer singletons (172) as compared to MLR-MCL (591), but

MCL places a lot of nodes in small clusters (2-5) as compared to MLR-MCL, whose distribution peaks at cluster size 10. MCL also produces 4 clusters of size greater than 100 (with the largest cluster of size 400) whereas the largest cluster output by MLR-MCL has only 97 nodes.

Looking at the overall trends from the three networks, MLR-MCL clearly outperforms MCL in all three. Further more, the performance of MLR-MCL remains relatively more stable across varying number of clusters, and generally achieves best performance when the number of clusters is around 15% of the number of nodes in the graph. MLR-MCL is also much more robust to noise, as can be seen from the BioGRID network, where the large number of noisy interaction data caused MCL to output extremely imbalanced clusters and resultant drop in performance.

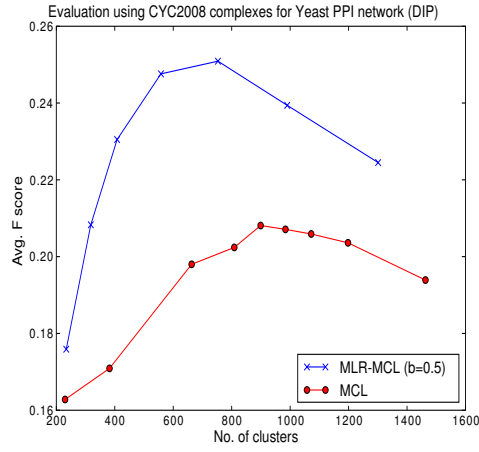
4.2.4 Speed comparison between MLR-MCL and MCL

Figure 4.6 compares the timing results between MCL and MLR-MCL on the two bigger networks, Yeast PPI (BioGRID) and Human PPI. On the Yeast PPI network (BioGRID)(Figure 4.6(b)), MCL takes 460-600 seconds, depending on the number of clusters, while MLR-MCL only takes 8-25 seconds. The difference is even more significant on the Human PPI network (Figure 4.6(b)), where MCL takes anywhere from 400-1700 seconds to cluster the data, while MLR-MCL's takes 2-5 seconds - a 2-3 orders of magnitude speed-up over MCL.

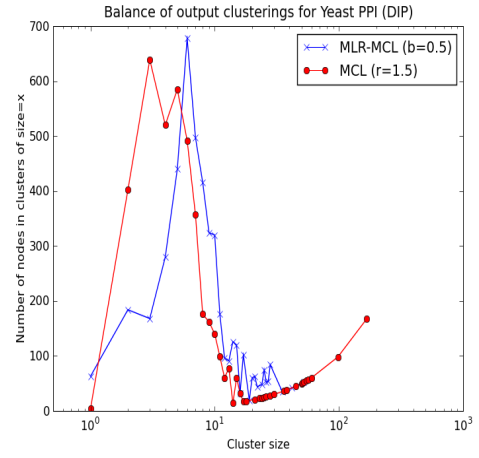
4.2.5 Effect of varying balance parameter b

We now discuss the effect of varying the balance parameter b for MLR-MCL on the DIP and BioGRID networks. (The trends in the results for the Human PPI network are very similar to the BioGRID network.)

Figure 4.4(a) compares the performance of MLR-MCL on the Yeast PPI (DIP) network for three balance parameter settings - $b=\{0, 0.5, 1\}$. The best F-scores for $b=0, 0.5$ and 1 are 23.68%, 25.09% and 24.42% respectively. We also show MCL's performance for comparison, and it can be seen that MLR-MCL outperforms MCL for all three parameter settings. Figure 4.1(b) clearly shows the improving balance with increasing values of b . However, merely producing balanced clusters is not sufficient, as can be seen from the fact that a setting of $b=0.5$ achieves better F-scores despite somewhat poorer balance than a setting of $b=1$.

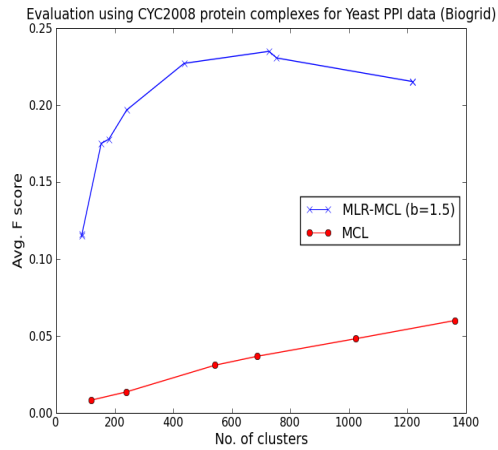


(a)

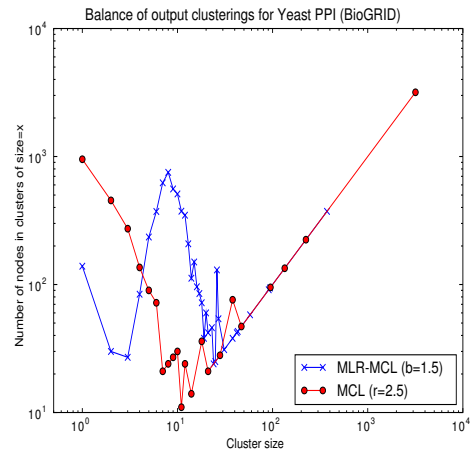


(b)

Figure 4.1: (a) Quality and (b) Balance, on Yeast PPI (DIP)

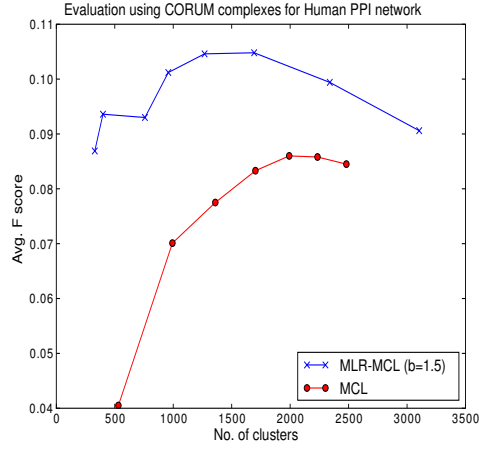


(a)

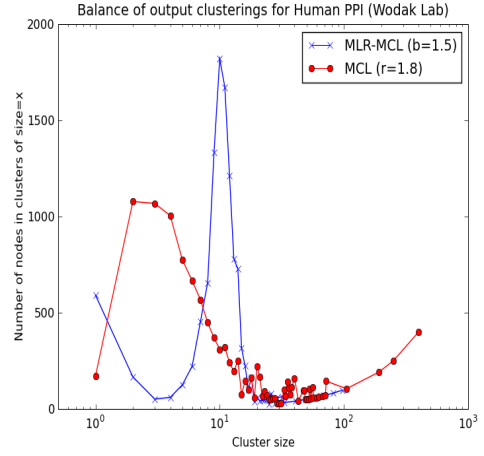


(b)

Figure 4.2: (a) Quality and (b) Balance, on Yeast PPI data from BioGRID

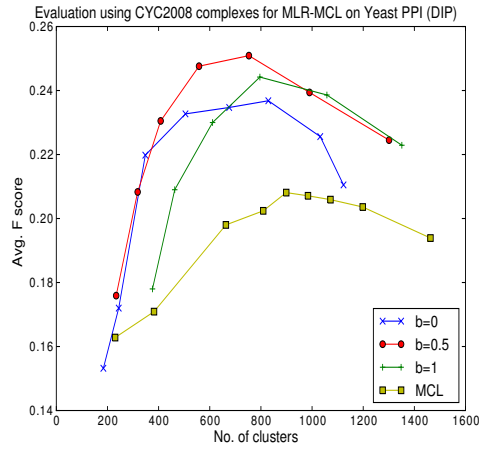


(a)

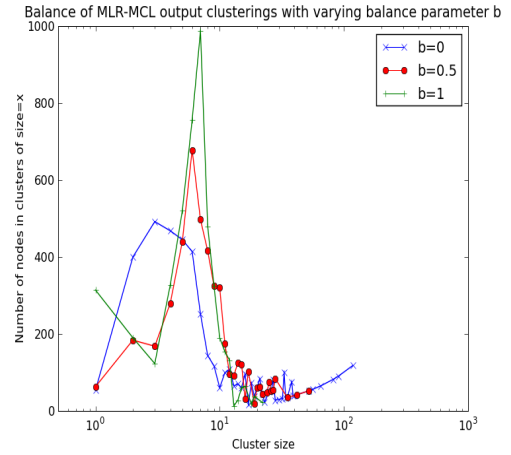


(b)

Figure 4.3: (a) Quality and (b) Balance, on Human PPI data



(a)



(b)

Figure 4.4: (a) Quality and (b) Balance, for varying b on Yeast PPI data from DIP

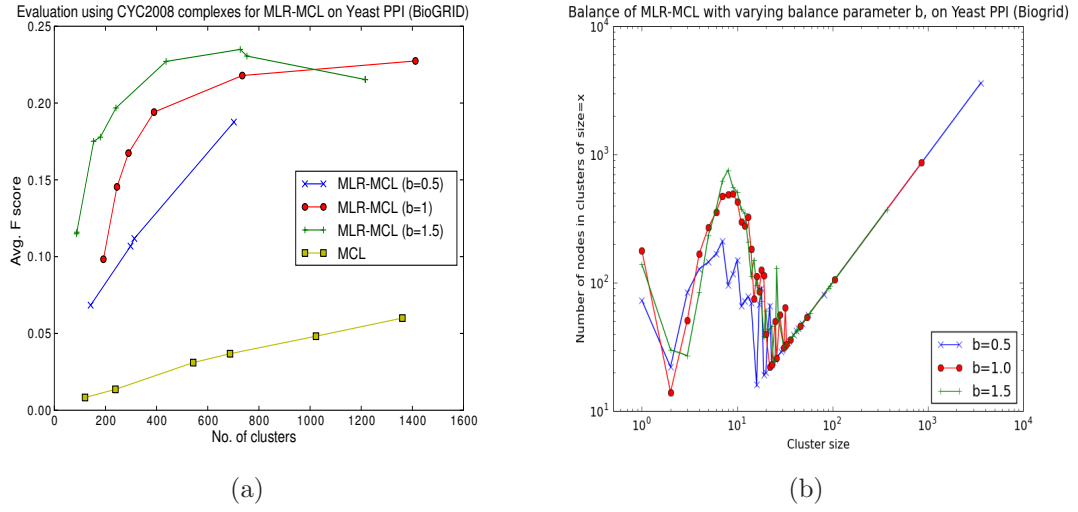


Figure 4.5: (a) Quality and (b) Balance, for varying b on Yeast PPI (Biogrid)

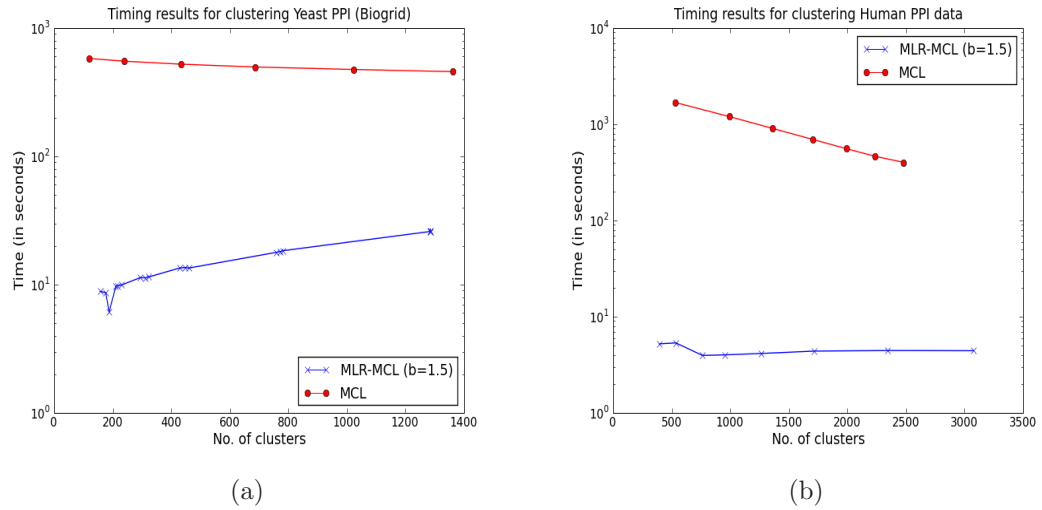


Figure 4.6: Timing comparison on (a) Yeast PPI (Biogrid) and (b) Human PPI

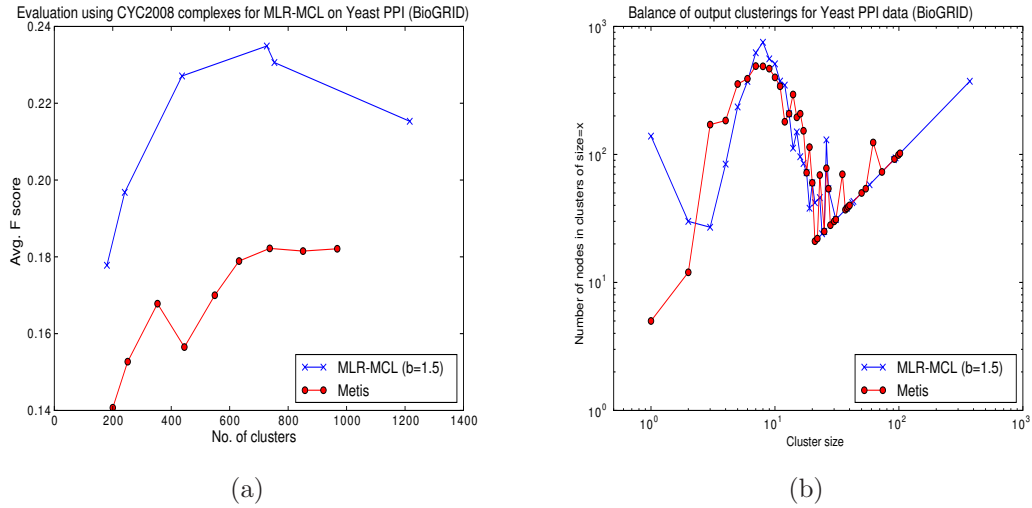


Figure 4.7: (a) Quality and (b) Balance, MLR-MCL vs Metis, (BioGRID)

Figure 4.5 compares the performance of MLR-MCL on the Yeast PPI (BioGRID) network for three balance parameter settings - $b=\{0.5,1,1.5\}$. Higher b values had to be used in this network in order to achieve comparable results on this graph due to the higher density of the network (witness the poor balance of MCL in Figure 4.2(b)). The peak F-scores for $b=0.5$, 1 and 1.5 are 18.76%, 22.74% and 23.49% respectively. Again, all three parameter settings handily outperform MCL, as can be seen from the figure. The balance of the three parameter settings is shown in Figure 4.5(b) (note the log-scale on the y-axis). Note that $b=0.5$ produces a giant cluster of size 3622, akin to MCL (see Figure 4.2(b)) (although it produces far fewer singletons than MCL). The balance for $b=1$ is much improved, with the largest cluster size reduced to 860. With $b=1.5$, the balance is improved further, with a largest cluster of size 373.

A good heuristic for picking the right balance parameter setting is that with more noisy/more dense data (such as BioGRID and the Human PPI data) one should use the higher b values such as 1 or 1.5. However, it should also be noted that MLR-MCL still delivers very good performance for different settings of b (always outperforming MCL) and therefore much effort need not be expended on picking the right parameter setting.

4.2.6 Comparison With Metis

Metis [63] is a popular graph clustering algorithm which optimizes the Kernighan-Lin objective; this objective function basically looks for the clustering arrangement with the least edge cut under the constraint that all clusters are of equal (or approximately equal) size. Metis is especially popular in the parallel and scientific computing community where balanced partitioning of the data so as to balance the workload among multiple processors is important. Figure 4.7 compares the performance of MLR-MCL and Metis w.r.t both quality and balance. As can be seen from Figure 4.7(a), MLR-MCL clearly out-performs Metis at all granularities. The best F score for Metis is around 18%, which it maintains in the 700-1000 clusters range, while MLR-MCL achieves a peak F-score of 23.49%, representing an improvement of 30%. Note that this is despite Metis achieving good balance, as can be seen from Figure 4.7(b). Metis has 5 singletons, compared to MLR-MCL’s 139, and the largest cluster size in Metis is 102, compared to MLR-MCL’s 373. The poor performance of Metis in comparison to MLR-MCL despite the marginally better balance shows that while balance is important, cluster quality cannot be sacrificed.

4.3 Results on Other Graphs

In this section, we present experimental results on graphs obtained from domains other than Bioinformatics.

4.3.1 Synthetic graphs

We generated 4 synthetic graphs of sizes 10,000, 100,000, 500,000 and 1 Million nodes each, with average degree 25, using a state-of-the-art synthetic graph generator for clustering algorithms [70]. The timing and quality for MLR-MCL, Graclus and Metis are shown in Figure 4.8. We set $b = 0.5$ for MLR-MCL.

While MLR-MCL is a little slower than Graclus and Metis on the smaller graphs, it can be seen that MLR-MCL gets faster with increasing size of the graphs. Note that Graclus suffered from thrashing effects on the 500K and 1M node graphs and did not finish execution. For the 1M node graph, MLR-MCL is actually about 35% faster than Metis, taking 11623 seconds compared to 15725 seconds for Metis. In terms of quality, it can be seen that MLR-MCL is much better than Metis and comparable to Graclus.

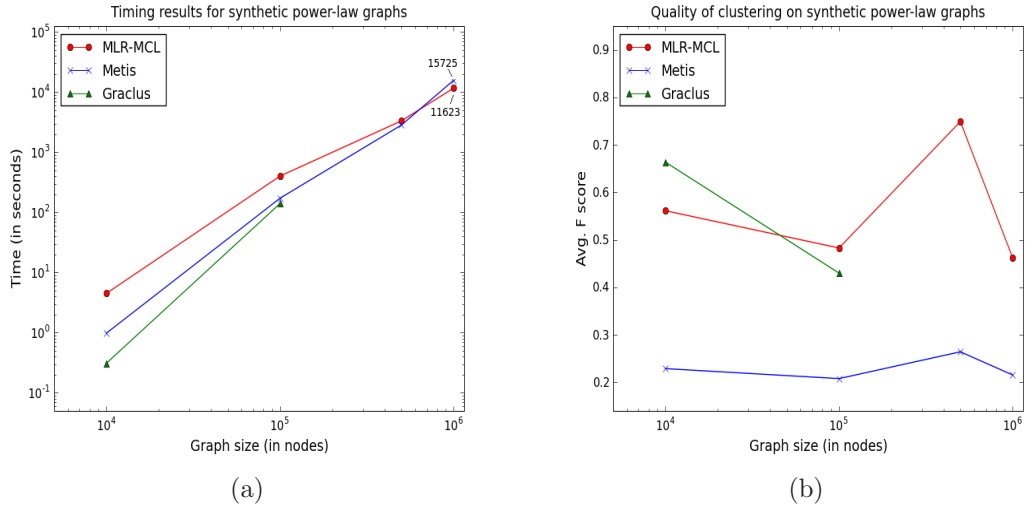


Figure 4.8: (a) Timing and (b) Quality, on Synthetic graphs

4.3.2 Results on Wikipedia

We also conducted experiments on the article-article hyperlink graph from Wikipedia. This graph consists of nearly 1.2M vertices and around 53M edges. For ground truth, we utilized the fact that Wiki articles are assigned to categories (visible at the bottom of the page), and so compared the quality of different output clusterings using the Weighted average F-score of the predicted clusters with the ground truth clusters.

Figure 4.9 compares MLR-MCL ($b = 0.5$) and Metis for clustering Wikipedia. Graclus is not included in these results as it failed to execute. More conventional approaches such as MCL, Spectral clustering, etc. also failed to execute. MLR-MCL is comparably fast compared to Metis up until around 8000 clusters, beyond which Metis is faster. In terms of quality, MLR-MCL is significantly better than Metis, with a peak F-score of 20.3 compared to 15.9 for Metis.

It is also interesting to compare the quality of MLR-MCL with $b=0.25$ vs MLR-MCL with $b=0.5$, shown in Figure 4.10. (The number of clusters in both cases is similar.) The N-Cut obtained with $b=0.25$ is 0.35, which is better than the N-Cut obtained with $b=0.5$, which is 0.5. However, in terms of match with the ground truth, the latter output is much better, scoring 19% compared to the former's 13%. A clear

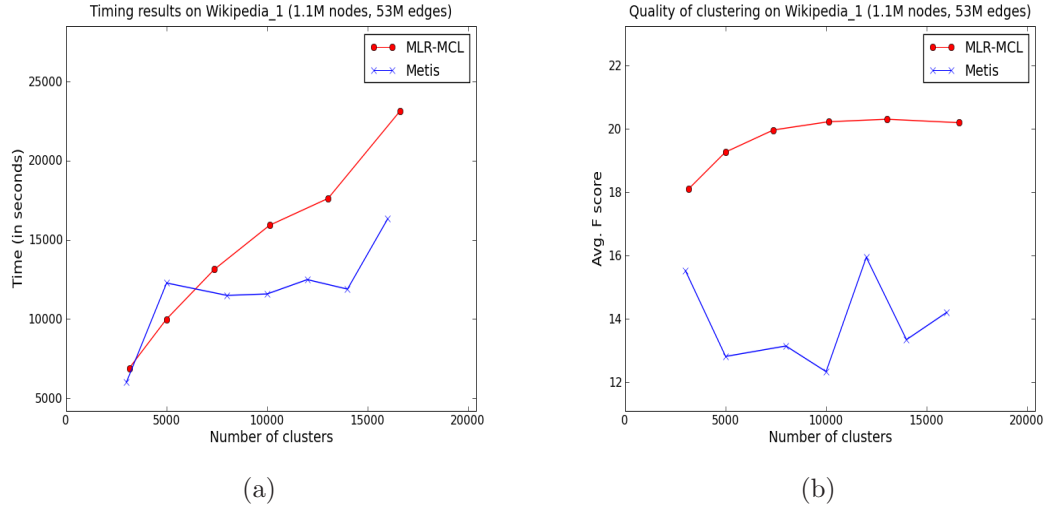


Figure 4.9: (a) Timing and (b) Quality, on Wikipedia

part of the reason is the balance of the two clusterings - with $b=0.25$, there is a giant cluster with nearly 40% of the vertices in it, while with $b=0.5$ the output is much more balanced.

The fact that clustering arrangements with high imbalance in cluster sizes can often achieve low n-cut scores and yet not be useful clusterings, as compared to arrangements with higher n-cut scores but better balance, is illustrated in Figure 4.10

4.3.3 Results on clustering text documents

We also experimented on the 20-newsgroups dataset, consisting of 18774 text documents belonging to 20 ground truth clusters. We first constructed a 50 nearest neighbor similarity graph, using cosine similarity on the TF-IDF representations of the documents. We then ran different clustering algorithms on the nearest neighbor similarity graph. For this dataset, we were able to get results with algorithms that wouldn't execute before, partly because the size of the graph is not too large and also because the desired number of clusters (20) is much smaller than in the case of protein interaction networks or large social networks. A new method that we compare against here which we haven't discussed before is Spherical K-Means [37], which operates

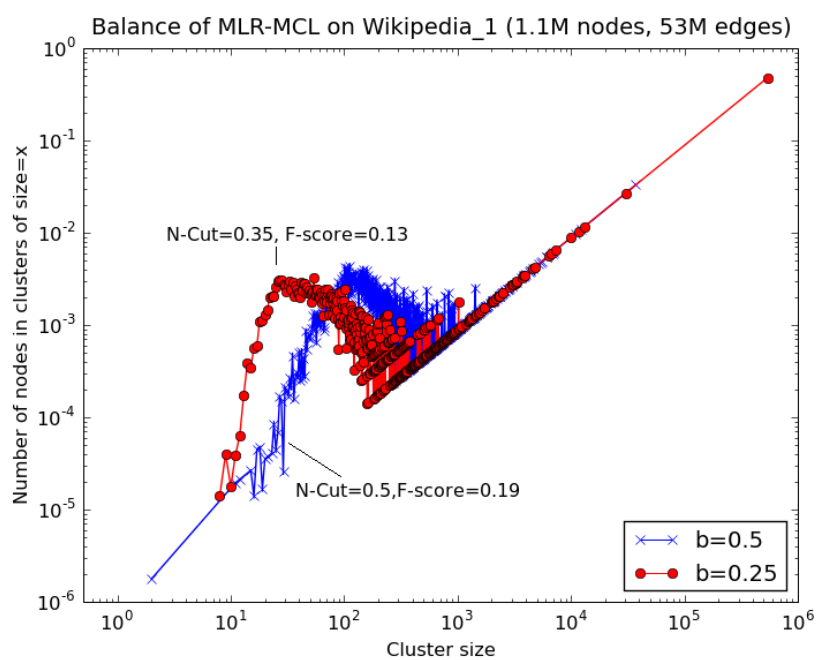


Figure 4.10: Importance of balance, Wikipedia

Method	Avg. F-score	Time (seconds)
MLR-MCL ($b = 0.5$)	61.43	3.1
MCL	40.65	635.4
Grclus	35.04	0.2
Metis	52.84	0.3
Spectral	53.0	4.7
Metis+MQI [72]	61.7	2.9
Spherical K-Means [38]	15.58	2.1

Table 4.2: Quality of clustering 20 newsgroups dataset

directly in the original tf-idf space; i.e. this is a general data clustering algorithm. The Avg. F-scores and the timing results are listed in Table 4.2. MLR-MCL as can be seen is much faster than MCL while also being much more accurate. Comparing against the other approaches, MLR-MCL is either better in terms of quality or in terms of time, or comparable.

4.4 Conclusion

In this chapter we have looked at a simple modification to R-MCL and MLR-MCL that enables the user to adjust the balance of the output clusterings. Such a feature is useful for two reasons. The first is that in some domains, e.g. clustering of protein interaction networks, domain experts have a clear idea of the desired distribution of sizes for the output clusters, and so in such a case, allowing the user to adjust the balance using a parameter is helpful. A second reason is that, clustering modern graphs purely on the basis of topology can yield extremely imbalanced clustering arrangements, even if such clustering arrangements achieve excellent scores on conventional quality metrics such as conductance. Our new algorithm now has an additional parameter b which can be tuned by the user depending on the desired amount of balance for the user. Setting $b = 0$ recovers the original MLR-MCL, and hence this new algorithm can be seen as a generalized version of the original. The new algorithm achieves excellent results in terms of both quality and running time on range of graphs such as protein interaction networks, synthetic graphs, Wikipedia network and nearest neighbor graphs of text documents.

In the next chapter, we will see how we can further address the issue of complicated topology in modern graphs via the use of a simple pre-processing step.

Chapter 5: Pre-processing graphs using Local Graph Sparsification

In Chapters 3, 4, we discussed algorithms that can cluster an undirected, possibly weighted graph. In this chapter and succeeding chapters, we will discuss *pre-processing* algorithms that can prepare a graph which has clear similarity structure at the local scale. Such graphs are then easy to be clustered using either MLR-MCL, discussed in previous chapters, or using other clustering algorithms, such as those discussed in Section 2.1.2.

Specifically, in this chapter, we will discuss *sparsification* strategies for undirected, unweighted networks. Sparsification refers to the process of reducing the size of the graph by removing edges (but not nodes). The basic premise is that operating on a smaller scale version of the problem will realize approximately similar results to processing the entire dataset, at a fraction of the execution time. At a broad level, this premise has been exploited for various machine learning problems and data mining problems in the past [93]. Related work in this direction can be found in Section 2.2. The sparsification we will discuss in this chapter is primarily aimed at scaling up graph clustering algorithms. Our goal is to sparsify the graph in such a way that the cluster structure is retained or even enhanced in the sparsified graph. We are not proposing a new graph clustering algorithm; rather we aim to transform the graph in such a way that will help almost any existing graph clustering algorithm, in terms of speed and at little to no cost in accuracy. Our sparsified graphs can also help in visualizing the cluster structure of the original graph.

An example illustrating the results that can be obtained using our proposed sparsification algorithm is shown in Figure 5.1. The original graph (Figure 5.1(a)) consists of 30 nodes and 214 edges, with the vertices belonging to 3 clusters (indicated by the color coding). The result of sparsifying this graph using our proposed sparsification algorithm is shown in Figure 5.1(b). Note that the sparsified graph only contains 64

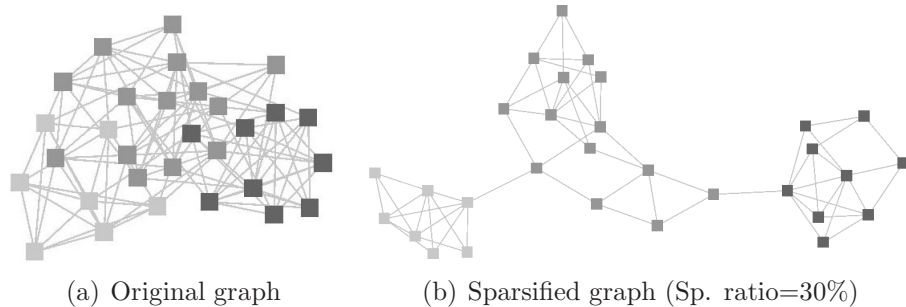


Figure 5.1: Proposed method on an example graph with 30 vertices and 3 clusters.

of the original 214 edges, and the cluster structure of the graph is also much clearer in this new sparsified graph.

The main idea behind our sparsification algorithm is to preferentially retain the edges that are likely to be part of the same cluster. We use a similarity-based sparsification heuristic, according to which edges that connect nodes which have many common neighbors are retained. Using a global similarity threshold for selecting which edges to retain in a graph, however, is problematic since different clusters may have different densities, and using a global threshold may choose far many more edges in the denser clusters, disconnecting the less dense clusters. To overcome this problem, we propose a *local* sparsification algorithm which chooses a few top edges per node, and thereby avoids this problem by using a locally appropriate threshold for including the edges in the sparsified graph. Local sparsification also ensures that all nodes in the graph are covered, i.e. there is at least one edge incident on each node in the sparsified graph. Specifically, we retain d^e edges for a node of degree d , where e is a parameter that helps control the overall sparsification ratio of the result. We analyze the characteristics of the resulting sparsified graph in terms of the degree distribution and the final sparsification ratio in terms of the parameter e .

Since exact similarity computation between each pair of connected nodes is very expensive, we efficiently approximate the similarity by hashing via *minwise independent permutations* [20] (minwise hashing). Estimating similarity for all pairs of connected nodes in the graph using minhashing is linear in the number of edges in the graph, allowing us to sparsify the graph very quickly.

We evaluate the performance of our proposed local sparsification algorithm (named **L-Spar**) on several real networks such as Wikipedia, Twitter, Orkut, Flickr and also biological networks with ground truth. Our main evaluation method is to compare the quality of clusters obtained from the original graph with the quality of those obtained from the sparsified graph, as well as compare the respective execution times and balance of the resulting cluster sizes. In terms of baselines, we compare against random sampling [62, 10, 2], sampling based on the ForestFire model [73] and the global similarity sparsification (named **G-Spar**) that we initially propose. We examine the performances of the different sparsifications when coupled with different state-of-the-art clustering algorithms - in addition to MLR-MCL discussed in previous chapters, we use Metis [63], Metis+MQI [72] and Graclus [36]). We summarize our key findings:

- For different networks and different clustering algorithms, L-Spar sparsification enables clustering that is, in terms of quality, comparable to the clustering result obtained from the original graph, despite containing far fewer edges (typically 10-20%), and significantly superior to the result obtained by clustering the baseline sparsifications. Indeed, clustering our sparsified graph often improves upon the clustering obtained from the original graph - for example, the F-score of Metis improves by 50% when clustering the sparsified Wiki graph. Similarly, Metis+MQI outputs a giant core cluster containing 85% of the nodes when clustering the original Flickr graph, while the clustering result obtained using the sparsified graph is much more meaningful.
- Clustering the L-Spar sparsified graph as well as the sparsification itself are extremely fast. Metis for instance obtains a 52x speedup (including clustering and sparsification times) on the Wiki graph. MLR-MCL similarly obtains speedups in the range of 20x.
- Examples of the kinds of edges that are retained and those that are discarded in the sparsification process confirm that the algorithm is able to discern noisier/weaker connections from the stronger/semantically closer connections.
- We also systematically vary the average degree and “mixing”-ness of the clusters of a synthetic graph generator [70] and find that our method becomes increasingly effective with higher average degrees as well as with higher mixing parameters.

5.1 Similarity sparsification

We aim to design a method for sparsifying a graph such that:

1. Clustering the sparsified graph should be much faster than clustering the original graph.
2. The accuracy of the clusters obtained from the sparsified graph is close to the accuracy of the clusters obtained from the original graph.
3. The sparsification method itself should be fast, so that it is applicable to the massive networks where it is most beneficial.

Our main approach to sparsification is to preferentially retain intra-cluster edges in the graph compared to inter-cluster edges, so that the cluster structure of the original graph is preserved in the sparsified graph. Of course, if a graph possesses cluster structure at all, then the majority of the edges in the graph will be contained inside clusters, and so to achieve any significant sparsification, one needs to inevitably discard some intra-cluster edges. Nevertheless, as long as a greater fraction of the inter-cluster edges are discarded, we should expect to still be able to recover the clusters from the original graph. The critical problem here, then is to be able to *efficiently* identify edges that are more likely to be within a cluster rather than between clusters.

Prior work has most commonly used various edge centrality measures in order to identify edges in the sparse parts of the graph. The edge betweenness centrality [89] of an edge (i, j) , for example, is proportional to the number of shortest paths between any two vertices in the graph that pass through the edge (i, j) . Edges with high betweenness centrality are “bottlenecks” in the graph, and are therefore highly likely to be the inter-cluster edges. However the key drawback of edge betweenness centrality is that it is prohibitively expensive, requiring $O(mn)$ time [89] (m is the number of edges and n is the number of nodes in the graph).

For this reason, we propose a simpler heuristic for identifying edges in sparse parts of the graph.

Similarity-based Sparsification heuristic

An edge (i, j) is likely to (not) lie within a cluster if the vertices i and j have adjacency lists with high (low) overlap.

The main intuition here is that the greater the number of common neighbors between two vertices i and j connected by an edge, the more likely it is that i and

j belong to the same cluster. Another way of looking at it is that an edge that is a part of many triangles is probably in a dense region i.e. a cluster. We use the Jaccard measure to quantify the overlap between adjacency lists. Let $Adj(i)$ be the adjacency list of i , and $Adj(j)$ be the adjacency list of j . For simplicity, we will refer to the similarity between $Adj(i)$ and $Adj(j)$ as the similarity between i and j itself.

$$Sim(i, j) = \frac{|Adj(i) \cap Adj(j)|}{|Adj(i) \cup Adj(j)|} \quad (5.1)$$

Global Sparsification

Based on the above heuristic, a simple recipe for sparsifying a graph is given in Algorithm 8. For each edge in the input graph, we calculate the similarity of its end points. We then sort all the edges by their similarities, and return the graph with the top $s\%$ of all the edges in the graph (s is the sparsification parameter that can be specified by the user). Note that selecting the top $s\%$ of all edges is the same as setting a similarity threshold (that applies to all edges) for inclusion in the sparsified graph.

Algorithm 8 Global Sparsification Algorithm

Input: Graph $G = (V, E)$, Sparsification ratio s

$G_{sparse} \leftarrow \emptyset$

for each edge $e=(i,j)$ in E **do**

$e.sim = Sim(i, j)$ according to Eqn 5.1

end for

Sort all edges in E by $e.sim$

Add the top $s\%$ edges to G_{sparse}

return G_{sparse}

However, the approach given in Algorithm 8 has a critical flaw. It treats all edges in the graph equally, i.e. it uses a global threshold (since it sorts all the edges in the graph), which is not appropriate when different clusters have different densities. For example, consider the graph in Figure 5.2(a). Here, the vertices $\{1, 2, 3, 4, 5, 6\}$ form a dense cluster, while the vertices $\{7, 8, 9, 10\}$ form a less dense cluster. The sparsified graph using Algorithm 8 and selecting the top 15 (out of 22) edges in the graph is shown in Figure 5.2(b). As can be seen, all the edges of the first cluster have been

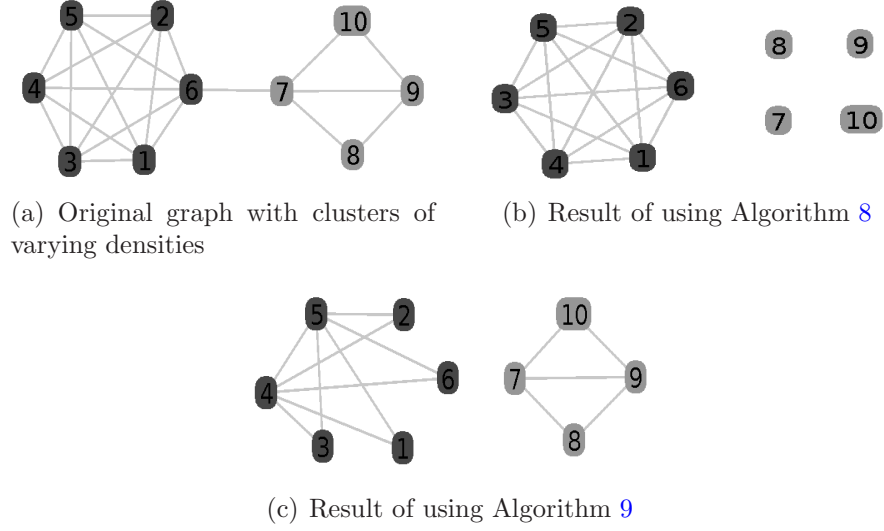


Figure 5.2: Global vs. local sparsifications

retained, while the second cluster consisting of $\{7, 8, 9, 10\}$ is completely empty, since all the edges in the first cluster have higher similarity than any edge in the second cluster. Therefore, clustering the resulting graph will be able to recover cluster 1, but not cluster 2.

Therefore, we look to devise an alternative sparsification strategy that still relies on the similarity-based sparsification heuristic, but which can also handle situations when different clusters have different densities.

Local Sparsification

We solve the problem with Algorithm 8 described above, by avoiding the need to set a global threshold. Instead, for each node i , with degree d_i , we pick the top $f(d_i) = d_i^e$ edges incident to i , ranked according to similarity (Eqn. 5.1). Sorting and thresholding the edges of each node separately allows the sparsification to adapt to the densities in that specific part of the graph; furthermore, this procedure ensures that we pick at least one edge incident to each node. Here e ($e < 1$) is the local sparsification exponent that affects the global sparsification ratio, with smaller values of e resulting in a sparser final graph. The full algorithm is given in Algorithm 9.

The result of sparsifying the example graph (14 out of 22 edges) in Figure 5.2(a) using the local sparsification algorithm is shown in Figure 5.2(c). This graph reflects

Algorithm 9 Local Sparsification Algorithm

Input: Graph $G = (V, E)$, Local Sparsification exponent e

Output: Sparsified graph G_{sparse}

```
 $G_{sparse} \leftarrow \emptyset$ 
for each node  $i$  in  $V$  do
    Let  $d_i$  be the degree of  $i$ 
    Let  $E_i$  be the set of edges incident to  $i$ 
    for each edge  $e=(i,j)$  in  $E_i$  do
         $e.sim = Sim(i,j)$  according to Eqn. 5.1
    end for
    Sort all edges in  $E_i$  by  $e.sim$ 
    Add top  $d_i^e$  edges to  $G_{sparse}$ 
end for
return  $G_{sparse}$ 
```

the cluster structure of the original graph much better than Figure 5.2(b), since the cluster $\{7,8,9,10\}$ is still recoverable from the sparsified graph, unlike the one that was obtained by global sparsification.

Discussion

For a node of degree d , what is a good choice of the function $f(d)$ that tells us the right number of edges to retain? We want to retain at least one edge per node, and so we must have $f > 0$ and also $f(d) \leq d, \forall d$, since one cannot retain more edges than are already present.

We first observe that for hub nodes (i.e. nodes with high degree), a higher fraction of the incident edges tend to be inter-cluster edges, since such nodes typically tend to straddle multiple clusters. For this reason, we would prefer to sparsify nodes with higher degree more aggressively than nodes with lower degree. This implies that we want a strictly concave function f , which rules out linear functions of the sort $f(d) = c * d, c < 1$. Two prominent choices for strictly concave functions are $f(d) = \log d$ and $f(d) = d^e, e < 1$.

The advantage with $f(d) = d^e, e < 1$ is that we can control the extent of the sparsification easily using the exponent e , while retaining the concave property of the function. Further more, we have the following in the case of $f(d) = d^e$.

Proposition 7. *For input graphs with a power-law degree distribution with exponent α , the locally sparsified graphs obtained using Algorithm 9 also have a power-law degree distribution with exponent $\frac{\alpha+e-1}{e}$.*

Proof. Let D_{orig} and D_{sparse} be random variables for the degree of the original and the sparsified graphs respectively.

Since D_{orig} follows a power-law with exponent α , we have $p(D_{orig} = d) = Cd^{-\alpha}$ and the complementary CDF is given by $P(D_{orig} > d) = Cd^{1-\alpha}$ (approximating the discrete power-law distribution with a continuous one, as is common [30]).

From Algorithm 9, $D_{sparse} = D_{orig}^e$. Then we have

$$\begin{aligned} P(D_{sparse} > d) &= P(D_{orig}^e > d) = P(D_{orig} > d^{1/e}) \\ &= C (d^{1/e})^{1-\alpha} = Cd^{1-\frac{\alpha+e-1}{e}} \end{aligned}$$

Hence, D_{sparse} follows a power-law with exponent $\frac{\alpha+e-1}{e}$.

Let the cut-off parameter for D_{orig} be d_{cut} (i.e. the power law distribution does not hold below d_{cut} [30]), then the corresponding cut-off for D_{sparse} will be d_{cut}^e . $\square$

We can use Proposition 7 to prove the next one.

Proposition 8. *The sparsification ratio (i.e. the number of edges in the sparsified graph $|E_{sparse}|$, versus the original graph $|E|$), for the power law part of the degree distribution is at most $\frac{\alpha-2}{\alpha-e-1}$.*

Proof. Notice that the sparsification ratio is the same as the ratio of the expected degree on the sparse graph versus the expected degree on the original graph. We have, from the expressions for the means of power-laws [30]

$$\begin{aligned} E[D_{orig}] &= \frac{\alpha-1}{\alpha-2} d_{cut} \\ E[D_{sparse}] &= \frac{\frac{\alpha+e-1}{e} - 1}{\frac{\alpha+e-1}{e} - 2} \cdot d_{cut}^e \end{aligned}$$

Then the sparsification ratio is

$$\frac{\frac{\frac{\alpha+e-1}{e} - 1}{\frac{\alpha+e-1}{e} - 2}}{\frac{\alpha-1}{\alpha-2}} \cdot d_{cut}^{e-1} \leq \frac{\frac{\frac{\alpha+e-1}{e} - 1}{\frac{\alpha+e-1}{e} - 2}}{\frac{\alpha-1}{\alpha-2}} = \frac{\alpha-2}{\alpha-e-1}$$

For a fixed e and a graph with known α , this can be calculated in advance of the sparsification. $\square$

Many real graphs are known to follow power-laws in the range $2 < \alpha < 3$, and assuming an $\alpha = 2.1$ and $e = 0.5$, the sparsification ratio will be less than 17%, according to the calculations above. Higher values of α (i.e. steeper power-law graphs) yield higher sparsification ratios, as do higher values of the exponent e (as expected).

Time complexity

The main component in the running time for both the Global and Local Sparsification is the computation of the similarities on each edge according to Eqn 5.1. Assuming the adjacency lists for each node are pre-sorted, intersecting the adjacency lists of two nodes i and j with degrees d_i and d_j takes number of operations proportional to $d_i + d_j$. Since a node of degree d_i requires d_i intersections, the total number of operations is proportional to $\sum_i d_i^2$. This is at least $n \cdot d_{avg}^2$ (by Jensen's inequality), where d_{avg} is the average degree of the graph, which is prohibitively large for most graphs.

We suggest a faster, approximate method next.

5.1.1 Minwise Hashing for Fast Similarity

Hashing by minwise independent permutations, or minwise hashing, is a popular technique for efficiently approximating the Jaccard similarity between two sets, first introduced by Broder et al. [20]. Minwise hashing has been used previously in problems such as compressing web graphs [23], discovery of dense subgraphs [51] and local triangle counting [13]. Our use of minwise hashing is largely orthogonal to these existing efforts as it has a completely different goal – as a simple mechanism to speed up graph sparsification, and eventually to scale up complete graph clustering.

Given two sets A and B , and a permutation π on the space of the universal set, randomly chosen from a family of minwise independent permutations [20], we have that the first element of set A under the permutation π is equal to the first element of set B under π with probability equal to their (Jaccard) similarity :

$$Pr(min(\pi(A)) = min(\pi(B))) = \frac{|A \cap B|}{|A \cup B|} \quad (5.2)$$

Based on Equation 5.2, a simple estimator for $\text{sim}(A, B)$ is $I[\min(\pi(A)) = \min(\pi(B))]$, where $I[x]$ is the indicator variable i.e. $I[x] = 1$ if x is true and 0 otherwise.

Proposition 9. $I[\min(\pi(A)) = \min(\pi(B))]$ is an unbiased estimator for $\text{sim}(A, B)$, with variance $\text{sim}(A, B) * (1 - \text{sim}(A, B))$

Proof. The estimator in question is a Bernoulli random variable, with probability of success $\text{sim}(A, B)$. The proposition follows. $\square$

The variance of the above estimator can be reduced by taking multiple independent minwise permutations. Let $\pi_i, i = 1 : k$ be k independent minwise permutations on the universal set. Let $mh_i(A) = \min(\pi_i(A))$ for any set A , i.e. $mh_i(A)$ represents the minimum element of A under the permutation π_i (i.e. it is the i^{th} “minhash” of A). We can construct a length- k signature for a set A consisting of the k minhashes of A in order - note that the signature is itself not a set, and the order is necessary for the similarity estimation.

A better estimator for $\text{sim}(A, B)$ then is

$$\hat{\text{sim}}(A, B)_k = \frac{1}{k} \left(\sum_{i=1}^k I[mh_i(A) = mh_i(B)] \right) \quad (5.3)$$

Proposition 10. $\hat{\text{sim}}(A, B)_k$ is an unbiased estimator for $\text{sim}(A, B)$, with the variance inversely proportional to k .

Proof. $\hat{\text{sim}}(A, B)_k$ is the average of k unbiased estimators, and therefore by the linearity of expectation is itself unbiased. It is easy to show the variance is $\frac{\text{sim}(A, B) * (1 - \text{sim}(A, B))}{k}$, and thus inversely proportional to k . $\square$

For the approximation of $\text{sim}(A, B)$ using the estimator in Eqn 5.3 to be fast, we still need an efficient way to generate random permutations. Since the space of all permutations is exponentially large, drawing a permutation uniformly at random from this space can be expensive. For this reason, it is typical to use *approximate* minwise independent permutations such as *linear permutations* [17]. A linear permutation is specified by a triplet (a, b, P) where P is a large prime number and a is an integer drawn uniformly at random from $[1, P-1]$ and b is an integer drawn uniformly at random from $[0, P-1]$; the permutation of an input $i \in [0, P-1]$ is given by $\pi(i) = (a * i + b) \% P$. Multiple linear permutations can be generated by generating random pairs of (a, b) . Since all we care about is only the minimum element under

the permutation π , minhashing a set using linear permutations simply involves one pass over the elements of the set, where we maintain a variable that keeps track of the minimum element seen thus far in the scan.

Local Sparsification using Minwise Hashing

We first generate k linear permutations, by generating k triplets (a, b, P) . Next, we compute a length k signature for each node by minhashing each node k times - more precisely, we minhash the adjacency list of each node in the graph k times - and fill up a hashtable of size $n * k$. For an edge (i, j) , we compare the signatures of i and j , minhash-by-minhash, and count the number of matching minhashes. Since the similarity of an edge is directly proportional to the number of matches, we can sort the edges incident to a node i by the number of matches of each edge. Furthermore, since the number of matches is an integer between 0 and k , one can use counting sort to sort the edges incident on a node in linear time. After the sort, we pick the top d_i^e edges (d_i is the degree of node i) for inclusion in the sparsified graph.

The implementation of global sparsification using minwise hashing is similar and is omitted.

Time complexity

Local sparsification using minwise hashing is extremely efficient. Generating one minwise hash of a set using linear permutation requires one scan of the set, therefore generating a minwise hash of a single node i takes $O(d_i)$ time. The time to generate one minwise hash for all nodes takes time $O(m)$, where m is the number of edges in the graph, and generating k minwise hashes for all nodes in the graph takes $O(km)$ time. Estimation of similarity for an edge (i, j) only takes time $O(k)$, since we only need to compare the length- k signatures of i and j . The estimation of similarity for all edges takes time $O(km)$ time. Sorting the edges incident on each node by similarity can be done in linear time using counting sort, since we only sort the number of matches which lie in $\{0, \dots, k\}$. Hence, sorting the edges for all the nodes requires time $O(m)$. In sum, the time complexity for local (or global) sparsification using minwise hashing is $O(km)$ i.e. linear in the number of edges.

Dataset	Vertices	Edges	d_{avg}	$ C (size_{avg})$
BioGrid	5,964	192,851	64.7	700(9)
DIP	4,741	15,147	6.4	700(7)
Human	11,750	63,419	10.8	900(13)
Wiki	1,129,060	53,021,551	93.9	10000(113)
Orkut	3,072,626	117,185,083	76.3	15000(205)
Twitter	146,170	83,271,147	1139.4	1500(97)
Flickr	31,019	520,040	33.5	300(103)

Table 5.1: Dataset details.

5.2 Empirical Evaluation

5.2.1 Datasets

We perform experiments on seven real-world networks, including information networks, social networks and protein-protein interaction networks (also see Table 5.1):

1. **Yeast-BioGrid (BioGrid):** This is a protein-protein interaction (PPI) network of Yeast, obtained from the BioGrid database. This network has a relatively high average degree of 65, with many noisy interactions.
2. **Yeast-DIP (DIP):** This is also a PPI network of Yeast, obtained from the Database of Interacting Proteins (DIP). The interactions in this network are more carefully derived (compared to BioGrid), resulting in a much lower average degree of 6.
3. **Human-PPI (Human):** The third network is a Human PPI network, obtained from iRefIndex.
4. **Wikipedia (Wiki):** This is an undirected version of the graph of hyperlinks between Wikipedia articles (from Jan–2008). The original downloaded corpus had nearly 12 million articles, but a lot of these were insignificant or noisy articles which we removed to obtain the final graph with 1.12M nodes and 53M edges.
5. **Orkut:** This is an anonymized friendship network crawled from Orkut [88]. This is our largest dataset consisting of more than 3M nodes and 117M edges.

6. **Twitter:** This is the Twitter follower-followee network collected by Kwak et al. [69]. We retain only users with at least 1000 followers, obtaining a network of 146,170 users, and we retain only edges (i, j) such that both i follows j and j follows i . The resulting number of edges in this graph is still very high - around 83M, with an average degree of 1140 per user.
7. **Flickr tags (Flickr):** We downloaded the tag information associated with ~6M photos from Flickr. We then built a tag network using the tags that were used by at least 5 distinct users, placing an edge between two tags if they were used together in at least 50 photos. The resulting network has 31019 tags with ~520K edges, and an average degree of 33.5 per tag.

Ground Truth

For evaluating the clustering results of the two Yeast PPI networks, we used the CYC2008 [94] protein complex data as the gold standard for ground truth validation. Note that only around 30% of the proteins in the PPI datasets have associated protein complex information, which means that the precision of any clustering result is at most 30%. For evaluating the results of the Human PPI dataset we used the CORUM protein complex data [96], which again has annotations for only 15% of the total proteins in the graph. For Wiki we prepared ground truth from the categories assigned at the bottom of the page by the editors. We removed many noisy categories; the final number of categories was 17950, covering around 65% of the articles in the dataset.

5.2.2 Baselines

We compare the following sparsification algorithms:

1. Our local similarity-based sparsification algorithm using minwise hashing (which we refer to as **L-Spar**, short for Local-Sparsifier, henceforth). It takes two parameters, k referring to the number of minhashes, and e referring to the sparsification exponent.
2. The global similarity-based sparsification algorithm using minwise hashing, referred to as **G-Spar** henceforth.
3. Random Edge Sampling [73, 62, 2, 10], that selects $r\%$ of the edges from the graph uniformly at random.

4. ForestFire [73] that selects a seed node at random and recursively burns edges and nodes in its vicinity, with burn probability $r\%$. We repeatedly initiate fires choosing an as-yet unburned seed node so as to cover most of the nodes in the graph.⁷

We set the parameters for the baselines 2-4 above so as to achieve the same sparsification ratio as that achieved using L-Spar. We use a default sparsification exponent of $e = 0.5$ (unless otherwise mentioned), and default number of min-hashes $k=30$.

5.2.3 Evaluation method

Our primary quantitative evaluation method is to cluster the original and the sparsified graphs, and to assess the quality of the clusterings either w.r.t. the ground truth or w.r.t. the structure of the (original) graph. We use four state-of-the-art algorithms for clustering the original and sparsified graphs - Metis [63], Metis+MQI [72], MLR-MCL [98] and Graclus [36].⁸ All the experiments were performed on a PC with a Quad Core Intel i5 CPU, each with 3.2 GHz, 16 GB RAM and running Linux. The output number of clusters for each network, along with the average cluster size, is provided in Table 5.1.⁹ We used a small average cluster size for the protein networks since most protein complexes are in the size range of 5-15. The average cluster sizes for the other networks is around 100, which is on average the right size for communities in such networks [74] (in the case of Orkut, we had to reduce the number of clusters as Metis and Metis+MQI would run out of memory otherwise).

When we have ground truth for a network, we measure cluster quality using the average F-score of the predicted clusters, the computation of which is described in Section 2.1.1. In the absence of ground truth, we use average ncuts, or conductance, denoted by ϕ ; again see Section 2.1.1 for how this is defined. There are two important points to note about the way we measure conductances: (i) For the clusters obtained from the sparsified graph, we cannot simply measure the conductance using the very same sparsified graph, since that would tell us nothing about how well the sparsified

⁷Note that a burned node may still remain a singleton if none of its edges get burned in the probabilistic burning process.

⁸We downloaded the softwares from the respective authors' webpages, except for Metis+MQI which we re-implemented using Metis and hipr as black-box modules. The versions are: Metis 4.0, Graclus 1.2, MLR-MCL 1.1 and hipr 3.4. We thank Kevin Lang for help with Metis+MQI.

⁹The number of clusters is only indirectly specifiable in MLR-MCL, and hence the final number of clusters in the case of MLR-MCL varied slightly from that reported in Table 5.1.

graph retained the cluster structure in the original graph. Therefore, we report the conductances of the clusters obtained from the sparsified graphs also using the *structure of the **original** graph*. (ii) The G-Spar, RandomEdge and ForestFire (but not L-Spar) often isolate a percentage of nodes in the graph in their sparsification. We *do not* include the contribution to the final average conductance arising from such singleton nodes. Note that this biases the comparison based on conductance against the results from the L-Spar and the original graph and in favor of the baselines.

Since low conductances can often be achieved by very imbalanced clusterings [74], we also report on the balance of a clustering arrangement using the *coefficient of variation* c_v of the cluster sizes, which is the ratio of the standard deviation of the cluster sizes to the average cluster size (i.e. $c_v = \sigma/\mu$). Therefore, a lower c_v represents a more balanced clustering.

5.2.4 Results

The results obtained using the various sparsification approaches as well as the original graph and using the different clustering algorithms are presented in Table 5.2.4. The sparsification ratios obtained using L-Spar by setting $e = 0.5$ vary for different networks depending on their specific degree distributions, but they never exceed 0.2 for the five biggest graphs - this means that for those graphs, the sparsified graphs contain at most a fifth of the edges in the original graph. The speedups are w.r.t. clustering the original graph, and take into account both sparsification as well as clustering times.

Let us first consider the results using Metis. For all four datasets with ground truth, clustering the L-Spar graph actually gives better results than clustering the original graph. This suggests that the removal of noisy edges by L-Spar helps Metis discover clusters that it was previously unable to. The Wiki graph provides a particularly stark example, where the F-score improves from 12.34 to 18.47, while also executing 52x times faster. Similarly on Orkut, the average conductance for the clusters discovered from the L-Spar graph is 0.76 as compared to 0.85 for the clustering from the original graph. On Flickr, the G-Spar clustering has a much lower conductance (0.71) than either the original or L-Spar, but on the flip side, we found that G-Spar introduced around 30% singletons in the sparsified graph. The speedups are 36x, 6x and 52x for our three biggest graphs (Orkut, Twitter and Wiki).

Looking at the results using MLR-MCL, we can see that L-Spar enables clusterings that are of comparable quality to clustering the original graph, and are much better compared to the other sparsifications. On Wiki, for instance, L-Spar enables an F-score of 19.3, far superior to that obtained with the other baselines, and quite close to the original F-score of 20.2. On Orkut, Flickr and Twitter - datasets where we have no ground truth - clustering the L-Spar graph results in clusters with better ϕ_{avg} (on the original graph), and also better balance (i.e. lower c_v). Furthermore, L-Spar followed by clustering is at least 22x faster on our three biggest graphs (Orkut, Twitter, Wiki), bringing the total times down from the order of hours down to the order of minutes.

Coming to the results obtained using Metis+MQI, we see results that are of a similar flavor to those obtained using MLR-MCL. On the four graphs with ground truth, the accuracy of clustering either the original graph or the L-Spar graph are very much comparable, and the other three sparsifications fare worse in comparison. On Orkut

and Twitter, we similarly find the results from clustering either the original graph or the L-Spar graph comparable. On the Flickr dataset, clustering the original graph results in a low ϕ_{avg} of 0.55, but only at the cost of high imbalance ($c_v=14.1$), with 85% of the nodes being placed in one giant cluster. The clusters from the L-Spar graph are much more balanced ($c_v=1.0$). Some examples of clusters obtained from the L-Spar graph that were previously merged together in the giant cluster are :{astronomy, telescope, astrophotography, griffithobservatory, solarsystem}, {arcade, atlanticcity, ac, poker, slots}, {historicdistrict, staugustine, staugustineflorida, castillodesanmarcos}. Looking at the speedups, we find a counter-intuitive result - Metis+MQI executes slower on the L-Spar graph for Wiki and Orkut. The explanation is as follows. Metis+MQI recursively bipartitions the graph, operating on the currently biggest partition at each step. The running time for Metis+MQI therefore depends a lot on the step at which Metis+MQI discovers a nearly balanced bipartition (since that will halve the size of the partition on which Metis+MQI subsequently needs to operate on). On Wiki, we found that a balanced bipartition is found at ~ 150 th step on the original graph, versus at ~ 1300 th step on the L-Spar graph, causing almost all the difference in speed we see. However, the balance of the clustering arrangement (given by c_v) from L-Spar at the end of all the recursive bipartitions is at least as good as the balance of the clustering of the original graph (much better for some graphs, as the example of Flickr above showed). Note that we still obtain a 14x speedup on the ~ 80 M edge Twitter graph.

In the results for Graclus, we see that L-Spar enables more accurate clusterings than the original graph on the three biological datasets ¹⁰. For Flickr, we note that ϕ_{avg} for the three baseline sparsifications are close to 0.99 if we include the contribution of singletons, and the shown ϕ_{avg} s are comparable to those obtained using L-Spar only because their contributions were not included. On Twitter, we find that L-Spar enables ϕ_{avg} that is comparable to clustering the original graph, with better balance and 5x faster.

We would like mention to an interesting cross-cutting result beyond the results showed in Table 5.2.4. The time taken to cluster the L-Spar and G-Spar sparsified graphs is typically smaller than the time taken to cluster the other baseline sparsifications (RandomEdge and ForestFire), although the reverse trend holds true for the sparsification itself. For a representative example, on the Wiki dataset, Metis requires

¹⁰Graclus ran out of memory for both Wiki and Orkut and could not be executed.

80 seconds to cluster the L-Spar sparsified graph and 8 seconds to cluster the G-Spar one, compared to 940 seconds for RandomEdge and 1040 seconds for ForestFire. This difference in clustering times, despite the fact that all the sparsified graphs contain almost the *same number of edges*, is because clustering algorithms generally tend to execute faster on graphs with clearer cluster structures.

5.2.5 Examining L-Spar sparsification in depth

Let us examine L-Spar sparsification in depth. The BioGrid protein interaction network consists mainly of edges derived from high-throughput interaction detection methods such as Yeast Two Hybrid analysis [32], which are well known to detect many false positive interactions. It is especially interesting that for this network, *all* four clustering algorithms enjoy better clustering accuracies on the L-Spar sparsified graph compared to the original graph, suggesting that the sparsification does remove many spurious interactions.

We provide some examples from the Wiki, Twitter and Flickr graphs in Table 5.3 that shine some light on the kinds of edges that are removed vis-a-vis the kinds that are retained. For the three Wikipedia examples of *machine learning*, *graph* and *Moby Dick*, the retained edges are clearly semantically closer to the example node, while the discarded edges are noise, arguably. For Twitter, we highlight the neighborhoods of Twitter founder Jack Dorsey, champion cyclist Lance Armstrong and conservative blogger Michelle Malkin. The retained neighbors for Dorsey are either high-ranking employees at Twitter (Stone, Williams, Goldman) or people with strong Silicon Valley connections (Dash, Lacy), while the discarded neighbors are more peripherally connected (Jet Blue Airways, Parul Sharma). The retained neighbors for Lance Armstrong are all professional bicyclists, while the discarded neighbors come from other backgrounds. For Michelle Malkin, the retained neighbors are Republican governors Bobby Jindal, Rick Perry and other conservatives or media personalities; clearly the discarded neighbors do not share as close a connection. The examples from Flickr similarly show tags that are semantically closer to the highlighted tag being retained, with the noisier tags being discarded.

Trend of speedup and f-score as e changes

Figures 5.3(a) and 5.3(b) report the trends of F-score and speedup on the Wiki dataset as e changes, using Metis. It is interesting that even with $e=0.3$, and retaining only 7% of the edges in the graph, Metis achieves an F-score of 17.73 on the sparsified

Clustering Algorithm: Metis											
Dataset	Original		Sparsified								
			Sp. Ratio	RandomEdge		G-Spar		ForestFire		L-Spar	
	F-score	Time (s)		F-score	Spdup	F-score	Spdup	F-score	Spdup	F-score	Spdup
BioGrid	17.78	3.02	0.17	15.98	11x	15.15	30x	16.18	9x	19.71	25x
DIP	20.04	0.11	0.53	17.58	2x	19.38	2x	15.41	2x	21.58	2x
Human	8.96	0.59	0.39	7.75	4x	8.64	4x	7.47	4x	10.05	5x
Wiki	12.34	7485	0.15	9.11	8x	9.38	104x	9.96	7x	18.47	52x
	$\phi_{avg}(c_v)$			$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$	
Orkut	0.85 (0.1)	14373	0.17	0.82 (0.4)	13x	0.76 (0.4)	30x	0.82 (0.4)	12x	0.76 (0.0)	36x
Flickr	0.87 (2.5)	4.7	0.2	0.91 (0.1)	8x	0.71 (0.1)	1x	0.91 (0.1)	9x	0.84 (0.3)	3x
Twitter	0.95 (0.1)	2307	0.04	1.0 (0.4)	35x	0.97 (0.0)	85x	0.99 (0.4)	14x	0.96 (1.7)	6x
Clustering Algorithm: MLR-MCL											
Dataset	Original		Sparsified								
			Sp. Ratio	RandomEdge		G-Spar		ForestFire		L-Spar	
	F-score	Time (s)		F-score	Spdup	F-score	Spdup	F-score	Spdup	F-score	Spdup
BioGrid	23.95	8.44	0.17	20.28	6x	18.29	38x	20.55	7x	24.90	17x
DIP	24.85	0.28	0.53	20.57	3x	22.45	3x	18.51	3x	24.38	3x
Human	10.55	1.68	0.39	8.81	4x	9.21	6x	8.37	4x	10.43	5x
Wiki	20.22	7898	0.15	8.74	19x	9.3	92x	11.59	14x	19.3	23x
	$\phi_{avg}(c_v)$			$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$	
Orkut	0.78 (6.4)	21079	0.17	0.85 (1.2)	6x	0.91 (10.1)	39x	0.86 (1.1)	6x	0.78 (0.5)	22x
Flickr	0.71 (0.6)	16.56	0.2	0.83 (2.2)	3x	0.72 (3.6)	2x	0.88 (1.9)	3x	0.70 (0.7)	4x
Twitter	0.90 (5.6)	14569	0.04	0.99 (0.6)	63x	0.89 (1.0)	188x	0.99 (11.0)	16x	0.86 (4.3)	22x
Clustering Algorithm: Metis+MQI											
Dataset	Original		Sparsified								
			Sp. Ratio	RandomEdge		G-Spar		ForestFire		L-Spar	
	F-score	Time (s)		F-score	Spdup	F-score	Spdup	F-score	Spdup	F-score	Spdup
BioGrid	23.16	4.0	0.17	19.76	11x	17.74	4x	19.13	11x	23.23	5x
DIP	23.09	0.32	0.53	19.55	1x	21.18	1x	16.09	2x	22.93	1x
Human	10.17	1.16	0.39	8.42	1x	9.1	1x	8.08	2x	10.28	1x
Wiki	19.21	35511	0.15	14.97	5x	9.98	360x	14.18	5x	18.32	0.46x
	$\phi_{avg}(c_v)$			$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$	
Orkut	0.756 (1.2)	19799	0.17	0.86 (0.5)	2x	0.77 (0.2)	1x	0.86 (0.3)	3x	0.755 (1.2)	0.7x
Flickr	0.55 (14.1)	72.35	0.2	0.68 (0.4)	3x	0.67 (0.3)	1x	0.70 (0.2)	5x	0.69 (1.0)	4x
Twitter	0.86 (0.6)	11708	0.04	0.99 (0.6)	35x	0.97 (0.0)	334x	0.99 (0.5)	19x	0.89 (0.6)	14x
Clustering Algorithm: Graclus											
Dataset	Original		Sparsified								
			Sp. Ratio	RandomEdge		G-Spar		ForestFire		L-Spar	
	F-score	Time (s)		F-score	Spdup	F-score	Spdup	F-score	Spdup	F-score	Spdup
BioGrid	19.15	0.32	0.17	17.59	4x	16.56	2x	16.67	2x	21.42	2x
DIP	21.77	0.19	0.53	18.27	2x	21.27	3x	15.59	5x	22.45	1x
Human	9.53	0.81	0.39	8.03	2x	8.75	5x	7.47	6x	9.90	1x
	$\phi_{avg}(c_v)$			$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$		$\phi_{avg}(c_v)$	
Flickr	0.66 (1.3)	1.35	0.2	0.72 (0.1)	2x	0.66 (0.1)	1x	0.71 (0.1)	2x	0.72 (1.7)	2x
Twitter	0.90 (2.4)	1518	0.04	1.0 (0.7)	138x	0.97 (0.0)	66x	0.99 (0.6)	138x	0.91 (0.9)	5x

Table 5.2: Quality and speedups after sparsification.

Node	Examples of retained edges (neighbors)	Examples of discarded edges (neighbors)
Wiki		
Machine Learning	Decision tree learning, Support Vector Machine, Artificial Neural Network, Predictive Analytics, Document classification	Co-evolution, Case-based reasoning, Computational neuroscience, Immune system
Graph (mathematics)	Graph theory, Adjacency list, Adjacency matrix, Model theory	Tessellation, Roman letters used in Mathematics, Morphism, And-inverter graph
Moby-Dick	Billy Budd, Clarel, Moby Dick (1956 film), Jaws (film), Western canon, Nathaniel Hawthorne	Bruce Sterling, 668, Dana Scully, Canadian English, Giant squid, Redburn
Twitter		
Jack Dorsey	Biz Stone, Evan Williams, Jason Goldman, Sarah Lacy, Anil Dash	Alyssa Milano, Parul Sharma, Nick Douglas, JetBlue Airways, Whole Foods Market
Lance Armstrong	Dave Zabriskie, Michael Rogers, Vande Velde, Levi Leipheimer, George Hincapie	Chris Sacca, Robin Williams, George Stephanopoulos, Alyssa Milano, Dr. Sanjay Gupta
Michelle Malkin	Bobby Jindal, Rick Perry, Howard Kurtz, Neal Boortz, George Stephanopoulos	Barack Obama, The Today Show, Jim Long, LA Times
Flickr		
gladiator	colosseum, worldheritage, site, colosseo, italy	europe, travel, canon, sky, summer
telescope	observatory, astronomy, telescopes, sutherland	2007, geotagged, travel, california, mountain
lincolnmemorial	memorials, reflectingpool, lincoln, nationalmall	travel, usa, night, sunset, america

Table 5.3: Examples of retained and discarded edges using L-Spar sparsification

graph, which is significantly better than the 12.34 on the original graph. The F-scores gradually increase with increasing e . Coming to the speedups, the biggest speedups are observed for smaller values for e - with an 81x speedup for $e=0.3$ (242x considering only the clustering times) - and the speedups gradually reduce to 13x for $e=0.7$.

Figures 5.3(c) and 5.3(d) report the corresponding trends for MLR-MCL and are quite similar. With $e=0.7$ (using 34% of the edges), clustering the sparsified graph delivers a better F-score than clustering the original graph, with a 35x speedup to boot.

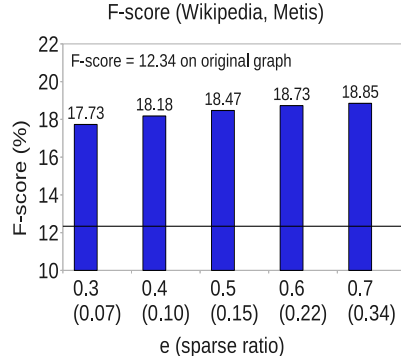
Speedup and F-score as k changes

Figures 5.3(e) and 5.3(f) show the effect of k , the number of min-wise hashes used for approximating similarities. We also include the result of sparsification using exact similarity calculations (denoted as ‘exact’). (We hold e constant at 0.5.) We see a gradual improvement in F-scores with increasing k , as the similarity approximation gets better with increasing k (refer Proposition 10). The F-scores obtained using the exact similarity calculation is 18.89, compared to the 18.47 obtained using $k=30$, which indicates that the similarity approximation works quite well. Also, exact similarity sparsification requires about 240x more time to compute compared to sparsification using minwise hashing with $k=30$ (4 hours vs 1 minute). Therefore we are gaining a lot in speedup by using minwise hashing while suffering little loss in quality. If we consider the clustering times alone, then clustering the exact sparsified graph is faster than clustering the minwise hashing sparsified graph with $k=30$ (50s vs 80s).

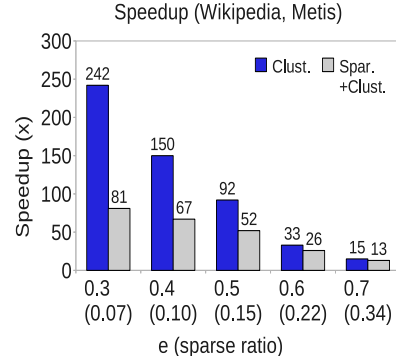
Figures 5.4(a) and 5.4(b) show the corresponding trends using MLR-MCL, which are quite similar.

Trends with varying degree and mixing parameter

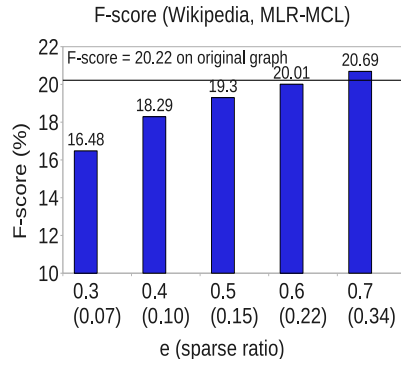
We examine the quality of the sparsification while varying the average degree of the graph and the “mixing”-ness of different clusters. We generated synthetic graphs using the LFR generator [70], with 10000 nodes and power-law exponent 2.1 for both degree distribution as well as community size distribution. The LFR generator allows the user to vary the average degree as well as the *mixing parameter* - a number between 0 and 1 which is effectively like the average conductance of the clusters; graphs with higher mixing parameters are more challenging to cluster accurately as the clusters are more mixed up in such graphs. Figures 5.4(c) and 5.4(d) show the change in F-scores for both original and sparsified graph using Metis+MQI



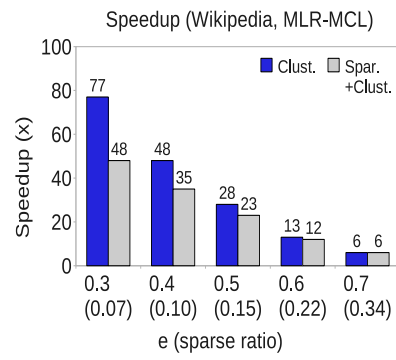
(a) F-score as e changes; Metis



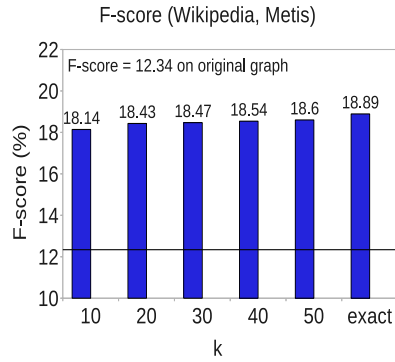
(b) Speedup as e changes; Metis



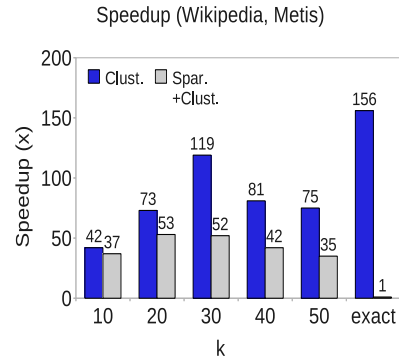
(c) F-score as e changes; MLR-MCL



(d) Speedup as e changes; MLR-MCL

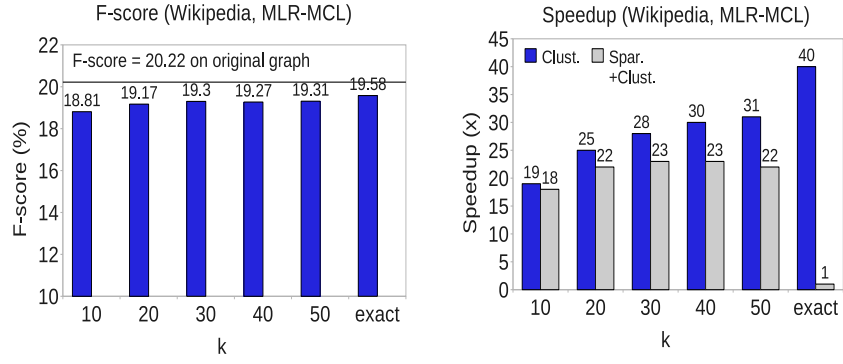


(e) F-score as k changes; Metis

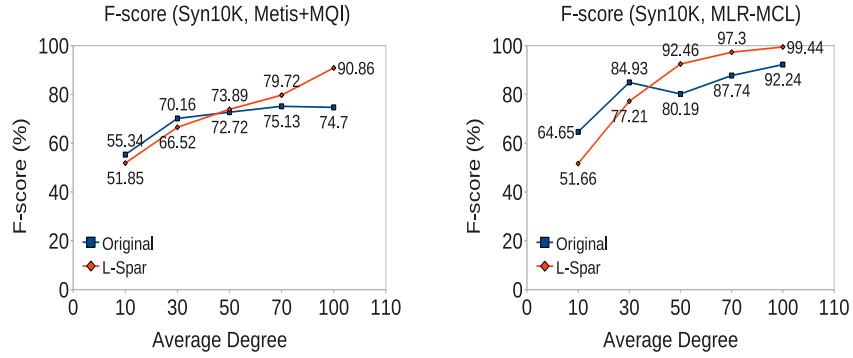


(f) Speedup as k changes; Metis

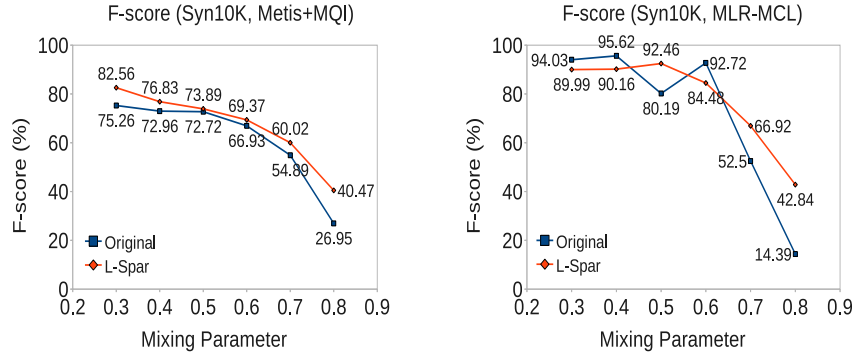
Figure 5.3: The performance of L-Spar under varying conditions



(a) F-score as k changes; MLR-MCL (b) Speedup as k changes; MLR-MCL



(c) F-score as average degree changes; Metis+MQI (d) F-score as average degree changes; MLR-MCL



(e) F-score as mixing parameter changes; Metis+MQI (f) F-score as mixing parameter changes; MLR-MCL

Figure 5.4: The performance of L-Spar under varying conditions

and MLR-MCL, while varying the average degree and keeping the mixing parameter constant at 0.5. The sparsification is more beneficial with increasing degree, and it actually outperforms the original clustering starting from degree 50. Figures 5.4(e) and 5.4(f) report the trend of F scores as we vary the mixing parameter keeping average degree to 50, using Metis+MQI and MLR-MCL. The performance on both original and sparsified graphs drops with increasing mixing parameter, which is to be expected. However, the sparsification does enable better performance with higher mixing parameters compared to clustering the original graph, suggesting that its ability to weed out irrelevant edges helps clustering algorithms. For instance, with the mixing parameter at 0.8, Metis+MQI achieves an F-score of 40.47 on the sparsified graph, improving upon the 26.95 achieved on the original graph.

5.2.6 Results on Randomized version of L-Spar

We experimented with a simple randomized version of L-Spar, which probabilistically samples edges in proportion to their (estimated) similarity. Interestingly, this approach achieves similar results to the deterministic version of L-Spar. For example, on the Yeast-DIP network, sparsifying using this sampling approach and subsequently clustering with MLR-MCL achieves an F-score of 24.21 which is comparable to the 24.35 obtained using the deterministic approach. Similarly, on Wiki, the sampling version of L-Spar enables an F-score of 19.99 using MLR-MCL, which is actually slightly higher than the 19.21 obtained using the deterministic approach.

The sampling approach is interesting because it enables one to build an ensemble of sparsified graphs for a given input graph by repeating the procedure multiple times with a different random seed. Robust models (including, but not restricted to, clustering) can then be built by averaging the models built on the ensemble, which is a common technique in statistical learning [49].

5.3 Discussion

We would like to begin by emphasizing that the local sparsification approach we present is extremely effective across the board for Metis, MLR-MCL and Graclus and often helpful for Metis+MQI. Accounting for local densities in the sparsification process provide significant gains across the board on all four clustering algorithms (seen by comparing results between global and local sparsification). In the case of Metis and Graclus, clearly observed for results with ground truth data, the local sparsification procedure has the added benefit of often improving the quality of the resulting partitions while also significantly speeding up the time to compute partitions. Another way of viewing this result is that these methods, particularly Metis, are less tolerant of noise in the data (presence of spurious edges often undermine their effectiveness in identifying good communities) and so post-sparsification where spurious edges are eliminated (see Table 5.3) the results are significantly better (e.g. Wiki). For MLR-MCL the general observation is that local sparsification results in a significant speedup at little or no cost to accuracy across the board. For Metis+MQI, as explained earlier the results are a bit mixed in the sense that we do find instances where the sparsification results in an overall slowdown to counter other instances where there is a significant speedup. We also note that the local sparsification mechanism, for all algorithms, typically results in clusters with significantly improved balance – an important criteria for many applications. Finally, the G-Spar sparsification mechanism seems well-suited for applications where the requirement is only to discover the densest clusters in the graph, although we have not directly tested this.

Out of Core Processing: L-Spar sparsification based on minwise hashing is well suited for processing large, disk-resident graphs and reducing their size so that they can be subsequently clustered in main memory. If the graph is stored as a sequence of adjacency lists (as is common practice with large graphs), minwise hashing to build length- k signatures for each node in the graph requires one sequential scan of the graph. The subsequent sparsification itself also requires only one sequential scan of the graph. Assuming the $n * k$ hashtable containing the signatures can fit in the main memory, the out-of-core version of L-Spar sparsification requires no disk I/O other than the two sequential scans. A blocked single pass algorithm may also be feasible under certain assumptions (e.g. dependent on degree distribution) and is currently under investigation.

5.4 Conclusion

In this chapter, we introduced an efficient and effective localized method to sparsify a graph i.e. retain only a fraction of the original number of edges. Our sparsification strategy outperforms baselines from previous literature, as well as enables clustering that is many times faster than clustering the original graph while maintaining at least as good quality as clustering the original graph. The proposed sparsification is thus an effective way to speed up graph clustering algorithms without sacrificing quality.

The methods proposed in this chapter are primarily aimed at large-scale graphs which are undirected and unweighted. In the succeeding chapters, we will consider pre-processing algorithms for input data that is of a different nature.

Chapter 6: Symmetrizations for Directed Graphs

In this chapter, we will discuss pre-processing algorithms that can transform directed graphs into a form that is suitable for clustering using off-the-shelf clustering algorithms. We next discuss why this is important.

Directed graphs are essential in domains where relationships between the objects may not be reciprocal i.e., there may be an implicit or explicit notion of directionality in the context of the complex system being modeled. A major challenge posed by directed graphs is that the nature of relationships captured by the edges in directed graphs is fundamentally different from that for undirected graphs. Consider a citation network where an edge exists from paper i to j if i cites j . Now i may be a paper on databases that cites an important result from the algorithmic literature (j). Our point is that paper i need not necessarily be similar to paper j . A common approach to handle directionality is to ignore it – i.e. eliminate directionality from edges and compute communities. In the above example that would not be the appropriate solution. Such a semantics of directionality is also evident in the directed social network of Twitter, where if a person i follows the feed of a person j , it tell us that i thinks the updates of j are interesting but says nothing about the similarity of i and j .

The central, and novel, insight of this chapter is that groups of vertices which share similar in-links and out-links make meaningful clusters in directed graphs. This is in direct contrast to previous research (summarized in Section 6.1) on clustering directed graphs, which either simply ignores the directionality of the edges or concentrates on new objective functions for directed graphs which do not take into account in-link and out-link similarity of the nodes. For detecting clusters with homogenous in-link and out-link structure, we suggest a two-stage framework; in the first stage, the graph is *symmetrized* i.e. transformed into an undirected graph, and in the second

stage, the symmetrized graph is clustered using existing state-of-the-art graph clustering algorithms. The advantages of the two-stage symmetrization framework are that (i) it is flexible - prior methods for directed graph clustering can also be equivalently expressed in this framework, (ii) it makes the underlying assumptions about which kinds of nodes should be clustered together explicit (i.e. the implicit similarity measure being used in the clustering), and (iii) it allows us to leverage the progress made in (undirected) graph clustering algorithms. We propose two novel symmetrization methods, *Bibliometric* and *Degree-discounted*. Bibliometric symmetrization sets the similarity between a pair of nodes as being the number of common in- and out-links between the two nodes. However, this approach does not work well with large scale power-law graphs, since the hub nodes in such graphs introduce many spurious connections in the symmetrized graph. To alleviate this problem, we propose *Degree-discounted* symmetrization which discounts the contribution of nodes according to their degree, and therefore eliminates or downweights such hub-induced connections in the symmetrized graph.

We perform evaluation on four real datasets, three of which (Wikipedia, LiveJournal and Flickr) have million plus nodes, and two (Wikipedia, Cora) of which have dependable ground truth for evaluating the resulting clusters. We examine the characteristics of the different symmetrized graphs in terms of their suitability for subsequent clustering. Our proposed Degree-discounting symmetrization approach achieves a 22% improvement in F scores over a state-of-the-art directed spectral clustering algorithm on the Cora dataset, and furthermore is two orders of magnitude faster. The degree-discounting symmetrization is also shown to enable clustering that is at least 4-5 times faster than other symmetrizations on our large scale datasets, as well as enabling a 12% qualitative improvement in Wikipedia. We also show examples of the clusters that our symmetrization enables recovery of in the Wikipedia dataset; such clusters validate our claim that interconnectivity is not the only criterion for clusters in directed graphs, and that in-link and out-link similarity is important as well. Ours is, to the best of our knowledge, the first comprehensive comparison of different graph symmetrization techniques.

In summary, the contributions of this chapter are as follows:

1. We argue and provide evidence for the merits of an explicit symmetrization-based approach to clustering directed graphs. This is in contrast to recent

work which attempted to design specialized spectral algorithms with limited scalability.

2. We propose the Bibliometric and Degree-discounted symmetrizations that take into account in-link and out-link similarities (which existing directed graph clustering approaches do not), with Degree-discounted also appropriately down-weighting the influence of hub nodes.
3. We extensively compare the different symmetrizations, as well as a state-of-the-art directed graph clustering algorithm, on real world networks, providing empirical evidence for the usefulness of our proposed approaches.

6.1 Prior work

6.1.1 Normalized cuts for directed graphs

Many popular methods for clustering undirected graphs search for subsets of vertices with low *normalized cut* [61, 87, 104] (or *conductance*[61], which is closely related). The normalized cut of a group of vertices $S \subset V$ is defined as[104, 87]

$$Ncut(S) = \frac{\sum_{i \in S, j \in \bar{S}} A(i, j)}{\sum_{i \in S} degree(i)} + \frac{\sum_{i \in S, j \in \bar{S}} A(i, j)}{\sum_{j \in \bar{S}} degree(j)} \quad (6.1)$$

where A is the (symmetric) adjacency matrix and $\bar{S} = V - S$ is the complement of S . Intuitively, groups with low normalized cut are well connected amongst themselves but are sparsely connected to the rest of the graph.

The connection between random walks and normalized cuts is as follows [87] : $Ncut(S)$ in Equation 6.1 is the same as the probability that a random walk that is started in the stationary distribution will transition either from a vertex in S to a vertex in $\bar{S}$ or vice-versa, in one step [87]

$$Ncut(S) = \frac{Pr(S \rightarrow \bar{S})}{Pr(S)} + \frac{Pr(\bar{S} \rightarrow S)}{Pr(\bar{S})} \quad (6.2)$$

Using the unifying concept of random walks, Equation 6.2 have been extended to directed graphs by Zhou et. al. [126] and Huang et. al. [58]. Let P be the transition matrix of a random walk on the directed graph, with π being its associated stationary distribution vector (e.g. PageRank vector) satisfying $\pi P = \pi$. The probability that a random walk started in the stationary distribution traverses a particular directed edge $u \rightarrow v$ is given by $\pi(u)P(u, v)$. The $Ncut$ of a cluster S is again the probability of a random walk transitioning from S to the rest of the graph, or from the rest of the graph into S in one step:

$$Ncut_{dir}(S) = \frac{\sum_{i \in S, j \in \bar{S}} \pi(i)P(i, j)}{\sum_{i \in S} \pi(i)} + \frac{\sum_{j \in \bar{S}, i \in S} \pi(j)P(j, i)}{\sum_{j \in \bar{S}} \pi(j)} \quad (6.3)$$

Meila and Pentney [86] introduce a general class of weighted cut measures on graphs, called *WCut*, parameterized by the vectors T, T' and the matrix A :

$$WCut(S) = \frac{\sum_{i \in S, j \in \bar{S}} T'(i) A(i, j)}{\sum_{i \in S} T(i)} + \frac{\sum_{j \in \bar{S}, i \in S} T'(j) A(j, i)}{\sum_{j \in \bar{S}} T(j)} \quad (6.4)$$

Different *NCut* measures can be recovered from the above definition by plugging in different values for T, T' and A , including the definitions for *NCut* and *NCut_{dir}* given above.

All of the above work minimizes these various cut measures via spectral clustering i.e. by post-processing the eigenvectors of suitably defined *Laplacian* matrices. The Laplacian matrix $\mathcal{L}$ for *NCut_{dir}*, e.g., is given by [126, 58, 28]

$$\mathcal{L} = I - \frac{\Pi^{1/2} P \Pi^{-1/2} + \Pi^{-1/2} P' \Pi^{1/2}}{2} \quad (6.5)$$

where P is the transition matrix of a random walk, and Π is a diagonal matrix with $diag(P) = \pi$, π being the stationary distribution associated with P .

Drawbacks of normalized cuts for directed graphs

A common drawback of the above line of research is that there exist meaningful clusters which do not necessarily have a low directed normalized cut. The prime examples here are groups of vertices which do not point to one another, but all of which point a common set of vertices (which may belong to a different cluster) We present an idealized example of such situations in Figure 1, where the nodes 4 and 5 can be legitimately seen as belonging to the same cluster, and yet the *NCut_{dir}* for such a cluster will be high (the probability that a random walk transitions out of the cluster $\{4, 5\}$ to the rest of the graph, or vice versa, in one step, is very high.) Such situations may be quite common in directed graphs. Consider, for example, a group of websites that belong to competing companies which serve the same market; they may be pointing to a common group of websites outside themselves (and, similarly be pointed at by a common group of websites), but may not point at one another for fear of driving customers to a competitor's website. Another example may be a group of research papers on the same topic which are written within a short span of time and therefore do not cite one another, but cite a common set of prior work and

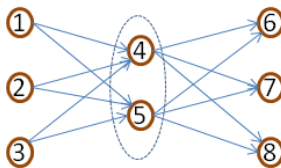


Figure 6.1: Toy example illustrating limitations of prior work.

are also in the future cited by the same papers. We present real examples of such clusters in Section 6.4.7.

Another drawback of the above line of research is the poor scalability as a result of the dependence on spectral clustering (except for Andersen et. al. [6] who use local partitioning algorithms). We further discuss this issue in Section 6.2.2.

6.1.2 Bibliographic coupling and co-citation matrices

The *bibliographic coupling* matrix was introduced by Kessler [65], in the field of bibliometrics, for the sake of counting the number of papers that are commonly cited by two scientific documents. It is given by $B = AA^T$, and $B[i, j]$ gives the number of nodes that the nodes i and j both point to in the original directed graph with adjacency matrix A .

$$\begin{aligned}
 B(i, j) &= \sum_k A(i, k)A(j, k) \\
 &= \sum_k A(i, k)A^T(k, j) \\
 B &= AA^T
 \end{aligned}$$

The *co-citation* matrix was introduced by Small [105], again in the field of bibliometrics. It is given by $C = A^T A$, and $C[i, j]$ gives the number of nodes that commonly point to both i and j in the original directed graph.

6.2 Graph symmetrizations

We adopt a two-stage approach for clustering directed graphs, schematically depicted in Figure 6.2. In the first stage we transform the directed graph into an undirected graph (i.e. symmetrize the directed graph) using one of different possible symmetrization methods. In the second stage, the undirected graph so obtained is clustered using one of several possible graph clustering algorithms. The advantage of this approach is that it allows a practitioner to employ a graph clustering algorithm of their choice in the second stage. For example, spectral clustering algorithms are typically state-of-the-art quality-wise, but do not scale well as eigenvector computations can be very time-consuming [36]. Under such circumstances, it is useful to be able to plug in a scalable graph clustering algorithm of our own choice, such as Graclus [36], MLR-MCL, Metis [63] etc. Note that it is not our objective here to propose a new (undirected) graph clustering algorithm or discuss the strengths and weaknesses of existing ones; all we are saying is that whichever be the suitable graph clustering algorithm, it will fit in our framework.

Of course, the effectiveness of our approach depends crucially on the the symmetrization method. If the symmetrization itself is flawed, even a very good graph clustering algorithm will not be of much use. But do we have reason to believe that an effective symmetrization of the input directed graph is possible? We believe the answer is yes, at least if the domain in question does indeed have some cluster structure. Fundamentally a cluster is a group of objects that are similar to one another and dissimilar to objects not in the cluster. If a domain admits of clusters, this means that there must exist some reasonable similarity measure among the objects in that domain. Since similarity measures are generally symmetric (i.e. $similarity(i, j) = similarity(j, i)$) and positive, defining a notion of similarity for a fixed set of input objects is equivalent to constructing an undirected graph among them, with edges between pairs of objects with non-zero similarity between them and the edge weight equal to the actual value of the similarity. In fact, our proposed degree-discounted symmetrization method can just as validly be thought of as measuring the similarity between pairs of vertices in the input directed graph.

We next discuss various ways to symmetrize a directed graph. In what follows, G will be the original directed graph with associated (asymmetric) adjacency matrix A . G_U will be the resulting symmetrized undirected graph with associated adjacency matrix U .

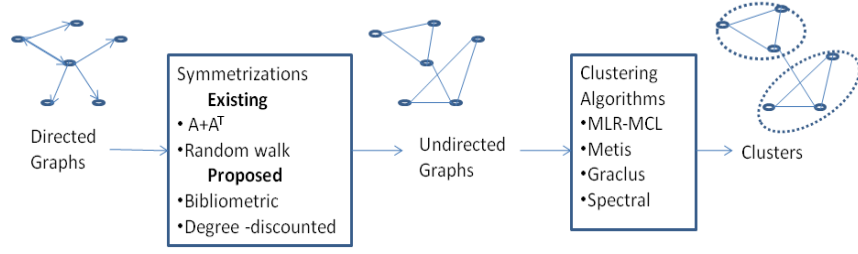


Figure 6.2: Schematic of our framework

6.2.1 $A + A^T$

The simplest way to derive an undirected graph from a directed one is via the transformation $U = A + A^T$. Note that this is very similar to the even simpler strategy of simply ignoring the directionality of the edges, except that in the case of pairs of nodes with directed edges in both directions, the weight of the edge in the symmetrized graph will be the sum of the weights of the two directed edges. It is important to empirically compare this scheme against other symmetrizations since this is the implicit symmetrization commonly used [75, 36, 86, 126].

The advantage of this method is, of course, its simplicity. On the other hand, this method will fare poorly with situations of the sort depicted in Figure 1; the nodes 4 and 5 will continue to remain unconnected in the symmetrized graph, making it impossible to cluster them together.

6.2.2 Random walk symmetrization

Is it possible to symmetrize a directed graph G into G_U such that the directed normalized cut of a group of vertices S , $NCut_{dir}(S)$ is equal to the (undirected) normalized cut of the same group of vertices in the symmetrized graph G_U ? The answer turns out to be yes.

Let P be the transition matrix of the random walk, π its associated stationary distribution, and Π is the diagonal matrix with π on the diagonal. Let U be the symmetric matrix such that

$$U = \frac{\Pi P + P^T \Pi}{2}$$

Gleich [54] showed that for the symmetrized graph G_U with associated adjacency matrix U , the (undirected) Ncut on this graph is equal to the directed Ncut on the original directed graph, for any subset of vertices S . This means that clusters with low directed ncut can be found by clustering the symmetrized graph G_U , and one can use any state-of-the-art graph clustering for finding clusters with low ncut in G_U , instead of relying on expensive spectral clustering using the directed Laplacian (given in Eqn 6.5) as previous researchers have [126, 58].

The matrix P can be obtained easily enough by normalizing the rows of input adjacency matrix A , and the stationary distribution π can be obtained via power iterations. However, the clusters obtained by clustering G_U will still be subject to the same drawbacks that we pointed out in Section 6.1.1. Also note that this symmetrization leads to the exact same set of edges as $A + A^T$, since P and P^T have the same non-zero structure as A and A^T and Π is a diagonal matrix. The actual weights on the edges will, of course, be different for the two methods.

6.2.3 Bibliometric symmetrization

One desideratum of the symmetrized graph is that edges should be present between nodes that share similar (in- or out-) links, and edges should be absent between nodes in the absence of shared (in- or out-)links. Both $A + A^T$ and Random walk symmetrization fail in this regard as they retain the exact same set of edges as in the original graph; only the directionality is dropped and, in the case of the Random walk symmetrization, weights are added to the existing edges.

The bibliographic coupling matrix (AA^T) and the co-citation strength matrix ($A^T A$) are both symmetric matrices that help us satisfy this desideratum. Recall that AA^T measures the number of common out-links between each pair of nodes, where as $A^T A$ measures the number of common in-links. As there does not seem to be any obvious reason for leaving out either in-links or out-links, it is natural to take the sum of both matrices so as to account for both. In this case $U = AA^T + A^T A$, and we refer to this as bibliometric symmetrization. ¹¹

Setting $A := A + I$ prior to the symmetrization ensures that edges in the input graph will not be removed from the symmetrized version.

¹¹Meila and Pentney [86] compare against the $A^T A$ symmetrization, but neither suggest nor compare against the $AA^T + A^T A$ symmetrization. To the best of our knowledge, this symmetrization is new to our work.

6.2.4 Degree-discounted symmetrization

As a consequence of the well-known fact that the degree distributions of many real world graphs follow a power law distribution [44, 24], nodes with degrees in the tens as well as in the thousands co-exist in the same graph. (This is true for both in-degrees and out-degrees.) This wide disparity in the degrees of nodes has implications for the Bibliometric symmetrization; nodes with high degrees will share a lot of common (in- or out-) links with other nodes purely by virtue of their higher degrees. This is the motivation for our proposed *Degree-discounted* symmetrization approach, where we take into account the in- and out-degrees of each node in the symmetrization process.

Another motivation for our proposed symmetrization is defining a useful similarity measure between vertices in a directed graph. As noted earlier in Section 6.2, a meaningful similarity measure will also serve to induce an effective symmetrization of the directed graph; ideally, we want our symmetrized graph to place edges of high weight between nodes of the same cluster and edges of low weight between nodes in different clusters.

How exactly should the degree of nodes enter into the computation of similarity between pairs of nodes in the graph? First we will consider how the computation of out-link similarity (i.e. the bibliographic coupling) should be changed to incorporate the degrees of nodes.

Consider the following two scenarios (see Figure 6.3(a)):

1. Nodes i and j both point to the node h , which has in-coming edges from many nodes apart from i and j . In other words, the in-degree of h , $D_i(h)$ is high.
2. Nodes i and j both point to the node k , but which has in-coming edges only from a few other nodes apart from i and j .

Intuition suggests that case 1 above is a more frequent (hence less informative) event than case 2, and hence the former event should contribute less towards the similarity between i and j than the latter event. In other words, *when two nodes i and j commonly point to a third node, say l , the contribution of this event to the similarity between i and j should be inversely related to the in-degree of l .*

Next we consider how the degree of two nodes should factor into the similarity computation of those two nodes themselves. Figure 6.3(b) illustrates the intuition here: sharing a common out-link k counts for less when one of the two nodes that

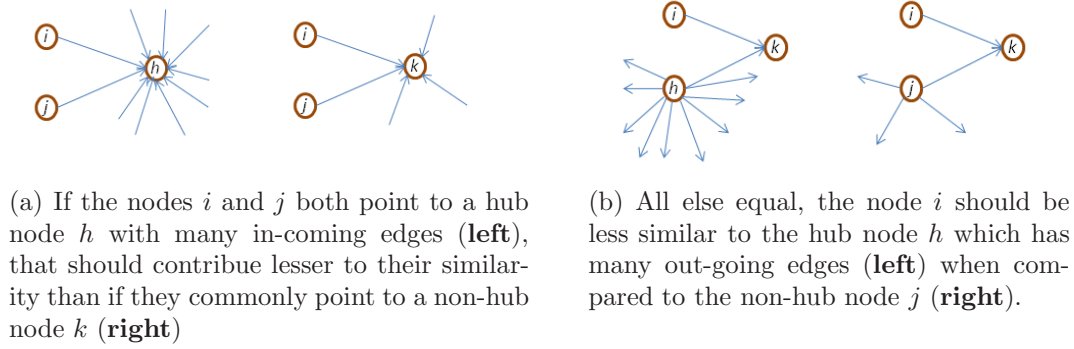


Figure 6.3: Scenarios illustrating the intuition behind degree-discounting.

are doing the sharing is a node with many out-links. In other words, *the out-link similarity between i and j should be inversely related to the out-degrees of i and j .*

We have determined qualitatively how we should take the in- and the out-degrees of the nodes into account, but the exact form of the relationship remains to be specified. We have found experimentally that discounting the similarity by the square root of the degree yields the best results; making the similarity inversely proportional to the degree itself turned out to be an excessive penalty.

With the above insights, we define the out-link similarity or degree-discounted bibliographic coupling between the nodes i and j as follows: (D_o is the diagonal matrix of out-degrees, and $D_o(i)$ is short-hand for $D_o(i, i)$. Similarly D_i is the diagonal matrix of in-degrees. α and β are the discounting parameters.)

$$\begin{aligned}
 B_d(i, j) &= \frac{1}{D_o(i)^\alpha D_o(j)^\alpha} \sum_k \frac{A(i, k) A(j, k)}{D_i(k)^\beta} \\
 &= \frac{1}{D_o(i)^\alpha D_o(j)^\alpha} \sum_k \frac{A(i, k) A^T(k, j)}{D_i(k)^\beta}
 \end{aligned}$$

Note that the above expression is symmetric in i and j . It can be verified that the entire matrix B_d with its (i, j) entries specified as above can be expressed as:

$$B_d = D_o^{-\alpha} A D_i^{-\beta} A^T D_o^{-\alpha} \quad (6.6)$$

Our modification for the co-citation (in-link similarity) matrix is exactly analogous to the above discussion; we proceed to directly give the expression for the matrix C_d containing the degree-discounted co-citation or in-link similarities between all pairs of nodes.

$$C_d = D_i^{-\beta} A^T D_o^{-\alpha} A D_i^{-\beta} \quad (6.7)$$

The final degree discounted similarity matrix U_d is simply the sum of B_d and C_d .

$$U_d = B_d + C_d$$

Empirically we have found $\alpha = \beta = 0.5$ to work the best. Using $\alpha = \beta = 1$ penalized hub nodes excessively, while smaller values such as 0.25 were an insufficient penalty. Penalizing using the log of the degree (similar to the IDF transformation [85]) was also an insufficient penalty. Therefore, the final degree-discounted symmetrization is defined as follows:

$$U_d = D_o^{-1/2} A D_i^{-1/2} A^T D_o^{-1/2} + D_i^{-1/2} A^T D_o^{-1/2} A D_i^{-1/2} \quad (6.8)$$

We point out that the degree-discounting intuition has been found to be effective for solving other problems on directed graphs previously. In the context of node ranking, Ding et al. [40] combine the mutual re-inforcement of HITS with the degree-discounting of PageRank to obtain ranking algorithms that are intermediate between the two. In the context of semi-supervised learning, Zhou et al. [127] propose to regularize functions on directed graphs so as to force the function to change slowly on vertices with high normalized in-link or out-link similarity.

6.2.5 Pruning the symmetrized matrix

One of the main advantages of Degree-discounted symmetrization over Bibliometric symmetrization ($AA^T + A^T A$) is that it is much easier to prune the resulting matrix. $AA^T + A^T A$ and the Degree-discounted similarity matrix U_d share the same non-zero structure, but the actual values are, of course, different. For big real world graphs, the full similarity matrix has far too many non-zero entries and clustering the entire resulting undirected graph is very time-consuming. For this reason, it is critical that it be possible for us to pick a threshold so as to be able to retain only those entries in the matrix which pertain to sufficiently similar pairs of nodes. However, picking

a threshold for $AA^T + A^T A$ can be very hard; as the degrees of nodes are not taken into account, the hub nodes in the graph generate a large number of non-zero entries with high non-zero values (this is because hubs will tend to share a lot of out-links and in-links with a lot of nodes just by virtue of their having high degrees). When we set a high threshold so as to keep the matrix sparse enough to be able to cluster in a reasonable amount of time, many of the rows corresponding to the other nodes become empty. When we lower the threshold in response, the matrix becomes very dense and it becomes impractical to cluster such a dense matrix.

This problem is considerably reduced when applying Degree-discounted symmetrization. This is because the matrix entries involving hub nodes no longer are the largest; this lets us choose a threshold such that when we retain only matrix entries above the threshold, we have a matrix that is sufficiently sparse and at the same time covers the majority of nodes in the graph.

6.2.6 Complexity analysis

The time complexity in general for multiplying dense matrices is $O(n^{2.8})$ using Strassen's algorithm, where n is the number of rows/columns. However, since our matrices are sparse, we can do significantly better than that. Each node i that has d_i connections (either through in-links or out-links), contributes to the similarity between each of the $\binom{d_i}{2}$ pairs of nodes it connects to. Therefore, the total number of similarity contributions that will need to be computed and added up is $\sum_i d_i^2$, which means that a new upper bound on the time complexity of the similarity computation is $O(\sum_i d_i^2)$. We can further improve upon this by exploiting the fact that we only want to compute those entries in the similarity matrix which are above a certain prune threshold. Bayardo et. al. [12] outline approaches for curtailing similarity computations that will provably lead to similarities lower than the prune threshold, and which can enable significant speedups compared to computing all the entries in the similarity matrix.

In terms of space complexity, the similarity computation requires no extra space in addition to that required to store the similarities themselves.

Dataset	Vertices	Edges	Percentage of symmetric links	No. of ground truth categories
Wikipedia	1,129,060	67,178,092	42.1	17950
Cora	17,604	77,171	7.7	70
Flickr	1,861,228	22,613,980	62.4	N.A.
Livejournal	5,284,457	77,402,652	73.4	N.A.

Table 6.1: Details of the datasets

6.3 Experimental Setup

6.3.1 Datasets

We perform experiments using four real datasets, detailed below. Also see Table 6.1.

1. Wikipedia: This is a directed graph of hyperlinks between Wikipedia articles. We downloaded a snapshot of the entire Wikipedia corpus from the Wikimedia foundation ¹² (Jan–2008 version). The corpus has nearly 12 million articles, but a lot of these were insignificant or noisy articles that we removed as follows. First, we retained only those articles with an abstract, which cut the number of articles down to around 2.1 million. We then constructed the directed graph from the hyperlinks among these pages and retained only those nodes with out-degree greater than 15. We finally obtained a directed graph with 1,129,060 nodes and 67,178,092 edges, of which 42.1% are bi-directional.

Pages in Wikipedia are assigned to one or more categories by the editors (visible at the bottom of a page), which we used to prepare ground truth assignments for the pages in our dataset. We removed the many categories that are present in Wikipedia for housekeeping purposes (such as “Articles of low significance”, “Mathematicians stubs”). We further removed categories which did not have more than 20 member pages in order to remove insignificant categories. We obtained 17950 categories after this process. Note that these categories are not disjoint, i.e. a page may belong to multiple categories (or none). Also, 35% of the nodes in the graph do not have any ground truth assignment.

¹²<http://download.wikimedia.org/>

2. Cora: This is a directed graph of CS research papers and their citations. It has been collected and shared by Andrew McCallum ¹³. Besides just the graph of citations, the papers have also been manually classified into 10 different fields of CS (such as AI, Operating Systems, etc.), with each field further sub-divided to obtain a total of 70 categories at the lowest level. Again, 20% of the nodes have not been assigned any labels. We utilize the classifications at the lowest level (i.e. 70 categories) for the sake of evaluation. This graph consists of 17,604 nodes with 77,171 directed edges. Note that although symmetric links are, strictly speaking, impossible in citation networks (two papers cannot cite one another as one of them will need to have been written before the other), there is still a small percentage (7.7%) of symmetric links in this graph due to noise.

3. Flickr and **4. Livejournal:** These are large scale directed graphs, collected by the Online Social Networks Research group at The Max Planck Institute [88]. The number of nodes and edges for these datasets can be found in Table 6.1. We use these datasets only for scalability evaluation as we do not have ground truth information for evaluating effectiveness of discovered clusters.

6.3.2 Setup

We compare four different graph symmetrization methods described in Section 6.2. For Random walk symmetrization, the stationary distribution was calculated with a uniform random teleport probability of 0.05 in all cases. We clustered the symmetrized graphs using MLR-MCL, Metis [63] and Graclus [36]. We are able to show the results of Graclus only on the Cora dataset as the program did not finish execution on any of the symmetrized versions of the Wikipedia dataset. Note that the number of output clusters in MLR-MCL can *only be indirectly controlled* via changing some other parameters of the algorithm; for this reason there is a slight variation in the number of clusters output by this algorithm for different symmetrizations.

We also compare against the BestWCut algorithm described by Meila and Pentney [86], but on the Cora dataset alone, as the algorithm did not finish execution on the Wikipedia dataset. It bears emphasis that BestWCut is not a symmetrization method. The directed spectral clustering of Zhou et. al. [126] did not finish execution on any of our datasets.

¹³<http://www.cs.umass.edu/mccallum/code-data.html>

All the experiments were performed on a dual core machine (Dual 250 Opteron) with 2.4GHz of processor speed and 8GB of main memory. However, the programs were single-threaded so only one core was utilized. The software for each of the undirected graph clustering algorithms as well as BestWCut [86] was obtained from the authors' respective webpages. We implemented the different symmetrization methods in C, using sparse matrix representations.

6.3.3 Evaluation method

The clustering output by any algorithm was evaluated with respect to the ground truth clustering by calculating Avg. weighted F-scores, as described in Section 2.1.1.

6.4 Results

6.4.1 Characteristics of symmetrized graphs

The number of edges in the resulting symmetrized graph for each symmetrization method for the different datasets is given in Table 6.2, along with the pruning thresholds used. To obtain more insight into the structure of the symmetrized graphs, we analyze the distribution of node degrees in the case of Wikipedia (see Figure 6.4). Note that $A + A^T$ and Random Walk have the same distributions, as they have the same set of edges. The Degree-discounted method ensures that most nodes have medium degrees in the range of 50-200 (which is about the size of the average cluster [75]), and completely eliminates hub nodes. These properties enable subsequent graph clustering algorithms to perform well. The Bibliometric graph, on the other hand, has many nodes with both very low degrees, as well as many hub nodes, making clustering the resulting graph difficult. The $A + A^T$ graph also has more hub nodes than the Degree-discounted graph.

6.4.2 Results on Cora

Results pertaining to cluster quality as well as clustering time on the Cora dataset are shown in Figures 6.5 and 6.6.

Figure 6.5 (a) compares the Avg. F scores obtained using MLR-MCL with different symmetrizations. For all symmetrizations, the performance reaches a peak at 50-70 clusters, which is close to the true number of clusters (70). With fewer clusters,

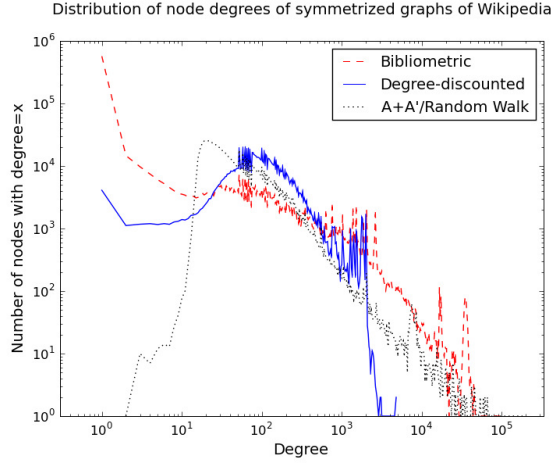


Figure 6.4: Distributions of node degrees for different symmetrizations of Wiki.

Dataset	$A + A^T$ / Random Walk	Bibliometric		Degree-discounted	
	Edges	Edges	Threshold	Edges	Threshold
Wikipedia	53,017,527	85,035,548	25	80,373,184	0.01
Flickr	15,555,041	79,765,961	20	45,167,216	0.01
Cora	74,180	986,444	0	986,444	0
Livejournal	51,352,001	143,759,001	5	91,624,309	0.025

Table 6.2: Details of symmetrized graphs.

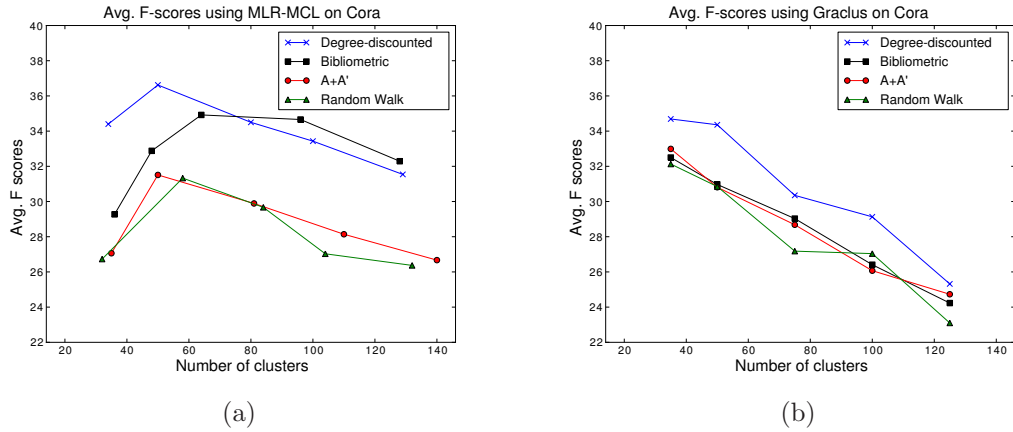


Figure 6.5: Quality comparisons on Cora using (a) MLR-MCL and (b) Graculus

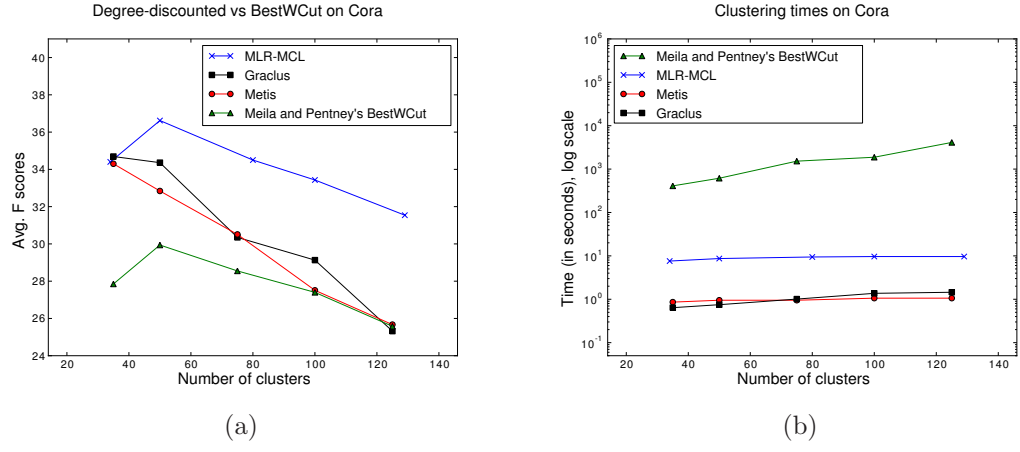


Figure 6.6: Degree-discounted vs BestWCut [86] (a) Effectiveness (b) Speed.

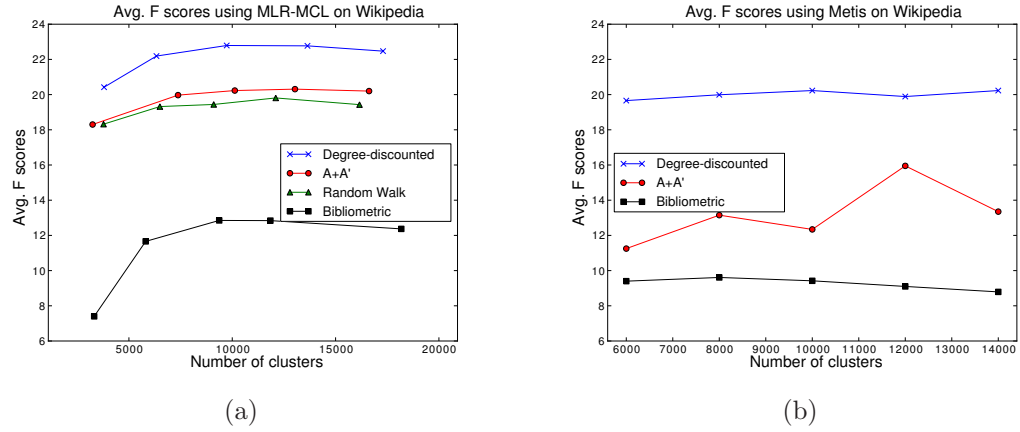
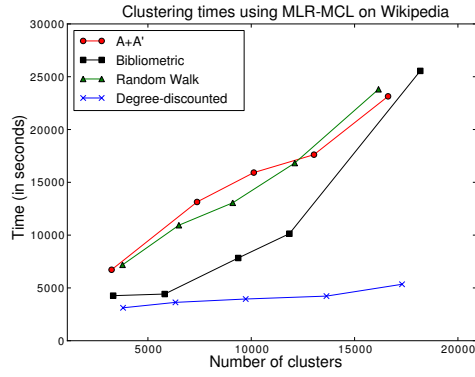
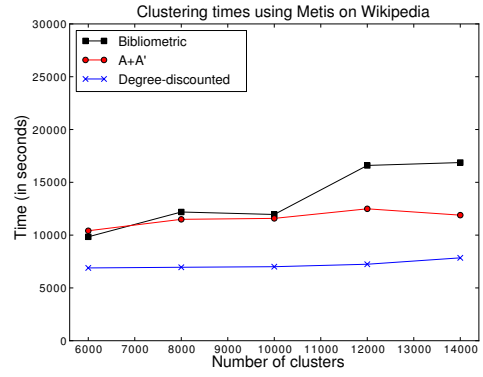


Figure 6.7: Quality Comparisons on Wiki using (a) MLR-MCL and (b) Metis.

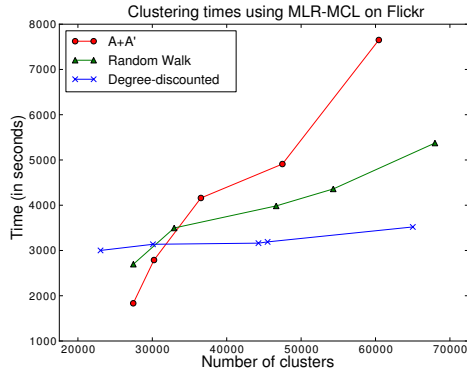


(a)

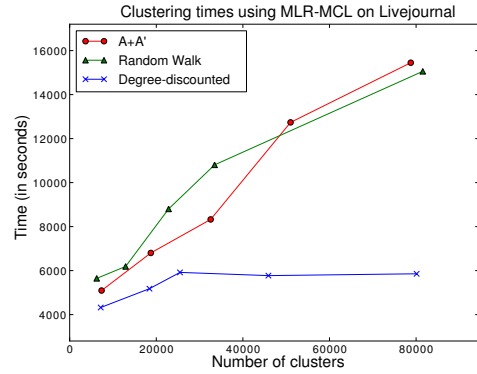


(b)

Figure 6.8: Clustering times on Wiki using (a) MLR-MCL (b) Metis



(a)



(b)

Figure 6.9: Clustering times using MLR-MCL on (a) Flickr and (b) LiveJournal

the precision is adversely impacted, while a greater number of clusters affects the recall. Degree-discounted symmetrization on the whole yields better F scores than the other methods, and also achieves the best overall F-value of 36.62. Bibliometric symmetrization also yields good F-scores with a peak of 34.92, and marginally improves on Degree-discounted for higher number of clusters. $A + A^T$ and Random walk perform similarly and are relatively poor compared to the other two methods. Figure 6.5 (b) shows the effectiveness of different symmetrizations, this time using a different clustering algorithm, Graclus. Degree-discounted symmetrization clearly delivers improvements over the other symmetrizations in this case as well. This shows that multiple clustering algorithms can benefit from the proposed symmetrizations.

Figure 6.6 (a) fixes the symmetrization to Degree discounted and compares MLR-MCL, Graclus and Metis with Meila and Pentney’s BestWCut [86]. The peak F score achieved by BestWCut is 29.94, while the peak F-scores for MLR-MCL, Graclus and Metis are 36.62, 34.69 and 34.30 respectively. Therefore Degree-discounted symmetrization combined with any of the three clustering algorithms - either MLR-MCL, Graclus or Metis - comfortably outperforms BestWCut. Using MLR-MCL, Degree-discounted symmetrization improves upon BestWCut by 22%.

Figure 6.6 (b) compares cluster times of MLR-MCL, Graclus and Metis with Degree-discounted symmetrization against the time taken by BestWCut. All three are much faster than BestWCut. The slow performance of BestWCut is because of the need for expensive eigenvector computations, which none of the other three algorithms involve.

6.4.3 Results on Wikipedia

We next turn to cluster quality and timing results on Wikipedia, depicted in Figures 6.7 and 6.8. In general, this dataset was harder to cluster than the Cora dataset, with an overall peak Avg. F score of 22.79, compared to 36.62 for Cora. Note that we do not have any results from BestWCut [86] on this dataset as it did not finish execution.

Figure 6.7 (a) and (b) compares the Avg. F scores with different symmetrizations using MLR-MCL and Metis. Degree-discounted symmetrization yields the best Avg F scores, with a peak F value of 22.79. $A + A^T$ gives the next best results, with a peak F value of 20.31. These peak scores were obtained using MLR-MCL. Metis on Degree-discounted symmetrization achieves a peak F-value of 20.15, a significant 27%

improvement on the next best F-value of 15.95, achieved using $A + A^T$. Therefore Degree-discounted symmetrization benefits both MLR-MCL and Metis. The performance of Random Walk is slightly worse than $A + A^T$ but is otherwise similar. We do not report Metis combined with Random Walk symmetrization as the program crashed when run with this input. Bibliometric performs very poorly, with F scores barely touching 13%. The main reason for the poor performance of Bibliometric is that explained in Section 6.2.5 - even though we pruned the outputs of both Bibliometric and Degree-discounted symmetrizations so that they contained a similar number of edges (around 80 milion), the Bibliometric graph still ended up with nearly 50% of the nodes as singletons. It is worth mentioning that there was no such problem with Degree-discounted.

Figure 6.8 (a) and (b) show the time to cluster different symmetrizations using MLR-MCL and Metis. We find that both MLR-MCL and Metis execute faster with Degree-discounted, than any of the other symmetrizations. The difference becomes more pronounced with increasing number of clusters; MLR-MCL executes nearly 4.5 to 5 times faster on Degree-discounted as compared to the other symmetrizations in the high clusters range (16000-18000). We believe that the absence of hub nodes (as can be seen in Fig 6.4), coupled with clearer cluster structures in the Degree-discounted graph explains its better performance. It is also interesting to note that on this dataset MLR-MCL is on average significantly faster (2000s) than Metis on the degree-discounted transformation.

Varying the prune threshold

How does the performance of Degree-discounted symmetrization change as we change the pruning threshold i.e. as more or fewer edges are retained in the graph? We experimented with four different thresholds. The obtained Avg F scores as well as times to cluster are given in Table 6.4.3, for both MLR-MCL and Metis. The trends depicted in the table accord very well with our intuition; as we raise the threshold, there are fewer edges in the graph, and there is a gradual drop in the cluster quality, but which is compensated by faster running times. In fact, even with a threshold of 0.025, and having only 60% as many edges as $A + A^T$, Degree-discounted+MLR-MCL still yields an F score of 21.72 (compared to 20.2 for $A + A^T$) and clusters in 1039 seconds (compared to nearly 23000 seconds for $A + A^T$). The trends are very similar for Metis as well.

Threshold	No. of edges	MLR-MCL		Metis	
		F score	Time	F score	Time
0.010	80,373,184	22.47	4225	20.15	7010
0.015	73,273,127	22.45	3615	20.06	4488
0.020	50,801,885	22.27	1912	20.04	1399
0.025	37,663,652	21.72	1039	19.86	547

Table 6.3: Effect of varying pruning threshold

These results also suggest that there is no single “correct” pruning threshold. Lower prune thresholds retain more edges in the symmetrized graphs and result in higher clustering accuracies, but on the flip side, take longer to cluster. Higher prune thresholds mean the accuracy may be lower, but the graph is also clustered faster. The user may therefore select a prune threshold according to their computational constraints. One can compute all the similarities corresponding to a small random sample of the nodes, and choose a prune threshold such that the average degree when this threshold is applied to the random sample approximates the final average degree that the user desires. For many real networks, an average degree of 50-150 in the symmetrized graph seems most reasonable, since this is the size of typical clusters in such networks [75].

6.4.4 Results on Livejournal and Flickr

In Figure 6.9(a) and (b), we show clustering times using MLR-MCL on the Livejournal and Flickr datasets. We could not evaluate cluster quality for lack of ground truth data. We do not report results on Bibliometric, since it is clear from the number of singletons for that transformation (see Table 6.2) that it is not viable for such large scale graphs. The trends for these datasets closely mimic the trends in Wikipedia, with Degree-discounted symmetrization once again proving at least two times as fast to cluster as the others at the higher range of the number of clusters. Similar to Wikipedia, the main reason for the faster performance of Degree-discounted symmetrization is the absence of hub nodes in the symmetrized graphs and a clearer

α	β	F-score on Cora	F-score on Wiki
0	0	28.48	9.42
log	log	30.92	19.43
0.25	0.25	30.79	18.13
0.5	0.5	31.66	20.15
0.75	0.75	29.82	19.97
1.0	1.0	30.58	18.70
0.25	0.50	30.42	19.79
0.25	0.75	31.42	19.52
0.50	0.25	30.51	18.65
0.50	0.75	30.93	20.04
0.75	0.25	30.07	18.42
0.75	0.50	31.07	19.38

Table 6.4: Effect of varying α, β (Metis). The best results are indicated in bold.

cluster structure (the normalized cuts [104, 36] obtained from clustering the Degree-discounted symmetrized graphs are much lower than those obtained using the original graph, indicating the presence of well-separated clusters in the former).

6.4.5 Effect of varying α and β

We next examine the effect of varying the out-degree discount parameter α and the in-degree discount parameter β . The Avg. F-scores obtained by clustering the symmetrized graph using Metis for a specific configuration of α and β is shown in Table 6.4.5 (for ease of comparison, the number of clusters is fixed at 70 for Cora and 10,000 for Wikipedia). In both the datasets, the best F-scores are obtained using $\alpha = \beta = 0.5$. However, doing degree-discounting using some configuration of α and β is better than doing no degree discounting at all (shown as $\alpha = \beta = 0$ in Table 6.4.5).

In fact, using $\alpha = \beta = 0.5$ is similar to using L2-norms for normalizing raw dot-products, as done when computing cosine distance. Spertus et. al. [106] empirically compared six different similarity measures for the problem of community recommendation and found that L2-normalization performed the best. Hence, it is not surprising that $\alpha = \beta = 0.5$ should similarly work well for us across different datasets.

6.4.6 Significance of obtained improvements

We emphasize that the improvements obtained using Degree-discounted symmetrizations are significant, both in the practical and the statistical sense. MLR-MCL, Graclus and Metis are quite different clustering algorithms, and combining *any* of them with the Degree-discounted symmetrization resulted in significant improvements over baseline approaches in terms of quality (in the range of 10-30%), as well as in terms of clustering time (2-5 times speedup on million-plus node graphs). We also found that the improvements obtained using Degree-discounted symmetrization over the baseline approaches were highly statistically significant. We used the very general *paired binomial sign test* to test the null hypothesis that there is no improvement. The sign test makes no assumptions about the underlying test distribution, and hence is suitable in our situation since we do not actually know the underlying test distribution. It was applied as follows. We count the number of graph nodes that were correctly clustered in one clustering but not in the other clustering (in a paired fashion i.e. each node in one clustering is compared with the same node in the other clustering), and also the other way around. The probability of the obtained counts (or more extreme counts) arising from the null hypothesis, calculated using the binomial distribution with $p=0.5$, gives us the final p-value. Small p-values tell us that the observed improvements are unlikely to have occurred by random chance.

The improvements in clustering accuracy reported above are all highly statistically significant. On Cora, MLR-MCL's improvement using Degree-discounted symmetrization over using $A + A^T$ is significant with p-value 1.0E-312, and the improvement over BestWCut is significant with p-value 1.0E-112. Similarly, the improvement of Graclus using Degree-discounted symmetrization over using $A + A^T$ is significant with p-value 1.0E-36, and the improvement over using BestWCut is significant with p-value 1.0E-44. The improvement of Metis over using BestWCut is significant with p-value 1.0E-79. Coming to Wiki, MLR-MCL's improvement when using Degree-discounted over $A + A^T$ is significant with p-value 1.0E-3367. The improvement for Metis is also significant with p-value 1.0E-22767.

6.4.7 A case study of Wikipedia clusters

Why exactly does Degree-discounted symmetrization out-perform other methods? We give some intuition on this question using examples of Wikipedia clusters that were

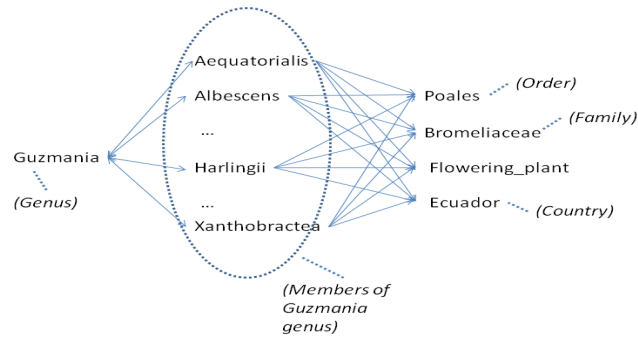


Figure 6.10: Wiki subgraph of plant species of the genus *Guzmania*

successfully extracted through this method but not with the other symmetrizations. Note that these example clusters were recovered by both MLR-MCL as well as Metis and is thus independent of the clustering algorithms.

A typical example is the cluster consisting of the plant species belonging to the genus *Guzmania*. The in-links and out-links of this group is shown in Figure 6.10. Example cluster consisting of plants belonging to the *Guzmania* family. The first notable fact about this cluster is that *none of the cluster members* links to one another, but they all point to some common pages - e.g. “Poales”, which is the Order containing the *Guzmania* genus; “Ecuador”, which is the country that all of these plants are endemic to; and so on. All group members are commonly pointed to by the *Guzmania* node as well as point to it in return.

Note that this cluster is not an isolated example. Clusters involving lists of objects particularly were found to satisfy a similar pattern to the *Guzmania* cluster. Other examples include *Municipalities in Palencia*, *Irish cricketers*, *Lists of birds by country* etc.

These examples provide empirical validation of our hypothesis - laid out in Section 6.2 and Figure 1 - that in-link and out-link similarity, and not inter-linkage, are the main clues to discovering meaningful clusters in directed graphs.

Symmetrization method	Node 1	Node 2	Edge weight
Random walk	Area	Square mile	3354848
	Mile	Square mile	2233110
	Geocode	Geographic coordinate system	1788953
	Degree (angle)	Geographic coordinate system	1766339
	Area	Octagon	1457427
Bibliometric	Area	Population density	2465
	Record label	Music genre	2423
	Population density	Geographic coordinate system	2301
	Square mile	Population density	2129
	Area	Time zone	2120
Degree-discounted	Cyathea	Cyathea (Subgenus Cyathea)	68
	Roman Catholic dioceses in England & Wales	Roman Catholic dioceses in Great Britain	57
	Sepiidae	Sepia (genus)	55
	Szabolcs-Szatmár-Bereg	Szabolcs-Szatmár-Bereg-related topics	53
	Canton of Lizy-sur-Ourcq	Communauté de communes du Pays de l'Ourcq	52

Table 6.5: Edges with highest weights for different symmetrizations on Wiki

6.4.8 Top-weight edges in Wikipedia symmetrizations

We pick the top-weighted edges in the different symmetrizations of Wikipedia to gain a better understanding into their workings. The top 5 edges from Degree-discounted, Bibliometric and Random Walk symmetrizations are shown in Table 6.5. Bibliometric heavily weights edges involving hub nodes such as ‘Area’, ‘Population density’ etc (‘Area’ has an in-degree of 71,146, e.g.), as expected. Similarly, Random walk heavily weights edges involving nodes with high Page Rank, which also typically tend to be hub nodes. The top-weighted edges of Degree-discounted, on the other hand, involve non-hub nodes with specific meanings; the particular examples listed in Table 6.5 are almost duplicates of one another.

6.5 Conclusion

In this chapter, we have investigated the problem of clustering directed graphs through a two-stage process of symmetrizing the directed graph followed by clustering the symmetrized undirected graph using an off-the-shelf graph clustering algorithm. We presented Random Walk and Bibliometric symmetrizations, drawing upon previous work, and based on an analysis of their weaknesses, presented the Degree-discounted symmetrization. We compared the different symmetrizations extensively on large scale real world datasets w.r.t. both quality and scalability, and found that Degree-discounted symmetrization yields significant improvements in both areas.

In the next chapter, we will discuss how to construct similarity weighted graphs in the case when the input data does not come in the form of a graph at all.

Chapter 7: Bayesian Locality Sensitive Hashing for Fast Nearest Neighbors

In this chapter, we will discuss the question of how to construct similarity weighted graphs in the general case when we just have a set of input objects along with a user specified similarity function. In particular, we will discuss an important special case of this problem, namely the all pairs similarity search problem with a threshold, in which the goal is to discover all pairs of objects from the input set which have similarity greater than some user specified threshold. The similarity threshold is necessary because very often we are only interested in the most similar pairs. Furthermore, in the absence of the threshold, the problem has time complexity $O(n^2)$ at least, which is intractable for large datasets.

The number of applications for this problem is impressive: clustering [95], semi-supervised learning [128], information retrieval (including text, audio and video), query refinement [12], near-duplicate detection [122], collaborative filtering, link prediction for graphs [79], and 3-D scene reconstruction [3] among others. In many of these applications, approximate solutions with small errors in similarity assessments are acceptable if they can buy significant reductions in running time e.g. in web-scale clustering [21, 95], information retrieval [42], near-duplicate detection for web crawling [84, 56] and graph clustering [100].

Roughly speaking, similarity search algorithms can be divided into two main phases - *candidate generation* and *candidate verification*. During the candidate generation phase, pairs of objects that are good candidates for having similarity above the user-specified threshold are generated using one or another indexing mechanism, while during candidate verification, the similarity of each candidate pair is verified against the threshold, in many cases by exact computation of the similarity. The traditional indexing structures used for candidate generation were space-partitioning approaches such as kd-trees and R-trees, but these approaches work well only in low

dimensions (less than 20 or so [33]). An important breakthrough was the invention of locality-sensitive hashing [59, 52], where the idea is to find a family of hash functions such that for a random hash function from this family, two objects with high similarity are very likely to be hashed to the same bucket. One can then generate candidate pairs by hashing each object several times using randomly chosen hash functions, and generating all pairs of objects which have been hashed to the same bucket by at least one hash function. Although LSH is a randomized, approximate solution to candidate generation, similarity search based on LSH has nonetheless become immensely popular because it provides a practical solution for high dimensional applications along with theoretical guarantees for the quality of the approximation [8].

In this chapter, we show how LSH can be exploited for the phase of similarity search subsequent to candidate generation i.e. candidate verification and similarity computation. We adopt a principled Bayesian approach that allows us to reason about the probability that a particular pair of objects will meet the user-specified threshold by inspecting only a few hashes of each object, which in turn allows us to quickly prune away unpromising pairs. Our Bayesian approach also allows us to estimate similarities to a user-specified level of accuracy without requiring any tuning of the number of hashes, overcoming a significant drawback of standard similarity estimation using LSH. We develop two algorithms, called **BayesLSH** and **BayesLSH-Lite**, where the former performs both candidate pruning and similarity estimation, while the latter only performs candidate pruning and computes the similarities of unpruned candidates exactly. Essentially, BayesLSH provides a way to trade-off accuracy for speed in a controlled manner. Both BayesLSH and BayesLSH-Lite can be combined with any existing candidate generation algorithm, such as AllPairs [12] or LSH. Concretely, BayesLSH provides the following probabilistic guarantees:

Given a collection of objects D , an associated similarity function $s(.,.)$, and a similarity threshold t ; recall parameter ϵ and accuracy parameters δ, γ ; return pairs of objects (x, y) along with similarity estimates $\hat{s}_{x,y}$ such that:

1. *$Pr[s(x, y) \geq t] > \epsilon$ i.e. each pair with a greater than ϵ probability of being a true positive is included in the output set.*
2. *$Pr[|\hat{s}_{x,y} - s(x, y)| \geq \delta] < \gamma$ i.e. each associated similarity estimate is accurate up to δ -error with probability $> 1 - \gamma$.*

With BayesLSH-Lite, the similarity calculations are exact, so there is no need for guarantee 2, but guarantee 1 from above stays. We note that the parametrization of BayesLSH is intuitive - the desired recall can be controlled using ϵ , while δ, γ together specify the desired level of accuracy of similarity estimation.

The advantages of BayesLSH are as follows:

1. The general form of the algorithm can be easily adapted to work for any similarity measure with an associated LSH family (see Section 7.1 for a formal definition of LSH). We demonstrate BayesLSH for Cosine and Jaccard similarity measures, and believe that it can be adapted to other measures with LSH families, such as kernel similarities.
2. There are no restricting assumptions about the specific form of the candidate generation algorithm; BayesLSH *complements* progress in candidate generation algorithms.
3. For applications which already use LSH for candidate generation, it is a natural fit since it exploits the hashes of the objects for candidate pruning, further amortizing the costs of hashing.
4. It works for both binary and general real-valued vectors. This is a significant advantage because recent progress in similarity search has been limited to binary vectors [122, 125].
5. Parameter tuning is easy and intuitive; the only parameters are γ, δ and ϵ , each of which, as we have seen, directly control the quality of the output result. In particular, there is no need for manually tuning the number of hashes, as one needs to with standard similarity estimation using LSH.

We perform an extensive evaluation of our algorithms and comparison with state-of-the-art methods, on a diverse array of 6 real datasets. We combine BayesLSH and BayesLSH-Lite with two different candidate generation algorithms AllPairs [12] and LSH, and find significant speedups, typically in the range 2x-20x over baseline approaches (see Table 7.2). BayesLSH is able to achieve the speedups primarily by being extremely effective at pruning away false positive candidate pairs. To take a typical example, BayesLSH is able to prune away 80% of the input candidate pairs after examining only 8 bytes worth of hashes per candidate pair, and 99.98% of the candidate pairs after examining only 32 bytes per pair. Notably, BayesLSH is able to do such effective pruning without adversely affecting the recall, which is still

quite high, generally at 97% or above. Furthermore, the accuracy of BayesLSH's similarity estimates is much more consistent as compared to the standard similarity approximation using LSH, which tends to produce very error-ridden estimates for low similarities. Finally, we find that parameter tuning for BayesLSH is intuitive and works as expected, with higher accuracies and recalls being achieved without leading to undue slow-downs.

7.1 Background

Following Charikar [25], we define a locality-sensitive hashing scheme as a distribution on a family of hash functions $\mathcal{F}$ operating on a collection of objects, such that for any two objects $\mathbf{x}, \mathbf{y}$,

$$Pr_{h \in \mathcal{F}}[h(\mathbf{x}) = h(\mathbf{y})] = sim(\mathbf{x}, \mathbf{y}) \quad (7.1)$$

It is important to note that the probability in Eqn 7.1 is for a random selection of the hash function from the family $\mathcal{F}$. Specifically, it is not for a random pair $\mathbf{x}, \mathbf{y}$ - i.e. the equation is valid for *any* pair of objects $\mathbf{x}$ and $\mathbf{y}$. The output of the hash functions may be either bits (0 or 1), or integers. Note that this definition of LSH, taken from [25], is geared towards similarity measures and is more useful in our context, as compared to the slightly different definition of LSH used by many other sources [33, 8], including the original LSH paper [59], which is geared towards *distance* measures.

Locality-sensitive hashing schemes have been proposed for a variety of similarity functions thus far, including Jaccard similarity [20, 78], Cosine similarity [25] and kernelized similarity functions (representing e.g. a learned similarity metric) [60].

Candidate generation via LSH:

One of the main reasons for the popularity of LSH is that it can be used to construct an index that enables efficient candidate generation for the similarity search problem. Such LSH-based indices have been found to significantly outperform more traditional indexing methods based on space partitioning approaches, especially with increasing dimensions [59, 33]. The general method works as follows [59, 33, 21, 95, 56]. For each object in the dataset, we will form l signatures, where each signature is a concatenation of k hashes. All pairs of objects that share at least one of the l signatures will be generated as a candidate pair. Retrieving each pair of objects that share a signature can be done efficiently using hashtables. For a given k and similarity threshold t , the number of length- k signatures required for an expected false negative rate ϵ can be shown to be $l = \lceil \frac{\log \epsilon}{\log(1-t^k)} \rceil$ [121].

Candidate verification and similarity estimation:

The similarity between the generated candidates can be computed in one of two ways: (a) by exact calculation of the similarity between each pair, or (b) using an estimate of the similarity, as the fraction of hashes that the two objects agree upon.

The pairs of objects with estimated similarity greater than the threshold are finally output. In terms of running time, approach (b) is often faster, especially when the number of candidates is large and/or exact similarity calculations are expensive, such as with more complex similarity measures or with larger vector lengths. The main overhead with approach (b) is in hashing each point sufficient number of times in the first place, but this cost is amortized over many similarity computations (especially in the case of all-pairs similarity search), and furthermore we need the hashes for candidate generation in any case. However, what is less clear is how good this simple estimation procedure is in terms of accuracy, and whether it can be made any faster. We will address these questions next.

7.2 Classical similarity estimation for LSH

Similarity estimation for a candidate pair using LSH can be considered as a statistical parameter inference problem. The parameter we wish to infer is the similarity, and the data we observe is the outcome of the comparison of each successive hash between the candidate pair. The probability model relating the parameter to the data is given by the main LSH equation, Equation 7.1. There are two main schools of statistical inference - classical (frequentist) and Bayesian.

Under classical (frequentist) statistical inference, the parameters of a probability model are treated as fixed, and it is considered meaningless to make probabilistic statements about the parameters - hence the output of classical inference is simply a point estimate, one for each parameter. The best known example of frequentist inference is maximum likelihood estimation, where the value of the parameter that maximizes the probability of the observed data is output as the point estimate. In the case of similarity estimation via LSH, let us say we have compared n hashes and have observed m agreements in hash values. The maximum likelihood estimator for the similarity $\hat{s}$ is:¹⁴

$$\hat{s} = \frac{m}{n}$$

While previous researchers have not explicitly labeled their approaches as using the maximum likelihood estimators, they have implicitly used the above estimator, tuning the number of hashes n [95, 25]. However, this approach has some important drawbacks, which we turn to next.

7.2.1 Difficulty of tuning the number of hashes

While the above estimator is unbiased, the variance is $\frac{s*(1-s)}{n}$, meaning that the variance of the estimator depends on the similarity s being estimated. This indicates that in order to get the same level of accuracy for different similarities, we will need to use *different* number of hashes.

¹⁴Proofs are elementary and are omitted.

We can be more precise and, for a given similarity, calculate exactly the probability of a smaller-than- δ error in $\hat{s}_n$, the similarity estimated using n hashes.

$$\begin{aligned} Pr[|\hat{s}_n - s| < \delta] &= Pr[(s - \delta) * n \leq m \leq (s + \delta) * n] \\ &= \sum_{m=(s-\delta)*n}^{(s+\delta)*n} \binom{n}{m} s^m (1-s)^{n-m} \end{aligned}$$

Using the above expression, we can calculate the minimum number of hashes needed to ensure that the similarity estimate is sufficiently concentrated, i.e within δ of the true value with probability $1 - \gamma$. A plot of the number of hashes required for $\delta = \gamma = 0.05$ for various similarity values is given in Figure 7.1. As can be seen, there is a great difference in the number of hashes required when the true similarities are different; similarities closer to 0.5 require far more hashes to estimate accurately than similarities close to 0 or 1. A similarity of 0.5 needs 350 hashes for sufficient accuracy, but a similarity of 0.95 needs only 16 hashes! Stricter accuracy requirements lead to even greater differences in the required number of hashes.

Since we don't know the true similarity of each pair a priori, we cannot choose the right number of hashes beforehand. If we err on the side of accuracy and choose a large n , then performance suffers since we will be comparing many more hashes than are necessary for some candidate pairs. If, on the other hand, we err on the side of performance and choose a smaller n , then accuracy suffers. With standard similarity estimation, therefore, it is *impossible* to tune the number of hashes for the entire dataset so as to achieve both optimal performance and accuracy.

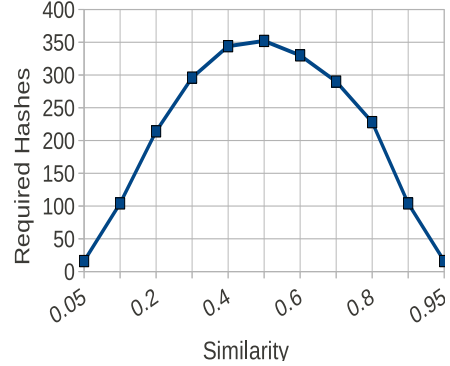


Figure 7.1: Hashes vs. similarity

7.2.2 Ignores the potential for early pruning

In the context of similarity search with a user-specified threshold, the standard similarity estimation procedure also misses opportunities for *early candidate pruning*. The intuition here is best illustrated using an example: Let us say the similarity threshold is 0.8 i.e. the user is only interested in pairs with similarity greater than 0.8. Let us say the similarity estimation is going to use $n = 1000$ hashes. But if we are examining a candidate pair for which, out of the first 100 hashes, only 10 hashes matched, then intuitively it seems very likely that this pair does not meet the threshold of 0.8. In general, it seems intuitively possible to be able to prune away many false positive candidates by looking only at the first few hashes, without needing to compare all the hashes. As we will see, most candidate generation algorithms produce significant number of false positives, and the standard similarity estimation procedure using LSH does not exploit the potential for early pruning of candidate pairs.

7.3 Candidate pruning and similarity estimation using BayesLSH

The key characteristic of Bayesian statistics is that it allows one to make probabilistic statements about any aspect of the world, including things that would be considered “fixed” under frequentist statistics and hence meaningless to make probabilistic statements about. In particular, Bayesian statistics allows us to make probabilistic statements about the parameters of probability models - in other words, parameters are also treated as random variables. Bayesian inference generally consists of starting with a *prior* distribution over the parameters, and then computing a *posterior* distribution over the parameters, conditional on the data that we have actually observed, using Bayes’ rule. A commonly cited drawback of Bayesian inference is the need for the prior probability distribution over the parameters, but a reasonable amount of data generally “swamps out” the influence of the prior (see Section 7.3.4). Furthermore, a good prior can often lead to *improved* estimates over maximum likelihood estimation - this is a common strategy for avoiding overfitting the data in machine learning and statistics. The big advantage of Bayesian inference in the context of similarity estimation is that instead of just outputting a point estimate of the similarity, it gives us the complete posterior distribution of the similarity. In the rest

of this section, we will avoid discussing specific choices for the prior distribution and similarity measure in order to keep the discussion general.

Fix attention on a particular pair (x, y) , and let us say that m out of the first n hashes match for this pair. We will denote this event as $M(m, n)$. The conditional probability of the event $M(m, n)$ given the similarity S (here S is a random variable), is given by the binomial distribution with n trials, where the probability of success of each trial is S itself, from the Equation 7.1. Note that each of the hashing functions are generated independently, hence the outputs of the different hashing functions are independent and identically distributed. Also note that we have already observed the event $M(m, n)$ happening i.e. m and n are not random variables, they are the data.

$$Pr[M(m, n) | S] = \binom{n}{m} S^m (1 - S)^{n-m} \quad (7.2)$$

What we are interested in knowing is the probability distribution of the similarity S , *given* that we already know that m out of n hashes have matched. Using Bayes' rule, the posterior distribution for S can be written as follows: ¹⁵

$$\begin{aligned} p(S | M(m, n)) &= \frac{p(M(m, n) | S)p(S)}{p(M(m, n))} \\ &= \frac{p(M(m, n) | S)p(S)}{\int_0^1 p(M(m, n), s)ds} \\ &= \frac{p(M(m, n) | S)p(S)}{\int_0^1 p(M(m, n) | s)p(s)ds} \end{aligned}$$

By plugging in the expressions for $p(M(m, n) | S)$ from Equation 7.2 and a suitable prior distribution $p(S)$, we can get, for every value of n and m , the posterior distribution of S conditional on the event $M(m, n)$. We calculate the following quantities in terms of the posterior distribution:

1. If after comparing n hashes, m matches agree, what is the probability that the similarity is greater than the threshold t ?

$$Pr[S \geq t | M(m, n)] = \int_t^1 p(s | M(m, n))ds \quad (7.3)$$

¹⁵In terms of notation, we will use lower-case $p(\cdot)$ for probability density functions of continuous random-variables. $Pr[\cdot]$ is used for probabilities of discrete events or discrete random variables.

2. If after comparing n hashes, m matches agree, what is the maximum-a-posteriori estimate for the similarity i.e. the similarity value with the highest posterior probability? This will function as our estimate $\hat{S}$

$$\hat{S} = \arg \max_s p(s | M(m, n)) \quad (7.4)$$

3. Assume after comparing n hashes, m matches agree, and we have estimated the similarity to be $\hat{S}$ (e.g. as indicated above). What is the concentration probability of $\hat{S}$ i.e. probability that this estimate is within δ of the true similarity?

$$Pr[|S - \hat{S}| < \delta | M(m, n)] = Pr[\hat{S} - \delta < S < \hat{S} + \delta | M(m, n)] \quad (7.5)$$

$$= \int_{\hat{S}-\delta}^{\hat{S}+\delta} p(s | M(m, n)) ds \quad (7.6)$$

Assuming we can perform the above three kinds of inference, we design our algorithm, BayesLSH, so that it satisfies the probabilistic guarantees outlined in the beginning of this chapter. The algorithm is outlined in Algorithm 10. For each candidate pair (x, y) we incrementally compare their respective hashes (line 8, the parameter k indicates the number of hashes we will compare at a time), until either one of two events happens. The first possibility is that the candidate pair gets pruned away because the probability of it being a true positive pair has become very small (lines 10, 11 and 12), where we use Equation 7.3 to calculate this probability. The alternative possibility is that the candidate pair does not get pruned away, and we continue comparing hashes until our similarity estimate (line 14) becomes sufficiently concentrated that it passes our accuracy requirements (lines 15 and 16). Here we use Equation 7.6 to determine the probability that our estimate is sufficiently accurate. Each such pair is added to the output set of candidate pairs, along with our similarity estimate (lines 19 and 20).

Our second algorithm, BayesLSH-Lite (see Algorithm 11) is a simpler version of BayesLSH, which calculates similarities exactly. Since the similarity calculations are exact, there is no need for parameters δ, γ ; however, this comes at the cost of some intuitiveness, as there is a new parameter h specifying the maximum number of hashes that will be examined for each pair of objects. BayesLSH-Lite can be faster than BayesLSH for those datasets where exact similarity calculations are cheap, e.g. because the object representations are simpler, such as binary, or if the average size of the objects is small.

BayesLSH clearly overcomes the two drawbacks of standard similarity estimation explained in Sections 7.2.1 and 7.2.2. Any candidate pairs that can be pruned away by examining only the first few hashes will be pruned away by BayesLSH. As we will show later, this method is very effective for pruning away the vast majority of false positives. Secondly, the number of hashes for which each candidate pair is compared is determined automatically by the algorithm, depending on the user-specified accuracy requirements, completely eliminating the need to manually set the number of hashes. Thirdly, each point in the dataset is only hashed as many times as is necessary. This will be particularly useful for applications where hashing a point itself can be costly e.g. for kernel LSH [60]. Also, outlying points which don't have any points with whom their similarity exceeds the threshold need only be hashed a few times before BayesLSH prunes all candidate pairs involving such points away.

Algorithm 10 BayesLSH

```

1: Input: Set of candidate pairs  $C$ ; Similarity threshold  $t$ ; recall parameter  $\epsilon$ ; accuracy
   parameters  $\delta, \gamma$ 
2: Output: Set  $O$  of pairs  $(x, y)$  along with similarity estimates  $\hat{S}_{x,y}$ 
3:  $O \leftarrow \emptyset$ 
4: for all  $(x, y) \in C$  do
5:    $n, m \leftarrow 0$  {Initialization}
6:    $isPruned \leftarrow \text{False}$ 
7:   while True do
8:      $m = m + \sum_{i=n}^{n+k} I[h_i(x) == h_i(y)]$  {Compare hashes  $n$  to  $n + k$ }
9:      $n = n + k$ 
10:    if  $Pr[S \geq t \mid M(m, n)] < \epsilon$  then
11:       $isPruned \leftarrow \text{True}$ 
12:      break {Prune candidate pair}
13:    end if
14:     $\hat{S} \leftarrow \arg \max_s p(s \mid M(m, n))$ 
15:    if  $Pr[|S - \hat{S} \mid M(m, n) < \delta] < \gamma$  then
16:      break {Similarity estimate is sufficiently concentrated}
17:    end if
18:  end while
19:  if  $isPruned == \text{False}$  then
20:     $O \leftarrow O \cup \{(x, y), \hat{S}\}$ 
21:  end if
22: end for
23: return  $O$ 

```

Algorithm 11 BayesLSH-Lite

```
1: Input: Set of candidate pairs  $C$ ; Similarity threshold  $t$ ; recall parameter  $\epsilon$ ; Number of
   hashes to use  $h$ 
2: Output: Set  $O$  of pairs  $(x, y)$  along with exact similarities  $\hat{S}_{x,y}$ 
3:  $O \leftarrow \emptyset$ 
4: for all  $(x, y) \in C$  do
5:    $n, m \leftarrow 0$  {Initialization}
6:    $isPruned \leftarrow \text{False}$ 
7:   while  $n < h$  do
8:      $m = m + \sum_{i=n}^{n+k} I[h_i(x) == h_i(y)]$  {Compare hashes  $n$  to  $n + k$ }
9:      $n = n + k$ 
10:    if  $Pr[S \geq t \mid M(m, n)] < \epsilon$  then
11:       $isPruned \leftarrow \text{True}$ 
12:      break {Prune candidate pair}
13:    end if
14:  end while
15:  if  $isPruned == \text{False}$  then
16:     $s_{x,y} = \text{similarity}(x, y)$  {Exact similarity}
17:    if  $s_{x,y} > t$  then
18:       $O \leftarrow O \cup \{(x, y), s_{x,y}\}$ 
19:    end if
20:  end if
21: end for
22: return  $O$ 
```

In order to obtain a concrete instantiation of BayesLSH, we will need to specify three aspects: (i) the LSH family of hash functions, (ii) the choice of prior and (iii) how to tractably perform inference. Next, we will look at specific instantiations of BayesLSH for different similarity measures.

7.3.1 BayesLSH for Jaccard similarity

We will first discuss how BayesLSH can be used for approximate similarity search for Jaccard similarity.

LSH family: The LSH family for Jaccard similarity is the family of minwise independent permutations [20, 18] on the universe from which our collection of sets is drawn. Each hash function returns the minimum element of the input set when the elements of the set are permuted as specified by the hash function (which itself is chosen at random from the family of minwise independent permutations). The output of this

family of hash functions, therefore, is an integer representing the minimum element of the permuted set.

Choice of prior: It is common practice in Bayesian inference to choose priors from a family of distributions that is *conjugate* to the likelihood distribution, so that the inference is tractable and also that the posterior belongs to the same distribution family as the prior (indeed, that is the definition of a conjugate prior). The likelihood in this case is given by a binomial distribution, as indicated in Equation 7.2. The conjugate for the binomial is the *Beta* distribution, which has two parameters $\alpha > 0, \beta > 0$ and is defined on the domain $(0, 1)$. The pdf for $Beta(\alpha, \beta)$ is defined as follows.

$$p(s) = \frac{s^{\alpha-1} * (1-s)^{\beta-1}}{B(\alpha, \beta)}$$

Here $B(\alpha, \beta)$ is the beta function, and it can also be thought of as a normalization constant to ensure the entire distribution integrates to 1.

Even assuming we want to model the prior using a Beta distribution, how do we choose the parameters α, β ? A simple choice is to set $\alpha = 1, \beta = 1$, which results in a uniform distribution on $(0, 1)$. However, we can actually learn α, β so as to best fit a random sample of similarities from candidate pairs output by the candidate generation algorithm. Let us assume we have r samples chosen uniformly at random from the total population of candidate pairs generated by the particular candidate generation algorithm being used, and their similarities are $s_1, s_2, \dots, s_r$. Then we can estimate α, β so as to best model the distribution of similarities among candidate pairs. For Beta distribution, a simple and effective method of learning the parameters is via method-of-moments estimation. In this method, we calculate the sample moments (sample mean and sample variance), assume that they are the true moments of the distribution and solve for the parameter values that will result in the obtained moments. In our case, we have the following estimates for α, β :

$$\hat{\alpha} = \bar{s} \left(\frac{\bar{s}(1 - \bar{s})}{\bar{s}_v} - 1 \right) ; \hat{\beta} = (1 - \bar{s}) \left(\frac{\bar{s}(1 - \bar{s})}{\bar{s}_v} - 1 \right)$$

where $\bar{s}$ and $\bar{s}_v$ are the sample mean and variance, given as follows:

$$\bar{s} = \frac{\sum_{i=1}^r s_i}{r} ; \bar{s}_v = \frac{\sum_{i=1}^r (s_i - \bar{s})^2}{r}$$

Assuming a prior $Beta(\alpha, \beta)$ distribution on the similarity, and we observe the event $M(m, n)$ i.e. m out of the first n hashes match, then the posterior distribution of the similarity looks as follows:

$$\begin{aligned} p(s|M(m, n)) &= \frac{\binom{n}{m} s^m (1-s)^{n-m} s^{\alpha-1} (1-s)^{\beta-1}}{\int_0^1 \binom{n}{m} s^m (1-s)^{n-m} s^{\alpha-1} (1-s)^{\beta-1} ds} \\ &= \frac{s^{m+\alpha-1} (1-s)^{n-m+\beta-1}}{B(m+\alpha, n-m+\beta)} \end{aligned}$$

Hence, the posterior distribution of the similarity also follows a Beta distribution with parameters $m + \alpha$ and $n - m + \beta$.

Inference: We next show concrete ways to perform inference, i.e. computing Equations 7.3, 7.4 and 7.6.

The probability that similarity is greater than the threshold after observing that m out of the first n hashes match is:

$$\begin{aligned} Pr[S \geq t|M(m, n)] &= \int_t^1 p(s|M(m, n)) ds \\ &= 1 - I_t(m + \alpha, n - m + \beta) \end{aligned}$$

Above $I_t(., .)$ refers to the *regularized incomplete beta function*, which gives the cdf for the beta distribution. This function is available in standard scientific computing libraries, where it is typically approximated using continued fractions [39].

Our similarity estimate, after observing m matches in n hashes, will be the mode of the posterior distribution $p(s|M(m, n))$. The mode of $Beta(\alpha, \beta)$ is given by $\frac{\alpha-1}{\alpha+\beta-2}$. Therefore, our similarity estimate after observing that m out of the first n hashes agree is $\hat{S} = \frac{m+\alpha-1}{n+\alpha+\beta-1}$.

The concentration probability of the similarity estimate $\hat{S}$ can be derived as follows (the expression for $\hat{S}$ indicated above can be substituted in the below equations):

$$\begin{aligned} Pr[|\hat{S} - S| < \delta|M(m, n)] &= \int_{\hat{S}-\delta}^{\hat{S}+\delta} p(s|M(m, n)) ds \\ &= I_{\hat{S}+\delta}(m + \alpha, n - m + \beta) \\ &\quad - I_{\hat{S}-\delta}(m + \alpha, n - m + \beta) \end{aligned}$$

Thus by substituting the above computations in the corresponding places in Algorithm 10, we obtain a version of BayesLSH specifically adapted to Jaccard similarity.

7.3.2 BayesLSH for Cosine similarity

We will next discuss instantiating BayesLSH for Cosine similarity.

LSH family: For Cosine similarity, each hash function h_i is associated with a random vector r_i , each of whose components is a sample from the standard gaussian ($\mu = 0, \sigma = 1$). For a vector x , $h_i(x) = 1$ if $\text{dot}(r_i, x) \geq 0$ and $h_i(x) = 0$ otherwise [25]. Note that each hash function outputs a bit, and hence these hashes can be stored with less space.

However, there is one challenge here that needs to be overcome that was absent for BayesLSH with Jaccard similarity: this LSH family is for a slightly different similarity measure than cosine - it is instead for $1 - \frac{\theta(x,y)}{\pi}$, where $\theta(x,y) = \arccos(\frac{\text{dot}(x,y)}{\|x\| \cdot \|y\|})$. For notational ease, we will refer to this similarity function as $r(x,y)$ i.e. $r(x,y) = 1 - \frac{\theta(x,y)}{\pi}$. Explicitly,

$$\begin{aligned} \Pr[h_i(x) == h_i(y)] &= r(x,y) \\ \Pr[M(m,n)|r] &= \binom{n}{m} r^m (1-r)^{n-m} \end{aligned}$$

Since the similarity function we are interested in is $\cos(x,y)$ and not $r(x,y)$ - in particular, we wish for probabilistic guarantees on the quality of the output in terms of $\cos(x,y)$ and not $r(x,y)$ - we will need to somehow express the posterior probability in terms of $s = \cos(x,y)$. One can choose to re-express the likelihood in terms of $s = \cos(x,y)$ instead of in terms of r but this introduces $\cos()$ terms into the likelihood, and makes it very hard to find a suitable prior that keeps the inference tractable. Instead we compute the posterior distribution of r which we transform appropriately into a posterior distribution of s .

Choice of prior: We will need to choose a prior distribution for r . Previously, we used a Beta prior for Jaccard BayesLSH; unfortunately r has range $[0.5, 1]$, while the standard Beta distribution has support on the domain $(0, 1)$. We can still map the standard Beta distribution onto the domain $(0.5, 1)$, but this distribution will

no longer be conjugate to the binomial likelihood.¹⁶ Our solution is to use a simple uniform distribution on $[0.5, 1]$ as the prior for t . Even when the true similarity distribution is very far from being uniform (as is the case in real datasets, including the ones used in our experiments), this prior still works well because the posterior is strongly influenced by the actual outcomes observed (see Section 7.3.4).

The prior pdf therefore is:

$$p(r) = \frac{1}{1 - 0.5} = 2$$

The posterior pdf, after observing that m out of the first n hashes agree, is:

$$\begin{aligned} p(r|M(m, n)) &= \frac{2 \binom{n}{m} r^m (1-r)^{n-m}}{\int_{0.5}^1 2 \binom{n}{m} r^m (1-r)^{n-m} dr} \\ &= \frac{r^m (1-r)^{n-m}}{\int_{0.5}^1 r^m (1-r)^{n-m} dr} \\ &= \frac{r^m (1-r)^{n-m}}{B_1(m+1, n-m+1) - B_{0.5}(m+1, n-m+1)} \end{aligned}$$

Here $B_x(a, b)$ is the incomplete Beta function, defined as $B_x(a, b) = \int_0^x y^{a-1} (1-y)^{b-1} dy$.

Inference: In order to calculate Equations 7.3, 7.6 and 7.4, we will first need a way to convert from r to s and vice-versa. Let $r2c : [0.5, 1] \rightarrow [0, 1]$ be the 1-to-1 function that maps from $r(x, y)$ to $\cos(x, y)$; $r2c()$ is given by $r2c(r) = \cos(\pi * (1-r))$. Similarly, let $c2r$ be the 1-to-1 function that does the same map in reverse; $c2r()$ is given by $c2r(c) = 1 - \frac{\arccos(c)}{\pi}$.

Let R be the random variable such that $R = c2r(S)$ and let $t_r = c2r(t)$. After observing that the m out of the first n hashes agree, the probability that cosine

¹⁶The pdf of a Beta distribution supported only on $(0.5, 1)$ with parameters α, β is $p(x) \propto (x - 0.5)^{\alpha-1} (1-x)^{\beta-1}$. With a binomial likelihood, the posterior pdf takes the form $p(x|M(m, n)) \propto x^m (x - 0.5)^{\alpha-1} (1-x)^{n-m+\beta-1}$. Unfortunately there is no simple and fast way to integrate this pdf.

similarity is greater than the threshold t is:

$$\begin{aligned}
Pr[S \geq t | M(m, n)] &= Pr[c2r(S) \geq c2r(t) | M(m, n)] \\
&= Pr[R \geq t_r | M(m, n)] \\
&= \int_{t_r}^1 p(r | M(m, n)) dr \\
&= \frac{\int_{t_r}^1 r^m (1-r)^{n-m} dr}{B_1(m+1, n-m+1) - B_{0.5}(m+1, n-m+1)} \\
&= \frac{B_1(m+1, n-m+1) - B_{t_r}(m+1, n-m+1)}{B_1(m+1, n-m+1) - B_{0.5}(m+1, n-m+1)}
\end{aligned}$$

The first step in the above derivation follows because $c2r()$ is a 1-to-1 mapping. Thus, we have a concrete expression for calculating Eqn 7.3.

Next, we need an expression for the similarity estimate $\hat{S}$, given that m out of n hashes have matched so far. Let $\hat{R} = \arg \max_r p(r | M(m, n))$. We can obtain a closed form expression for $\hat{R}$ by solving for $\frac{\partial p(r | M(m, n))}{\partial r} = 0$; when we do this, we get $r = \frac{m}{n}$. Hence, $\hat{R} = \frac{m}{n}$. Now $\hat{S} = r2c(\hat{R})$, therefore $\hat{S} = r2c(\frac{m}{n})$. This is our expression for calculating Eqn 7.4.

Next, let us consider the concentration probability of $\hat{S}$.

$$\begin{aligned}
Pr[|\hat{S} - S| < \delta | M(m, n)] &= Pr[\hat{S} - \delta < S < \hat{S} + \delta | M(m, n)] \\
&= Pr[c2r(\hat{S} - \delta) < c2r(S) < c2r(\hat{S} + \delta) | M(m, n)] \\
&= Pr[c2r(\hat{S} - \delta) < R < c2r(\hat{S} + \delta) | M(m, n)] \\
&= \frac{\int_{c2r(\hat{S}-\delta)}^{c2r(\hat{S}+\delta)} r^m (1-r)^{n-m} dr}{B_1(m+1, n-m+1) - B_{0.5}(m+1, n-m+1)} \\
&= \frac{B_{c2r(\hat{S}+\delta)}(m+1, n-m+1) - B_{c2r(\hat{S}-\delta)}(m+1, n-m+1)}{B_1(m+1, n-m+1) - B_{0.5}(m+1, n-m+1)}
\end{aligned}$$

Thus, we have concrete expressions for Equations 7.3, 7.4 and 7.6, giving us an instantiation of BayesLSH adapted to Cosine similarity.

7.3.3 Optimizations

The basic BayesLSH can be optimized without affecting the correctness of the algorithm in a few ways. The main idea behind the optimizations here is to minimize the number of times inference has to be performed, in particular the Equations 7.3 and 7.6.

Pre-computation of minimum matches: We pre-compute the minimum number of matches a candidate pair needs to have in order for $Pr[S \geq t|M(m, n)] > \epsilon$ to be true, thus completely eliminating the need for any online inference in line 10 of Algorithm 10. For every value of n that we will consider (upto some maximum), we pre-compute the function $minMatches(n)$ defined as follows:

$$minMatches(n) = \arg \min_m Pr[S \geq t|M(m, n)] \geq \epsilon$$

This can be done via binary search, since $Pr[S \geq t|M(m, n)]$ increases monotonically with m for a fixed n . Now, for each candidate pair, we simply check if the actual number of matches for that pair at every n is at least $minMatches(n)$. Note that we will not encounter every possible value of n upto the maximum - instead, since we compare k hashes at a time, we need to compute $minMatches()$ only once for all multiples of k upto the maximum.

Cache results of inference: We maintain a cache indexed by (m, n) that indicates whether or not the similarity estimate that is obtained after m hashes out of n agree is sufficiently concentrated or not (Equation 7.6). Note that for each possible n , we only need to cache the results for $m \geq minMatches(n)$, since lower values of m are guaranteed to result in pruning. Thus, in the vast majority of cases, we can simply fetch the result of the inference from the cache instead of having to perform it afresh.

Cheaper storage of hash functions: For cosine similarity, storing the random gaussian vectors corresponding to each hash function can take up a fair amount of space. To reduce this storage requirement, we developed a scheme for storing each float using only 2 bytes, by exploiting the fact that random gaussian samples from the standard 0-mean, 1-standard deviation gaussian lie well within a small interval around 0. Let us assume that all of our samples will lie within the interval $(-8, 8)$ (it is astronomically unlikely that a sample from the standard gaussian lies outside this interval). For any float $x \in (-8, 8)$, it can be represented as a 2-byte integer

$x' = \lfloor (x + 8) * \frac{2^{16}}{16} \rfloor$. The maximum error of this scheme is 0.0001 for any real number in $(-8, 8)$.

7.3.4 The Influence of Prior vs. Data

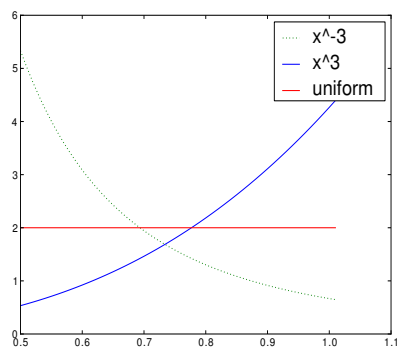
In this section, we show how the observed outcomes (i.e. hashes) are much more influential in determining the posterior distribution than the prior itself. Even if we start with very different prior distributions, the posterior distributions typically become very similar after observing a surprisingly small number of outcomes.

Consider the similarity measure we worked with in the case of cosine similarity, $r(x, y) = 1 - \frac{\theta(x, y)}{\pi}$, which ranges from $[0.5, 1]$ - note that $r(x, y) = 0.5$ corresponds to an actual cosine similarity of 0 between x, y . Consider three very different prior distributions for this similarity measure, as follows (the normalization constants have been omitted):

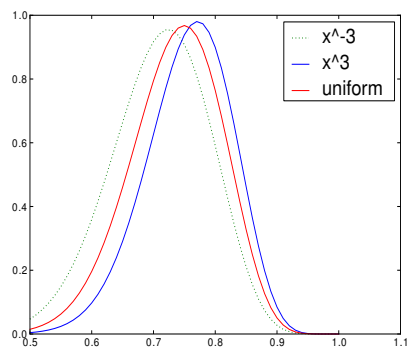
- Negatively sloped power law prior: $p(s) \propto x^{-3}$
- Uniform prior: $p(s) \propto 1$
- Positively sloped power law prior: $p(s) \propto x^3$

In Figures 7.2(a)-7.2(d), we show the posteriors for each of these three priors after observing a hypothetical series of outcomes for a pair of points x, y with cosine similarity 0.70, corresponding to $r(x, y) = 0.75$. Although to start off with, the three priors are very different (see Figure 7.2(a)), the posteriors are already quite close after observing only 32 hashes and 24 agreements (Figure 7.2(b)), and the posteriors get closer quickly with increasing number of hashes (Figures 7.2(c) and 7.2(d)).

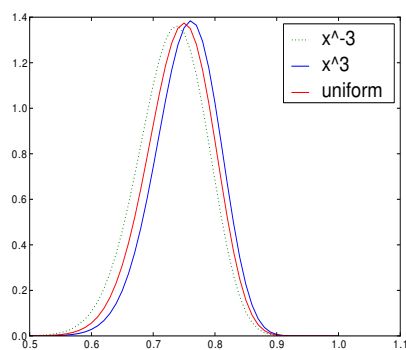
In general, the likelihood term - which, after observing n hashes with m agreements, is $s^m(1 - s)^{(n-m)}$ - is much more sharply concentrated than a justifiable prior, very quickly as we increase n . In other words, a prior would itself have to be very sharply concentrated for it to match the influence of the likelihood - using sharply concentrated priors however brings the danger of not letting the data speak for themselves.



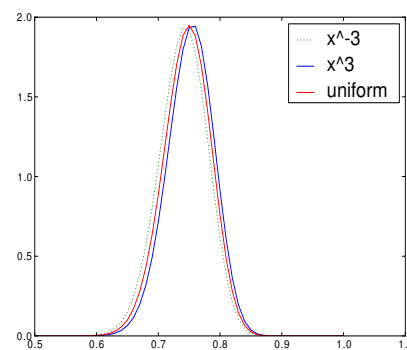
(a) Prior distributions



(b) Posterior after examining 32 hashes, with 24 agreements



(c) Posterior after examining 64 hashes, with 48 agreements



(d) Posterior after examining 128 hashes, with 96 agreements

Figure 7.2: Different priors converge to similar posteriors

7.4 Experiments

We experimentally evaluated the performance of BayesLSH and BayesLSH-Lite on 6 real datasets with widely varying characteristics (see Table 7.1).

- **RCV1** is a text corpus of Reuters articles and is a popular benchmarking corpus for text-categorization research [76]. We use the standard pre-processed version of the dataset with word stemming and tf-idf weighting.
- **Wiki** datasets. We pre-processed the article dump of the English Wikipedia¹⁷ - Sep 2010 version - to produce both a text corpus of Wiki articles as well as the directed graph of hyperlinks between Wiki articles. Our pre-processing includes the removal of stop-words, removal of insignificant articles, and tf-idf weighting (for both the text and the graph). Words occurring at least 20 times in the entire corpus are used as features, resulting in a dimensionality of 344,352. The **WikiWords100K** dataset consists of text vectors with at least 500 non-zero features, of which there are 100,528. The **WikiWords500K** dataset consists of vectors with at least 200 non-zero features, of which there are 494,244. The **WikiLinks** dataset consists of the entire article-article graph among ~1.8M articles, with Tf-Idf weighting.
- **Orkut** consists of a subset of the (undirected) friendship network among nearly 3M Orkut users, made available by [88]. Each user is represented as a weighted vector of their friends, with Tf-Idf weighting.
- **Twitter** consists of the directed graph of follower/followee relationships among the subset of Twitter users with at least 1,000 followers, first collected by Kwak et. al. [69]. Each user is represented as a weighted vector of the users they follow, with Tf-Idf weighting.

We note that all our datasets represent realistic applications for all pairs similarity search. Similarity search on text corpuses can be useful for clustering, semi-supervised learning, near-duplicate detection etc., while similarity search on the graph datasets can be useful for link prediction, friendship recommendation and clustering. Also, in our experiments we primarily focus on similarity search for general real-valued vectors using Cosine similarity, as opposed to similarity search for binary vectors (i.e. sets). Our reasons are as follows:

¹⁷<http://download.wikimedia.org>

Dataset	Vectors	Dimensions	Avg. len	Nnz
RCV1	804,414	47,236	76	61e6
WikiWords100K	100,528	344,352	786	79e6
WikiWords500K	494,244	344,352	398	196e6
WikiLinks	1,815,914	1,815,914	24	44e6
Orkut	3,072,626	3,072,626	76	233e6
Twitter	146,170	146,170	1369	200e6

Table 7.1: Dataset details. Nnz stands for number of non-zeros.

1. Representations of objects as general real-valued vectors are generally more powerful and lead to better similarity assessments, Tf-Idf style representations being the classic example here (see [99] for another example from graph mining).
2. Similarity search is generally harder on real-valued vectors. With binary vectors (sets), most similarity measures are directly proportional to the overlap between the two sets, and it is easier to obtain bounds on the overlap between two sets by inspecting only a few elements of each set, since each element in the set can only contribute the same, fixed number (1) to the overlap. On the other hand, with general real-valued vectors, different elements/features have different weights (also, the same feature may have different weights across different vectors), meaning that it is harder to bound the similarity by inspecting only a few elements of the vector.

7.4.1 Experimental setup

We compare the following methods for all-pairs similarity search.

1. **AllPairs** [12] (AP) is one of the state-of-the-art approaches for all-pairs similarity search, especially for cosine similarity on real-valued vectors. AllPairs is an exact algorithm.
- 2,3. **AP+BayesLSH, AP+BayesLSH-Lite**: These are variants of BayesLSH and BayesLSH-Lite where the input is the candidate set generated by AllPairs.
- 4,5. **LSH, LSH Approx**: These are two variants of the standard LSH approach for all pairs similarity search. For both LSH and LSH Approx, candidate pairs are generated as described in Section 7.1 ; for LSH, similarities are calculated exactly,

whereas for LSH Approx, similarities are instead *estimated* using the standard maximum likelihood estimator, as described in Section 7.2. For LSH Approx, we tuned the number of hashes and set it to 2048 for cosine similarity and 360 for Jaccard similarity. Note that the hashes for Cosine similarity are only bits, while the hashes for Jaccard are integers.

6,7. LSH+BayesLSH, LSH+BayesLSH-Lite: These are variants of BayesLSH that take as input the candidate set generated by LSH as described in Section 7.1.

8. PPJoin+ [122] is a state-of-the-art exact algorithm for all-pairs similarity search, however it only works for binary vectors and we only include it in the experiments with Jaccard and binary cosine similarity.

For all BayesLSH variants, we report the full execution time i.e. including the time for candidate generation. For BayesLSH variants, $\epsilon = \gamma = 0.03$ and $\delta = 0.05$ (γ, δ don't apply to BayesLSH-Lite). For the number of hashes to be compared at a time, k , it makes sense to set this to be a multiple of the word size, since for cosine similarity, each hash is simply a bit. We set $k = 32$, although higher multiples of the word size work well too. In the case of BayesLSH-Lite, the number of hashes to be used for pruning was set to $h = 128$ for Cosine and $h = 64$ for Jaccard. For LSH and LSH Approx, the expected false negative rate is set to 0.03. The randomized algorithms (LSH variants, BayesLSH variants) were each run 3 times and the average results are reported.

All of the methods work for both Cosine and Jaccard similarities, for both real-valued as well as binary vectors, except for PPJoin+, which only works for binary vectors. The code for PPJoin+ was downloaded from the authors' website, all the other methods were implemented by us.¹⁸ All algorithms are single-threaded and are implemented in C/C++. The experiments were run by submitting jobs to a cluster, where each node on the cluster runs on a dual-socket, dual-core 2.3 GHz Opteron with 8GB RAM. Each algorithm was allowed 50 hrs (180K secs) before it was declared timed out and killed.

We executed the different algorithms on both the weighted and binary versions of the datasets, using Cosine similarity for the weighted case and both Jaccard and Cosine for the binary case. For Cosine similarity, we varied the similarity threshold from 0.5 to 0.9, but for Jaccard we found that very few pairs satisfied higher similarity

¹⁸Our AllPairs implementation is slightly faster than the original implementation of the authors due to a simple implementational fix. This has since been incorporated into the authors' implementation.

Dataset	Fastest BayesLSH	Speedup w.r.t baselines			
		AP	LSH	LSH Approx	PPJoin
variant Tf-Idf, Cosine					
RCV1	LSH + BayesLSH	7.1x	4.8x	2.4x	-
WikiWords-100K	LSH + BayesLSH	31.4x	15.1x	2.0x	-
WikiWords-500K	LSH + BayesLSH	$\geq 42.1x$	$\geq 13.3x$	2.8x	-
WikiLinks	AP + BayesLSH-Lite	1.8x	$\geq 248.2x$	$\geq 246.3x$	-
Orkut	AP + BayesLSH-Lite	1.2x	$\geq 114.9x$	$\geq 155.6x$	-
Twitter	LSH + BayesLSH	26.7x	33.4x	3.0x	-
Binary, Jaccard					
WikiWords-500K	LSH + BayesLSH	2.0x	$\geq 16.8x$	3.7x	5.2x
Orkut	AP + BayesLSH-Lite	0.8x	2.9x	2.8x	1.1x
Twitter	LSH + BayesLSH	1.8x	48.4x	4.2x	8.0x
Binary, Cosine					
WikiWords-500K	LSH + BayesLSH	2.3x	$\geq 10.2x$	1.2x	5.6x
Orkut	AP + BayesLSH-Lite	0.8x	$\geq 201x$	$\geq 201x$	1.0x
Twitter	AP + BayesLSH-Lite	1.2x	27.4x	1.2x	3.7x

Table 7.2: Comparison of fastest BayesLSH variant with baselines.

thresholds (e.g. for Orkut, a 3M record dataset, only 1648 pairs were returned at threshold 0.9), and hence varied the threshold from 0.3 to 0.7. For Jaccard and Binary Cosine, we only report results on WikiWords500K, Orkut and Twitter, which are our three largest datasets in terms of total number of non-zeros.

7.4.2 Results comparing BayesLSH variants with baselines

Figures 7.3 and 7.4 show a 'comparison of timing results for all algorithms across a variety of datasets and thresholds. Table 7.2 compares the fastest BayesLSH variant with all the baselines. The quality of the output of BayesLSH can be seen in Table 7.3 where we show the recall rates for AP+BayesLSH and AP+BayesLSH-Lite, and in Table 7.4 where we compare the accuracies of LSH and LSH+BayesLSH. The recall and accuracies of the other BayesLSH variants follow similar trends and are omitted. The main trends from the results are distilled and discussed below:

1. BayesLSH and BayesLSH-Lite improve the running time of both AllPairs and LSH in almost all the cases, with speedups usually in the range **2x-20x**. It can be seen from Table 7.2 that a BayesLSH variant is the fastest algorithm (in terms of total time across all thresholds) for the majority of datasets and similarities, with

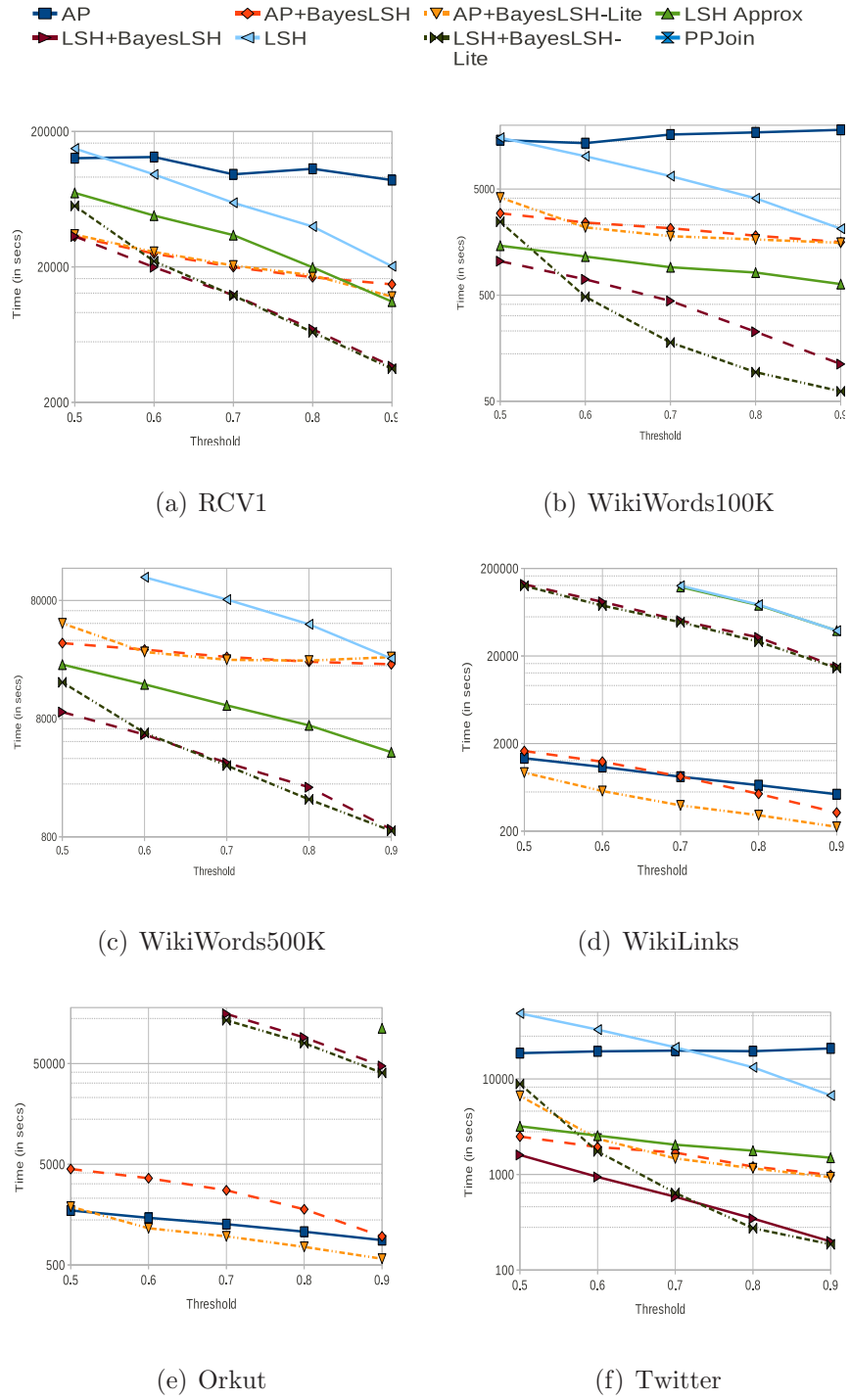


Figure 7.3: Timing comparisons between different algorithms.

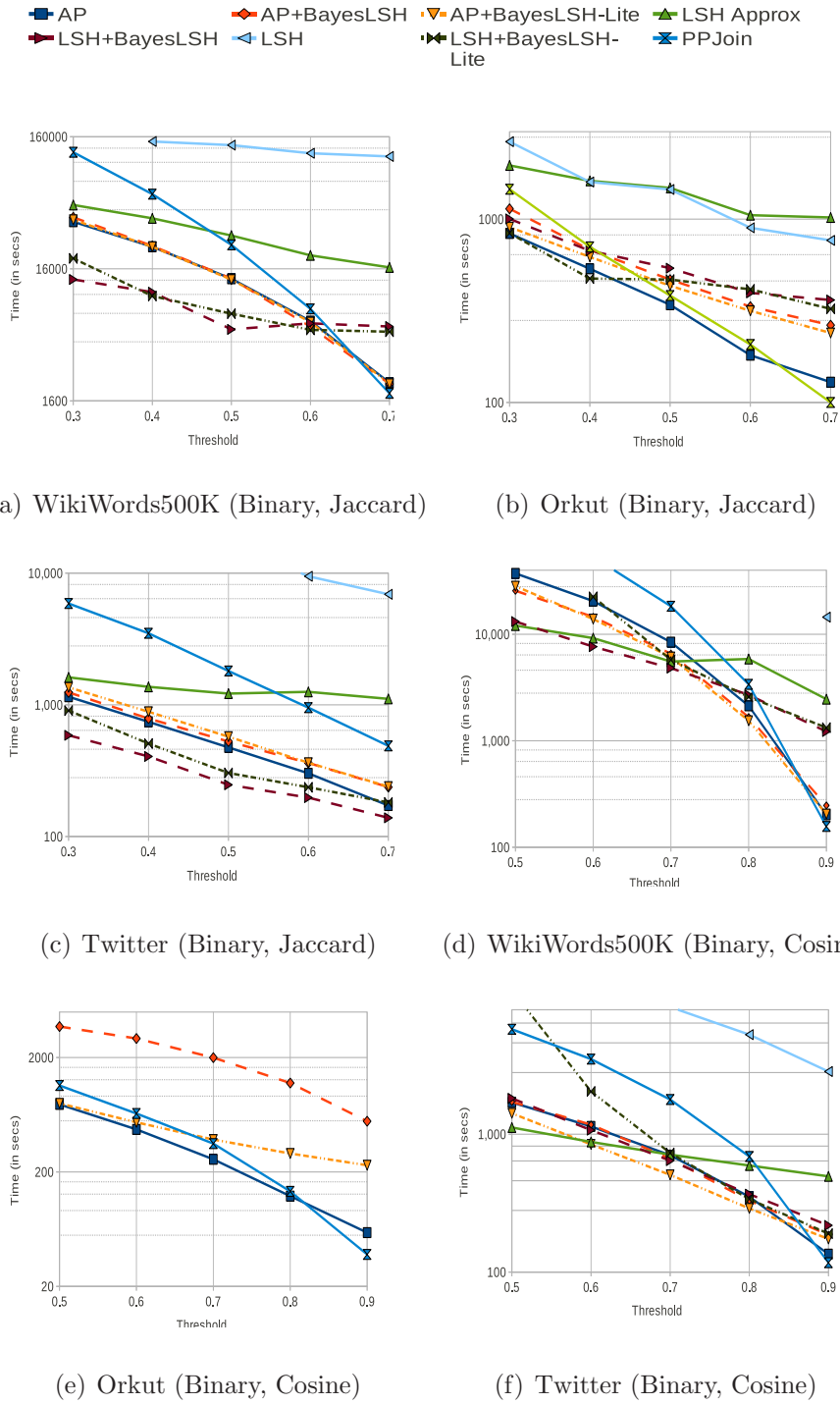


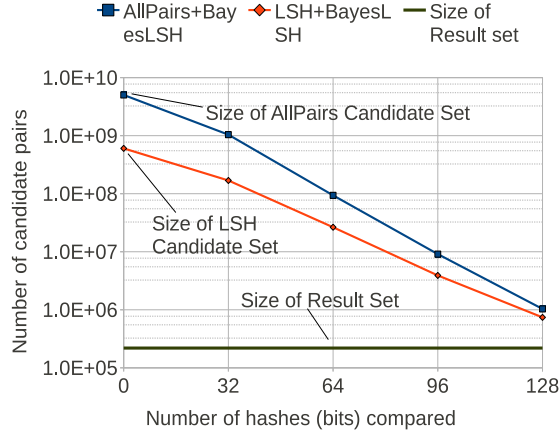
Figure 7.4: Timing comparisons between different algorithms (binary datasets).

Dataset	t=0.5	t=0.6	t=0.7	t=0.8	t=0.9
AllPairs+BayesLSH					
RCV1	97.97	98.18	98.47	99.08	99.36
WikiWords100K	98.52	98.84	99.2	98.58	96.69
WikiWords500K	97.54	97.82	98.21	98.16	96.66
WikiLinks	97.45	98.04	98.46	98.68	99.18
Orkut	97.1	97.8	98.86	99.84	99.99
Twitter	97.7	96	96.88	97.33	98.77
AllPairs+BayesLSH-Lite					
RCV1	98.73	98.82	98.89	99.26	99.55
WikiWords100K	98.88	99.31	99.62	99.69	99.5
WikiWords500K	98.79	98.72	98.98	98.74	98.83
WikiLinks	98.53	98.91	99.16	99.18	99.45
Orkut	98.4	98.64	99.3	99.87	99.99
Twitter	99.44	98.82	97.17	97.18	99.06

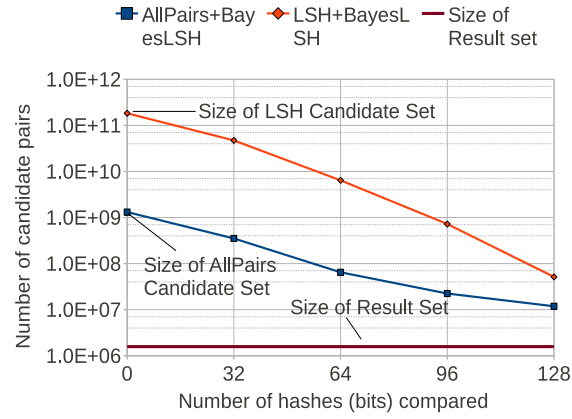
Table 7.3: Recalls for AP+BayesLSH and AP+BayesLSH-Lite.

	t=0.5	t=0.6	t=0.7	t=0.8	t=0.9
LSH Approx					
RCV1	7.8	4.3	2.25	0.8	0.04
WikiWords100K	4.7	3.6	1	0.3	0.02
WikiWords500K	8.3	5.7	2.9	0.9	0.1
WikiLinks	-	-	1.6	0.4	0.06
Orkut	-	-	-	-	0.0072
Twitter	4	5.1	2.6	0.4	0.02
LSH + BayesLSH					
RCV1	3.2	2.9	3.2	2	1.4
WikiWords100K	2.7	2.3	3.5	4.9	2.2
WikiWords500K	3.4	3.4	3.2	2.9	2.1
WikiLinks	2.96	2.82	2.3	2	1.6
Orkut	-	-	1.5	0.6	0.09
Twitter	2.3	4	3.1	4.8	4.3

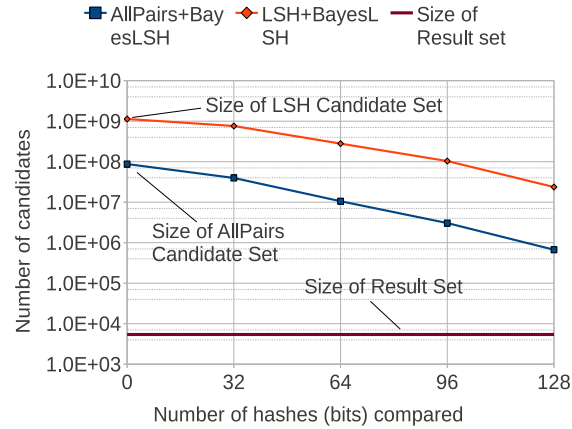
Table 7.4: Percentage of similarity estimates with errors greater than 0.05



(a) WikiWords100K, $t=0.7$, Cosine



(b) WikiLinks, $t=0.7$, Cosine



(c) WikiWords100K, $t=0.7$, Binary Cosine

Figure 7.5: The pruning power of BayesLSH.

Parameter value	Fraction errors > 0.05 for varying γ	Mean error for varying δ	Recall for varying ϵ
0.01	0.7%	0.001	98.76%
0.03	2%	0.01	97.79%
0.05	3%	0.017	97.33%
0.07	4.2%	0.022	96.06%
0.09	5.4%	0.027	95.35%

Table 7.5: The effect of varying the parameters γ, δ, ϵ . WikiWords100K, $t=0.7$

the exception of Orkut for Jaccard and binary cosine. Furthermore, the quality of BayesLSH output is high; the recall rates are usually above 97% (see Table 7.3), and similarity estimates are accurate, with usually no more than 5% output pairs with error above 0.05 (see Table 7.4).

2. BayesLSH is fast primarily by being able to prune away the vast majority of false positives after comparing only a few hashes. This is illustrated in Figure 7.5. For WikiWords100K at a threshold of 0.7, (see Figure 7.5(a)) AllPairs supplies BayesLSH with nearly $5e09$ candidates, while the result set only has $2.2e05$. BayesLSH is able to prune away **$4.0e+09$** (80%) of the input candidate pairs after examining only 32 hashes - in this case, each hash is a bit, so BayesLSH compared only 4 bytes worth of hashes between each pair. By the time BayesLSH has compared 128 hashes (16 bytes) there are only $1.0e06$ candidates remaining. Similarly LSH supplies BayesLSH with $6.0e08$ candidates - better than AllPairs, but nonetheless orders of magnitude larger than the final result set - and after comparing 128 hashes (16 bytes), BayesLSH is able to prune that down to only $7.4e05$, only about 3.5x larger than the result set. On the WikiLinks dataset (see Figure 7.5(b)), we see a similar trend with the roles of AllPairs and LSH reversed - this time it is AllPairs instead which supplies BayesLSH with fewer candidates. After examining only 128 hashes, BayesLSH is able to reduce the number of candidates from $1.3e09$ down to $1.2e07$ for AllPairs, and from $1.8e11$ down to $5.1e07$ for LSH. Figure 7.5(c) shows a similar trend, this time on the binary version of WikiWords100K.

3. We note that BayesLSH and BayesLSH-Lite often (but not always) have comparable speeds, since most of the speed benefit is coming from the ability of BayesLSH to prune, which is an aspect that is common to both algorithms. The difference between the two is mainly in terms of the hashing overhead. BayesLSH needs to obtain many more hashes of each object in order for similarity estimation; this cost is amortized at lower thresholds, where the number of similarity calculations needed to perform is much greater. BayesLSH-Lite is faster at higher thresholds or when exact similarity calculations are cheaper, such as datasets with low average vector length.
4. AllPairs and LSH have complementary strengths and weaknesses. On the datasets RCV1, WikiWords100K, WikiWords500K and Twitter (see Figures 7.3(a)-7.3(c), 7.3(f)), LSH is clearly the faster algorithm than AllPairs (in the case of WikiWords500K, AllPairs did not finish execution even for the highest threshold of 0.9). On the other hand, AllPairs is the much faster algorithm on WikiLinks and Orkut (see Figures 7.3(d)-7.3(e)), with LSH timing out in most cases. Looking at the characteristics of the datasets, one can discern a pattern: AllPairs is faster on datasets with smaller average length and greater variance in the vector lengths, as is the case with the graph datasets WikiLinks and Orkut. The variance in the vector lengths allows AllPairs to upper-bound the similarity better and thus prune away more false positives, and in addition the exact similarity computations that AllPairs does are faster when the average vector length is smaller. However, BayesLSH and BayesLSH-Lite enable speedups on *both* AllPairs and LSH, not only when each algorithm is slow, but even when each algorithm is *already fast*.
5. The accuracy of BayesLSH’s similarity estimates is much more consistent as compared to the standard LSH approximation, as can be seen from Table 7.4. LSH generally produces too many errors when the threshold is low and too few errors when the threshold is high. This is mainly because LSH uses the same number of hashes (set to 2048) for estimating all similarities, low and high. This problem would persist even if the number of hashes was set to some other value, as explained in Section 7.2.1. BayesLSH, on the other hand, maintains similar accuracies at both low and high thresholds, *without requiring any tuning at all on the number of hashes to be compared*, and only based on the user’s specification of the desired accuracy using δ, γ parameters.

6. LSH Approx is often much faster than LSH with exact similarity calculations, especially for datasets with higher average vector lengths, where the speedup is often 3x or more - on Twitter, the speedup is as much as 10x (see Figure 7.3(f)).
7. BayesLSH does not enable speedups that are as significant for AllPairs in the case of binary vectors. We found that this was because AllPairs was already doing a very good job at generating a small candidate set, thus not leaving much room for improvement. In contrast, LSH was still generating a large candidate set, leaving room for LSH+BayesLSH to enable speedups. Interestingly, even though LSH generates about 10 times more candidates than AllPairs, the LSH variants of BayesLSH are about 50-100% *faster* than AllPairs and its BayesLSH versions, on WikiWords500K and Twitter (see Figures 7.4(a),7.4(c)). This is because LSH is a faster indexing and candidate generation strategy, especially when the average vector length is large.
8. PPJoin+ is often the fastest algorithm at the highest thresholds (see Figures 7.4(a)-7.4(f)), but its performance degrades very rapidly with lower thresholds. A possible explanation is that the pruning heuristics used in PPJoin+ are effective only at higher thresholds.
9. We performed a small experiment to quantify the effect of the approximation error of BayesLSH. On the Twitter dataset, we ran both AllPairs, and LSH+BayesLSH, with similarity thresholds 0.9 and 0.8. We clustered the resulting similarity weighted graph using MLR-MCL, and compared the results of clustering the graph output by AllPairs (which is exact), versus clustering the graph output by LSH+BayesLSH. The agreement in the output of the clusterings, quantified using Avg. Weighted F-scores, was 92.9% and 92.1% respectively, indicating that the clusterings are highly similar.

7.4.3 Effect of varying parameters of BayesLSH

We next examine the effect of varying the parameters of BayesLSH - namely the accuracy parameters γ, δ and the recall parameter ϵ . We vary each parameter from 0.01 to 0.09 in increments of 0.02, while fixing the other two parameters to 0.05, and fix the dataset to WikiWords100K and threshold to 0.7 (cosine similarity). The effect of varying each of these parameters on the execution time is plotted in Figure 7.6. Varying the recall parameter ϵ and the accuracy parameter γ have barely any effect on the running time - however setting δ to lower values does increase the running time significantly. Why does lowering δ penalize the running time much more than

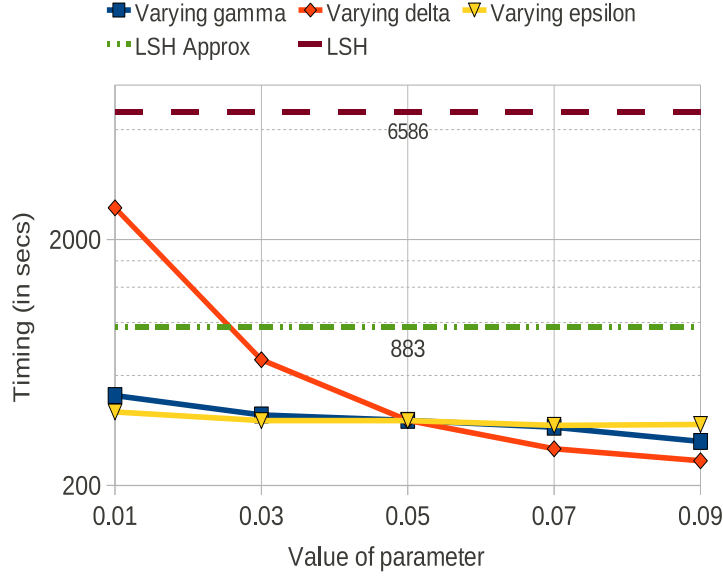


Figure 7.6: LSH+BayesLSH - varying γ, δ, ϵ on WikiWords100K, $t=0.7$.

lowering γ ? This is because lowering δ increases the number of hashes that have to be compared for *all* result pairs, while lowering γ increases the number of hashes that have to be compared only for those result pairs that have uncertain similarity estimates. It is interesting to note that even though $\delta = 0.01$ requires 2691 secs, it achieves a very low mean error of 0.001, while being much faster than LSH exact, which requires 6586 secs. Approximate LSH requires 883 secs but is much more error-prone, with a mean error of 0.014. With $\gamma = 0.01$, BayesLSH achieves a mean error of 0.013, while still being around 2x faster than approximate LSH.

In Table 7.5, we show the result of varying these parameters on the output quality. When varying a parameter, we show the change in output quality only for the relevant quality metric - e.g. for changing γ we only show how the fraction of errors > 0.05 changes, since we find that recall is largely unaffected by changes in γ and δ (which is as it should be). Looking at the column corresponding to varying γ , we find that the fraction of errors > 0.05 increases as we expect it to when we increase γ , without ever exceeding γ itself. When varying δ , we can see that the mean error reduces as expected for lower values of δ . Finally, when varying the recall parameter ϵ , we find

that the recall reduces with higher values of ϵ as expected, with the false negative rate always less than ϵ itself.

7.5 Conclusions

In this chapter, we have presented BayesLSH (and a simple variant BayesLSH-Lite), a general candidate verification and similarity estimation algorithm for approximate similarity search, which combines Bayesian inference with LSH in a principled manner and has a number of advantages compared to standard similarity estimation using LSH. BayesLSH takes a largely orthogonal direction to a lot of recent research in LSH, which concentrates on more effective indexing strategies, ultimately with the goal of candidate generation, such as Multi-probe LSH [80] and LSB-trees [111]. Furthermore, a lot of research on LSH is concentrated on nearest-neighbor retrieval for distance measures, rather than all pairs similarity search with a similarity threshold t . BayesLSH enables significant speedups for two state-of-the-art candidate generation algorithms, AllPairs and LSH, across a wide variety of datasets, and furthermore the quality of BayesLSH is easy to tune. As can be seen from Table 7.2, a BayesLSH variant is typically the fastest algorithm on a variety of datasets and similarity measures. A significant advantage of BayesLSH is that it can be adapted to any similarity function as long as it has an associated locality sensitive hashing family.

Chapter 8: Conclusions and Future Work

In this dissertation, we have focused on a fundamental analytical task related to graphs - the discovery of the natural groups or clusters from a graph, or graph clustering in short. We have discussed the limitations of existing approaches to this problem, particularly due to the changing nature of the graphs arising from some of the common domains, such as social and information networks, and the world wide web. The central contention of this thesis is that *the discovery of clusters from modern graph data can be done effectively and efficiently by using a combination of pre-processing algorithms that clarify the local similarity structure of the input data and clustering algorithms based on multi-level simulations of stochastic flows*. In terms of pre-processing algorithms that clarify the local similarity structure of the input data, we have proposed (i) Local graph sparsification for large, noisy graphs, (ii) Symmetrizations for directed graphs, and (iii) Bayesian Locality Sensitive Hashing algorithms for nearest neighbors from general non-graph data. In terms of clustering algorithms based on multi-level simulations of stochastic flows, we have proposed Regularized MCL and Multi-Level Regularized MCL. Taken together, this dissertation provides solutions for the clustering problem that are commensurate with the complexity, scale and noisiness of modern data. Equally importantly, this body of work provides a set of building blocks which can be used to develop algorithms for more complex and novel data mining problems.

Next, we summarize the important innovations and experimental benefits obtained from each of our contributions:

- **Graph Clustering via Multi-level simulation of stochastic flows**

Chapters 3, 4

In this work, we started with analyzing a popular graph clustering algorithm called Markov Clustering (MCL) [41], which works via an intuitive process of repeated application of two simple operators, *Expand* and *Inflate*, on the

stochastic flow (or transition probability) matrix of the input graph. MCL is especially popular within Bioinformatics for its simplicity and noise-tolerance; however, it is too slow for large graphs and produces very imbalanced clusters for many kinds of modern social and information networks. We introduced two innovations into MCL in order to deal with these two problems. The first was the replacement of the Expand operator with a theoretically grounded *Regularize* operator which significantly improved the accuracy and balance of the output clusters. The second was embedding the entire process in a multi-level framework, so that the algorithm first operates on a small, very coarsened version of the original graph and then successively operates on bigger, less coarse versions culminating with the original graph itself. The multi-level framework ensures that the global structure of the graph influences the final result as well as significantly accelerates the overall algorithm, since the first few iterations, which are the most expensive, are run on small graphs. The final algorithm, called Multi-level Regularized Markov Clustering (MLR-MCL), is around **two orders of magnitude** faster than the original MCL and also much more accurate and balanced. MLR-MCL was found to improve on existing approaches such as Metis, Graclus, Metis+MQI, Spectral and MCL on a variety of graphs including protein interaction networks, online social (Orkut, Twitter) and information networks (Wikipedia, Flickr) as well as similarity-weighted graphs of standard text corpora (Twenty20). For example, on the 1.2 million node Wikipedia network, MLR-MCL is **50%** more accurate than Metis while running comparably fast, and is 5% more accurate than Metis+MQI while running **5x** faster. The software that we have developed for MLR-MCL has been used by many other researchers in different contexts. For example, VLSI CAD researchers from a major chip manufacturer¹⁹ used MLR-MCL to much benefit and stated that “*MLR-MCL was the fastest program from its kind and fully reliable [...] and holds great promise for advancing VLSI automation.*”

- **Local Graph Sparsification as a Pre-processing strategy**

Chapter 5

Large-scale modern networks are difficult to cluster not just due to the sheer size of the networks, but also because they have many noisy edges and hub nodes

¹⁹unnamed due to confidentiality reasons; suffice to say that you would have heard of them.

which obfuscate the cluster structure of the graph. We developed a simple pre-processing strategy that sparsifies the input graph so that the resulting graph has far fewer edges yet retains a clarified essence of the cluster structure in the original graph. It relies on a simple heuristic ranking of edges based on the similarity in the adjacency lists of the two nodes incident on each edge, and which can be approximated efficiently using a hashing trick. A crucial innovation here is to retain the top fraction of edges for each node and discard the others, rather than retain the top fraction of edges for the graph as a whole. This local aspect of the sparsification ensures that clusters of varying densities are well represented, as well as allows us to diminish the influence of hub nodes by retaining a smaller fraction of edges incident on high-degree nodes. This simple method enables **20x-50x** speedups on large real world networks while at the same time improving accuracy by **50%** or more on noisy graphs. It also compresses the graph to **20%** or lesser of its original size, with larger compression ratios for denser graphs. The algorithm can be implemented in only two passes over the input graph and hence is well suited as a pre-processing step for large disk-resident graphs. Furthermore, it can also be used to more easily visualize the cluster structure in the graph.

- **Symmetrizations for clustering directed graphs**

Chapter 6

A large fraction of real world networks are directed e.g. the Twitter follower network, the WWW itself, citation networks etc. Our work on clustering directed graphs was motivated by the observation that existing methods (implicitly) use a similarity function that does not take into account the true semantics of directed edges in real world graphs. To rectify this, we proposed to cluster directed graphs by first explicitly converting the input directed graph into a similarity-weighted undirected graph using a well motivated similarity function, a process we refer to as a *symmetrization*. The symmetrized graph can subsequently be clustered using an off-the-shelf clustering algorithm. The similarity function we proposed takes into account how similar the in-links and out-links of two nodes are, while also weighting the contribution of each shared link in inverse proportion to the degree of the nodes involved. On a real world citation network, our process of symmetrization followed by clustering is around **2 orders of magnitude** faster than a state-of-the-art spectral algorithm for

clustering directed graphs while also being **20%** more accurate. Similarly, our proposed method is also faster and more accurate than the naive strategy of ignoring edge direction.

- **Bayesian Locality Sensitive Hashing for Fast Nearest Neighbors**

Chapter 7

An important pre-requisite to clustering non-graph data (e.g. a set of text documents or images) is the generation of a similarity-weighted graph or a similarity matrix given a similarity function and the set of objects. For example, each object may be represented as a vector in some multi-dimensional space, and the cosine of the angle between two vectors may be used as their similarity. Given n objects, it is neither necessary nor practical to generate all n^2 similarities, instead we often only care about similarities above a certain user-specified similarity threshold. Even with a similarity threshold, this is a tough problem to solve, especially in high dimensions and with increasing number of vectors. A breakthrough invention in this area is Locality-Sensitive Hashing (LSH) [59] which enables practical indexing and retrieval of near neighbors. However, we found that LSH (and other existing solutions) still generate orders of magnitude more candidate neighbors than the true result, necessitating powerful algorithms for quickly pruning the generated candidates.

For this purpose, we developed BayesLSH, a Bayesian algorithm that exploits LSH for pruning the generated candidates efficiently, as well as for estimation of the actual similarities (rather than exact computation) with probabilistic guarantees on the quality of the final result. BayesLSH is a natural fit for the many applications in which LSH is already being used for similarity search, and it further amortizes the overhead of hashing the objects initially by deriving additional benefit from the hashes beyond candidate generation. BayesLSH exploits, using a principled Bayesian approach, the following intuition: if the number of hashes that match for a generated pair of candidates is small, then this candidate is probably a false positive. BayesLSH is able to prune the vast majority of false positives very quickly by inspecting only a few hashes, leading to significant speedups, in the range **2x-20x**, over state-of-the-art solutions, while at the same time maintaining high recall and accuracy in the similarity approximations.

8.1 Future work

The long-term goal of the research in this dissertation is the scalable discovery of structure with minimal supervision for diverse, complex domains and fields such as the World Wide Web, modern social science and computational biology. The understanding of the structure that characterizes a domain is helpful both to human experts, who can use this knowledge to formulate novel hypotheses (as happens regularly in Biology), and to computer algorithms, which can use it to solve even bigger tasks (as happens regularly in A.I.). There are a number of important directions in which we plan to extend the current work.

8.1.1 Parallel algorithms for large-scale data

Both the increasing scales of data and trends in computing infrastructure are pointing towards the necessity of data mining algorithms that can run on the cloud and on modern multi-core and GPU architectures. We intend to develop parallel algorithms, beginning with the methods proposed in this thesis for graph clustering, pre-processing such as sparsification, symmetrization and similarity search but eventually covering a wider range of graph analysis tasks. An important challenge for parallel algorithms on both MapReduce and GPU is the problem of skew [123] - for example, degree distributions in many modern networks are famously heavy-tailed. A pre-processing strategy such as local graph sparsification (Chapter 5) can be significantly helpful here, as it outputs a graph with a more balanced degree distribution. A second challenge is to distribute the data intelligently so as to minimize dependencies across different compute nodes [57]. This can be a chicken-and-egg problem since this is very similar to the goal of the clustering itself, but I believe that progress can be made here by exploiting multi-level tricks, where we cluster a small summary of the input data first, and use it to drive the partition of the entire data. Another promising possibility here is to use locality sensitive hashing-like tricks for very approximate data partitioning but which may nonetheless improve over random partitioning.

To make some of the above discussion concrete, here we give the outlines of how MLR-MCL can be parallelized using the popular MapReduce framework [35] for parallel computing. We assume that each processing element (node) stores the adjacency lists corresponding to a fraction of the vertices in the graph. We point out the easy parts first: the inflate/prune/normalize steps, are very easy to parallelize, since these

steps can be executed on each column independent of the other columns. The coarsening step of MLR-MCL can be done by coarsening the portion of the graph that is stored on each node of the cluster independently, essentially ignoring the edges that connect the vertices stored on different nodes of the cluster. This strategy is simple, but may end up being very sub-optimal. An alternative strategy works by recognizing that a coarsening is essentially an edge-matching, and that an independent set on the vertices of the line graph transformation of the input graph is equivalent to an edge matching in the original graph. There exist efficient parallel randomized algorithms for computing a maximal independent set of the vertices of a graph [68]. The most compute-intensive aspect of stochastic-flow clustering algorithms, be it MCL or MLR-MCL, is the matrix multiplication step in the *Expand/Regularize* phase of each iteration. This is also the phase that is the most non-trivial to parallelize, primarily because of the sparsity and irregularity of transition matrices representing modern web-scale graphs. By thinking of each vertex in the graph to be a parallel process (in a manner similar to Google’s Pregel[83]), the regularization step of MLR-MCL can be described as involving the (asynchronous) communication of probability distributions between the processes of neighboring vertices, followed by a local reduction to obtain the final probability distribution at the end of the *Regularize* step. This high-level outlook has similarities with parallel PageRank implementations, where each vertex in the graph communicates with neighboring vertices and performs a local reduction to obtain the new PageRank score at that vertex.

8.1.2 Novel data models and clustering variants

While undirected and directed graphs are flexible data models, there are increasingly domains where more complex data models - such as graphs with attributes on the nodes and edges, graphs with multiple kinds of nodes (e.g. k -partite graphs) - are appropriate. Before we consider the question of how to cluster such data, we must first answer the question of what the most appropriate notion of similarity is to the domain or task at hand. In this context, recent work on learning similarity measures from a small number of labeled examples is very relevant [34]. For constructing similarity-weighted graphs using the learned similarity measures, the techniques used in BayesLSH (Chapter 7) provide a promising starting point. Off-the-shelf clustering algorithms can be used in the final stage to cluster the similarity-weighted graphs.

Coming to variants of the traditional clustering problem, one important problem here is to generate meaningful overlapping clustering arrangements to accommodate the cases when each node is potentially part of multiple groups. The general pre-processing followed by clustering pipeline advocated in this thesis can be again brought to bear here, with a line graph transformation with suitable weighting of the edges followed by off-the-shelf clustering algorithms. This approach has shown promising results in the recent literature [4] - however the main drawback here is scalability. We believe that this line of work can be improved in significant ways by using BayesLSH-like similarity search algorithms for hastening the line graph transformation as well as by using MLR-MCL as a clustering algorithm in the second stage.

Another variant of the traditional clustering problem, is to cluster dynamic (e.g. time-evolving) graphs. We have some promising preliminary ideas for solving this problem by adapting the existing MLR-MCL algorithm. MLR-MCL can be simultaneously executed on multiple graph snapshots, with the iterations proceeding in lock-step fashion. The important difference is that the *Regularize* operator can be modified so as to minimize not only the KL divergences between a node and its neighbors but also the KL divergences between the flow distributions of the same node from the temporally neighboring graph snapshots. One of the important issues in dynamic graph clustering is a mechanism for trading off the quality of clustering w.r.t. the current snapshot versus the change in the clustering w.r.t. the clusterings from previous/neighboring snapshots. Such trade-offs may be expressed in our framework via the use of weights for the KL divergences between flow distributions corresponding to different versions of the same node.

An interesting general purpose framework in which to embed many of these algorithms is to operate on ensembles of samples of the input graph. One can extend the local sparsification discussed in Chapter 5 to a probabilistic variant, and use this to build multiple, smaller replicas of the input graph. By performing model averaging (or its relevant analogues) on the models built on each of these samples, one can obtain a robust solution [49], in a manner reminiscent of the prior work on ensemble clustering [110].

8.1.3 Dimensionality Reduction, Similarity Search and Clustering

Three of the canonical tasks in unsupervised learning are dimensionality reduction, similarity search and clustering. While the connections between these three fundamental tasks of unsupervised learning have long been clear to researchers, these connections have primarily been exploited in one direction, e.g. rounding low dimensional representations for clustering, or lower dimensional representations for similarity search. However, recent research has started to show that the connections can be exploited in the opposite direction e.g. clustering can be used to improve dimensionality reduction [101], and similarity search is a vital pre-requisite for non-linear dimensionality reduction [14]. We would like to explore these connections in depth: under what conditions is it advantageous to use dimensionality reduction as a precursor to clustering, and when to use clustering to improve dimensionality reduction? When it comes to clustering and similarity search, researchers have similarly exploited the connections in both directions, which raises the interesting question: can we perform clustering and similarity search incrementally, using the results from clustering to inform similarity search and vice versa? To give some more concreteness to the idea, consider that similarity search can be performed in layers, by starting with a high similarity threshold, which will help us identify the densest parts of the input space, and thereby the densest clusters in the data. We subsequently perform similarity search with a lower similarity threshold, but this time leveraging the clusters found earlier to speed up the similarity search at this threshold, and identifying the next level of clusters from the results of the similarity search. Yet another direction in which similarity search could be exploited to yield insights into the data is to analyze the similarity-weighted graphs at successive thresholds for their global connectivity. For example, at high similarity thresholds, the graph of output pairs will be highly disconnected, but as one lowers the threshold, there will likely be a point at which the graph of output pairs will become connected. Knowing the point at which such a “phase transition” occurs, as well as the rate and nature in which the graph connectivity increases with lowering threshold, can yield important insights into the underlying data.

Another important direction in which we plan to extend BayesLSH is to similarity functions other than Jaccard and Cosine. Recently, there has been work showing the existence of Locality Sensitive Hashing families for Mahalanobis metrics [60] and

general kernelized similarity functions [67]. Since BayesLSH does not make any assumptions about the specific form of the hash functions, as long as they satisfy the locality sensitive property, we expect we should be able to speed up similarity search for these new and general similarity functions using BayesLSH.

Bibliography

- [1] *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Microsoft Research, 2009.
- [2] D. Achlioptas and F. McSherry. Fast computation of low rank matrix approximations. In *STOC '01*, pages 611–618, 2001.
- [3] S. Agarwal, N. Snavely, I. Simon, S.M. Seitz, and R. Szeliski. Building rome in a day. In *ICCV*, pages 72–79, 2009.
- [4] Y.Y. Ahn, J.P. Bagrow, and S. Lehmann. Link communities reveal multiscale complexity in networks. *Nature*, 466(7307):761–764, 2010.
- [5] R. Andersen, F. Chung, and K. Lang. Local graph partitioning using pagerank vectors. In *FOCS '06: Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science*, pages 475–486, Washington, DC, USA, 2006. IEEE Computer Society.
- [6] R. Andersen, F. R. K. Chung, and K. J. Lang. Local partitioning for directed graphs using pagerank. In *WAW*, pages 166–178, 2007.
- [7] R. Andersen and K.J. Lang. Communities from seed sets. In *WWW '06: Proceedings of the 15th international conference on World Wide Web*, page 232. ACM, 2006.
- [8] A. Andoni and P. Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Communications of the ACM*, 51:117–122, 2008.
- [9] V. Arnau, S. Mars, and I. Marin. Iterative Cluster analysis of protein interaction data. *Bioinformatics*, 21(3):364–78, 2005.
- [10] S. Arora, E. Hazan, and S. Kale. A fast random sampling algorithm for sparsifying matrices. *APPROX-RANDOM '06*, pages 272–279, 2006.

- [11] S.T. Barnard and H.D. Simon. Fast multilevel implementation of recursive spectral bisection for partitioning unstructured problems. *Concurrency Practice and Experience*, 6(2):101–118, 1994.
- [12] R.J. Bayardo, Y. Ma, and R. Srikant. Scaling up all pairs similarity search. In *WWW*, 2007.
- [13] L. Becchetti, P. Boldi, C. Castillo, and A. Gionis. Efficient semi-streaming algorithms for local triangle counting in massive graphs. In *KDD '08*, pages 16–24, 2008.
- [14] M. Belkin and P. Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6), 2003.
- [15] A. A. Benczúr and D. R. Karger. Approximating s-t minimum cuts in $O(n^2)$ time. In *STOC '96*, pages 47–55, 1996.
- [16] A. Beyer and T. Wilhelm. Dynamic simulation of protein complex formation on a genomic scale. *Bioinformatics*, 21(8):1610, 2005.
- [17] T. Bohman, C. Cooper, and A. Frieze. Min-Wise independent linear permutations. *The Electronic Journal of Combinatorics*, 7(R26):2, 2000.
- [18] T. Bohman, C. Cooper, and A. Frieze. Min-Wise independent linear permutations. *The Electronic Journal of Combinatorics*, 7(R26):2, 2000.
- [19] U. Brandes, D. Delling, M. Gaertler, R. Görke, M. Hoefer, Z. Nikoloski, and D. Wagner. On finding graph clusterings with maximum modularity. In *Graph-Theoretic Concepts in Computer Science*, pages 121–132. Springer, 2007.
- [20] A. Z. Broder, M. Charikar, A. M. Frieze, and M. Mitzenmacher. Min-wise independent permutations (extended abstract). In *STOC '98: Proceedings of the thirtieth annual ACM symposium on Theory of computing*, pages 327–336, New York, NY, USA, 1998. ACM.
- [21] A. Z. Broder, Steven C. Glassman, M. S. Manasse, and Geoffrey Zweig. Syntactic clustering of the web. In *WWW*, 1997.
- [22] S. Brohee and J. van Helden. Evaluation of clustering algorithms for protein-protein interaction networks. *BMC Bioinformatics*, 7, 2006.
- [23] G. Buehrer and K. Chellapilla. A scalable pattern mining approach to web graph compression with communities. In *WSDM '08: Proceedings of the international conference on Web search and web data mining*, pages 95–106, New York, NY, USA, 2008. ACM.

- [24] D. Chakrabarti and C. Faloutsos. Graph mining: Laws, generators, and algorithms. *ACM Comput. Surv.*, 38(1):2, 2006.
- [25] M. S. Charikar. Similarity estimation techniques from rounding algorithms. In *STOC '02*, 2002.
- [26] D. Cheng, R. Kannan, S. Vempala, and G. Wang. On a recursive spectral algorithm for clustering from pairwise similarities. 2003.
- [27] F. Chung. Spectral graph theory. *CBMS Regional Conference Series in Mathematics*, 1997.
- [28] F. Chung. Laplacians and the Cheeger inequality for directed graphs. *Annals of Combinatorics*, 9(1):1–19, 2005.
- [29] A. Clauset, M.E.J. Newman, and C. Moore. Finding community structure in very large networks. *Physical review E*, 70(6):066111, 2004.
- [30] A. Clauset, C.R. Shalizi, and M.E.J. Newman. Power-law distributions in empirical data. *SIAM review*, 51(4):661–703, 2009.
- [31] T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein. *Introduction to algorithms*. The MIT press, 2001.
- [32] M. Costanzo, A. Baryshnikova, J. Bellay, Y. Kim, E.D. Spear, C.S. Sevier, H. Ding, J.L.Y. Koh, K. Toufighi, S. Mostafavi, et al. The Genetic Landscape of a Cell. *Science*, 327(5964):425, 2010.
- [33] M. Datar, N. Immorlica, P. Indyk, and V.S. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *SOCG*, pages 253–262. ACM, 2004.
- [34] J. Davis, B. Kulis, P. Jain, S. Sra, and I. Dhillon. Information-theoretic metric learning. In *Proc. 24th International Conference on Machine Learning, ICML '07*.
- [35] J. Dean and S. Ghemawat. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.
- [36] I. S. Dhillon, Y. Guan, and B. Kulis. Weighted Graph Cuts without Eigenvectors: A Multilevel Approach. *IEEE Trans. Pattern Anal. Mach. Intell.*, 29(11):1944–1957, 2007.
- [37] I.S. Dhillon, Y. Guan, and J. Kogan. Iterative clustering of high dimensional text data augmented by local search. In *Data Mining, 2002. ICDM 2002. Proceedings. 2002 IEEE International Conference on*, pages 131–138. IEEE, 2002.

- [38] I.S. Dhillon and D.S. Modha. Concept decompositions for large sparse text data using clustering. *Machine Learning*, 42(1):143–175, 2001.
- [39] A. R. Didonato and A. H. Morris, Jr. Algorithm 708: Significant digit computation of the incomplete beta function ratios. *ACM Trans. Math. Softw.*, 18, 1992.
- [40] C. Ding, X. He, P. Husbands, H. Zha, and H. Simon. Pagerank, hits and a unified framework for link analysis. In *SIAM Conference on Data Mining*, 2003.
- [41] S. Van Dongen. *Graph Clustering by Flow Simulation*. PhD thesis, University of Utrecht, 2000.
- [42] T. Elsayed, J. Lin, and D. Metzler. When close enough is good enough: Approximate positional indexes for efficient ranked retrieval. In *CIKM*, 2011.
- [43] C. Faloutsos, K. S. McCurley, and A. Tomkins. Fast discovery of connection subgraphs. In *KDD '04*, pages 118–127, New York, NY, USA, 2004. ACM.
- [44] M. Faloutsos, P. Faloutsos, and C. Faloutsos. On power-law relationships of the internet topology. In *Proceedings of the conference on Applications, technologies, architectures, and protocols for computer communication*. ACM, 1999.
- [45] M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.
- [46] S. Fields and O. Song. A novel genetic system to detect protein protein interactions author=Fields, S. and Song, O., journal=Nature,. *Nature*, 340(6230):245–246, 1989.
- [47] G.W. Flake, S. Lawrence, and C.L. Giles. Efficient identification of web communities. In *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 150–160. ACM, 2000.
- [48] S. Fortunato. Community detection in graphs. *Physics Reports*, 486:75–174, 2010.
- [49] J. Friedman, T. Hastie, and R. Tibshirani. *The elements of statistical learning*, volume 1. Springer Series in Statistics, 2001.
- [50] M.R. Garey and L. Johnson. Some simplified NP-complete graph problems. *Theoretical computer science*, 1(3):237–267, 1976.
- [51] D. Gibson, R. Kumar, and A. Tomkins. Discovering large dense subgraphs in massive graphs. In *VLDB '05*, pages 721–732, 2005.

- [52] A. Gionis, P. Indyk, and R. Motwani. Similarity search in high dimensions via hashing. In *VLDB*, 1999.
- [53] JB Glattfelder and S. Battiston. Backbone of complex networks of corporations: The flow of control. *Phys. Rev. E*, 80(3):36104, 2009.
- [54] D. Gleich. Hierarchical Directed Spectral Graph Partitioning. 2006.
- [55] S. Gregory. An algorithm to find overlapping community structure in networks. *Lecture Notes in Computer Science*, 4702:91, 2007.
- [56] M. Henzinger. Finding near-duplicate web pages: a large-scale evaluation of algorithms. In *SIGIR*, 2006.
- [57] J. Huang, D. Abadi, and K. Ren. Scalable sparql querying of large rdf graphs. *PVLDB*, 4(11), 2011.
- [58] J. Huang, T. Zhu, and D. Schuurmans. Web communities identification from random walks. *Lecture Notes in Computer Science*, 4213:187, 2006.
- [59] P. Indyk and R. Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *STOC*, 1998.
- [60] P. Jain, B. Kulis, and K. Grauman. Fast image search for learned metrics. In *IEEE CVPR*, 2008.
- [61] R. Kannan, S. Vempala, and A. Veta. On clusterings-good, bad and spectral. In *FOCS '00*, page 367. IEEE Computer Society, 2000.
- [62] D. R. Karger. Random sampling in cut, flow, and network design problems. In *STOC '94*, pages 648–657, 1994.
- [63] G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20, 1999.
- [64] B. Kernighan and S. Lin. An Efficient Heuristic Procedure for partitioning graphs. *The Bell System Technical J.*, 49, 1970.
- [65] M. Kessler. Bibliographic coupling between scientific papers. *American Documentation*, 14:10–25, 1963.
- [66] V. Krishnamurthy, M. Faloutsos, M. Chrobak, L. Lao, J.H. Cui, and AG Percus. Reducing large internet topologies for faster simulations. *NETWORKING '05*, pages 328–341, 2005.

- [67] B. Kulis and K. Grauman. Kernelized locality-sensitive hashing for scalable image search. In *Computer Vision, 2009 IEEE 12th International Conference on*, pages 2130–2137. Ieee, 2009.
- [68] V. Kumar, A. Grama, A. Gupta, and G. Karypis. *Introduction to parallel computing: design and analysis of algorithms*, volume 400. Benjamin/Cummings, 1994.
- [69] H. Kwak, C. Lee, H. Park, and S. Moon. What is Twitter, a social network or a news media? In *Proceedings of the 19th international conference on World wide web*, pages 591–600. ACM, 2010.
- [70] A. Lancichinetti, S. Fortunato, and F. Radicchi. Benchmark graphs for testing community detection algorithms. *Physical Review E*, 78(4):46110, 2008.
- [71] K. Lang. Fixing two weaknesses of the spectral method. In *NIPS*, 2005.
- [72] K. Lang and S. Rao. A flow-based method for improving the expansion or conductance of graph cuts. *Lecture notes in computer science*, pages 325–337, 2004.
- [73] J. Leskovec and C. Faloutsos. Sampling from large graphs. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, page 636. ACM, 2006.
- [74] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney. Statistical properties of community structure in large social and information networks. In *WWW '08*, pages 695–704, New York, NY, USA, 2008. ACM.
- [75] J. Leskovec, K. J. Lang, Anirban Dasgupta, and M. W. Mahoney. Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters. *CoRR*, abs/0810.1355, 2008.
- [76] D.D. Lewis, Y. Yang, T.G. Rose, and F. Li. Rcv1: A new benchmark collection for text categorization research. *JMLR*, 5:361–397, 2004.
- [77] L. Li, C. J. Stoeckert, and D. S. Roos. OrthoMCL: identification of ortholog groups for eukaryotic genomes. *Genome Res*, 13(9):2178–2189, September 2003.
- [78] P. Li and C. König. b-bit minwise hashing. In *WWW*, 2010.
- [79] D. Liben-Nowell and J. Kleinberg. The link-prediction problem for social networks. *J. Am. Soc. Inf. Sci. Technol.*, 58:1019–1031, May 2007.
- [80] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and Kai Li. Multi-probe lsh: efficient indexing for high-dimensional similarity search. In *VLDB*, pages 950–961, 2007.

- [81] S.A. Macskassy and F. Provost. Classification in networked data: A toolkit and a univariate case study. *The Journal of Machine Learning Research*, 8:935–983, 2007.
- [82] A. S. Maiya and T. Y. Berger-Wolf. Sampling community structure. In *WWW '10*, pages 701–710, 2010.
- [83] G. Malewicz, M.H. Austern, A.J.C. Bik, J.C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 international conference on Management of data*, pages 135–146. ACM, 2010.
- [84] G. S. Manku, A. Jain, and A. Das Sarma. Detecting near-duplicates for web crawling. In *WWW*, 2007.
- [85] C.D. Manning, P. Raghavan, and H. Schütze. *An introduction to information retrieval*. 2008.
- [86] M. Meila and W. Pentney. Clustering by Weighted Cuts in Directed Graphs. In *SDM*, 2007.
- [87] M. Meila and J. Shi. A random walks view of spectral segmentation. In *Artificial Intelligence and Statistics AISTATS*, 2001.
- [88] A. Mislove, Massimiliano Marcon, Krishna P. Gummadi, Peter Druschel, and Bobby Bhattacharjee. Measurement and Analysis of Online Social Networks. In *Proceedings of the 5th ACM/Usenix Internet Measurement Conference (IMC'07)*, San Diego, CA, October 2007.
- [89] M. E. J. Newman and M. Girvan. Finding and evaluating community structure in networks. *Phys. Rev. E*, 69(2):026113, Feb 2004.
- [90] M.E.J. Newman. Fast algorithm for detecting community structure in networks. *Physical Review E*, 69(6):066133, 2004.
- [91] A.Y. Ng, M.I. Jordan, and Y. Weiss. On spectral clustering: Analysis and an algorithm. *Advances in neural information processing systems*, 2:849–856, 2002.
- [92] G. Palla, I. Derényi, I. Farkas, and T. Vicsek. Uncovering the overlapping community structure of complex networks in nature and society. *Arxiv preprint physics/0506133*, 2005.
- [93] F. Provost and V. Kolluri. A survey of methods for scaling up inductive algorithms. *DMKD*, 3(2):131–169, 1999.

- [94] S. Pu, J. Wong, B. Turner, E. Cho, and S.J. Wodak. Up-to-date catalogues of yeast protein complexes. *Nucleic acids research*, 2008.
- [95] D. Ravichandran, P. Pantel, and E. Hovy. Randomized algorithms and nlp: using locality sensitive hash function for high speed noun clustering. In *ACL*, 2005.
- [96] A. Ruepp, B. Brauner, I. Dunger-Kaltenbach, G. Frishman, C. Montrone, M. Stransky, B. Waagele, T. Schmidt, O.N. Doudieu, V. Stumpflen, et al. CO-RUM: the comprehensive resource of mammalian protein complexes. *Nucleic Acids Research*.
- [97] R. Sabry, M. George, and D. Ian. iRefIndex: A consolidated protein interaction database with provenance. *BMC Bioinformatics*, 9.
- [98] V. Satuluri and S. Parthasarathy. Scalable graph clustering using stochastic flows: applications to community discovery. In *KDD '09*, pages 737–746, New York, NY, USA, 2009. ACM.
- [99] V. Satuluri, S. Parthasarathy, and Y. Ruan. Local graph sparsification for scalable clustering. Technical Report OSU-CISRC-11/10-TR25, The Ohio State University.
- [100] V. Satuluri, S. Parthasarathy, and Y. Ruan. Local graph sparsification for scalable clustering. In *SIGMOD*, 2011.
- [101] B. Savas and I. Dhillon. Clustered low rank approximation of graphs in information science applications. In *SDM*, pages 164–175, 2011.
- [102] S.E. Schaeffer. Graph clustering. *Computer Science Review*, 1(1):27–64, 2007.
- [103] R. Sharan, I. Ulitsky, and R. Shamir. Network-based prediction of protein function. *Molecular Systems Biology*, 3, 2007.
- [104] J. Shi and J. Malik. Normalized Cuts and Image Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2000.
- [105] H. Small. Co-citation in the scientific literature: A new measure of the relationship between documents. *Journal of the American Society for Information Science*, 24:265–269, 1973.
- [106] E. Spertus, M. Sahami, and O. Buyukkokten. Evaluating similarity measures: a large-scale study in the orkut social network. In *Proceedings of the eleventh ACM SIGKDD international conference on Knowledge discovery in data mining*, KDD '05, pages 678–684, New York, NY, USA, 2005. ACM.

- [107] D. A. Spielman and N. Srivastava. Graph sparsification by effective resistances. In *STOC '08: Proceedings of the 40th annual ACM symposium on Theory of computing*, pages 563–568, New York, NY, USA, 2008. ACM.
- [108] D. A. Spielman and S.H. Teng. Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems. In *Proceedings of the thirty-sixth annual ACM symposium on Theory of computing*, pages 81–90. ACM New York, NY, USA, 2004.
- [109] C. Stark, B.J. Breitkreutz, T. Reguly, L. Boucher, A. Breitkreutz, and M. Tyers. BioGRID: a general repository for interaction datasets. *Nucleic acids research*, 34(Database Issue):D535, 2006.
- [110] A. Strehl and J. Ghosh. Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *The Journal of Machine Learning Research*, 3:583–617, 2003.
- [111] Y. Tao, K Yi, C. Sheng, and P. Kalnis. Quality and efficiency in high dimensional nearest neighbor search. In *SIGMOD*, 2009.
- [112] S.H. Teng. Coarsening, sampling, and smoothing: Elements of the multilevel method. *Algorithms for Parallel Processing*, 105:247–276, 1999.
- [113] The Gene Ontology Consortium. Gene Ontology: tool for the unification of biology. *Nature Genetics*, 25:25–29, 2000.
- [114] Koji Tsuda. Propagating distributions on a hypergraph by dual information regularization. In *ICML*, pages 920–927, 2005.
- [115] M. Tumminello, T. Aste, T. Di Matteo, and RN Mantegna. A tool for filtering information in complex systems. *PNAS*, 102(30):10421, 2005.
- [116] P. Uetz, L. Giot, G. Cagney, T.A. Mansfield, R.S. Judson, J.R. Knight, V. Lockshon, D. a nd Narayan, M. Srinivasan, P. Pochart, et al. A comprehensive analysis of protein–protein interactions in *Saccharomyces cerevisiae*. *Nature*, 403(6770):623–627, 2000.
- [117] S. Van Dongen. Graph clustering via a discrete uncoupling process. *SIAM Journal on Matrix Analysis and Applications*, 30:121, 2008.
- [118] J. Vlasblom and S.J. Wodak. Markov clustering versus affinity propagation for the partitioning of protein interaction graphs. *BMC bioinformatics*, 10(1):99, 2009.
- [119] U. Von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007.

- [120] I. Xenarios, L. Salwinski, X.J. Duan, P. Higney, S.M. Kim, and D. Eisenberg. DIP, the Database of Interacting Proteins: a research tool for studying cellular networks of protein interactions. *Nucleic acids research*, 30(1):303, 2002.
- [121] C. Xiao, W. Wang, X. Lin, J. X. Yu, and G. Wang. Efficient similarity joins for near duplicate detection. *ACM Transactions on Database systems*, 2011.
- [122] C. Xiao, W. Wang, X. Lin, and J.X. Yu. Efficient similarity joins for near duplicate detection. In *WWW*, 2008.
- [123] X. Yang, S. Parthasarathy, and P. Sadayappan. Fast sparse matrix-vector multiplication on gpus: Implications for graph mining. *PVLDB*, 4(4), 2011.
- [124] W.W. Zachary. An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4):452–473, 1977.
- [125] J. Zhai, Y. Lou, and J. Gehrke. Atlas: a probabilistic algorithm for high dimensional similarity search. In *SIGMOD*, 2011.
- [126] D. Zhou, J. Huang, and B. Schölkopf. Learning from labeled and unlabeled data on a directed graph. In *ICML '05*, pages 1036–1043, 2005.
- [127] D. Zhou, B. Scholkopf, and T. Hofmann. Semi-supervised learning on directed graphs. *Advances in neural information processing systems*, 17:1633–1640, 2005.
- [128] X. Zhu and A.B. Goldberg. Introduction to semi-supervised learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 3(1):1–130, 2009.